# Why Learn Data Structures and Algorithms?

If you want to become a successful software developer or software architect, then it is mandatory for you to very good knowledge of Data structures and Algorithms. The reason for this is, Data Structure and Algorithms are considered as the base to develop good software.

The data structure is a core subject for programmers or IT professionals. So for computer science graduates and undergraduates who are pursuing the engineering course, they have this as a subject. So they have to study as a part of academics and the second thing, in the industry, if you're working as a programmer then you have to use data structures in your application development. Without data structure, you cannot develop applications at all. So you should have a good understanding of data structure.

# Data Structures and Algorithms?

A data structure is a particular way to organize and store data in memory of a computer in such a way that the operations on data can be performed efficiently.
A data structure does not only stores data but it also represents some relationship between data.

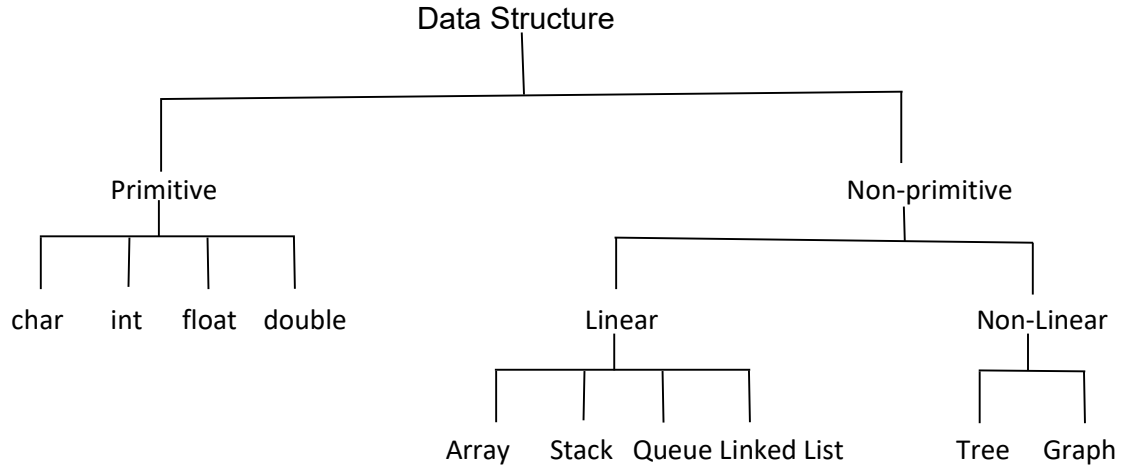
An algorithm is a set of one–by–one instruction to solve a particular problem.

## Advantages of Data Structures?

We need data structures, a few of them are as follows:

1. They are essential ingredients in creating a fast and powerful algorithm.
2. They help to manage and organize data.
3. They make code cleaner and easier to understand.

## Classification of Data Structure



## Choice of right Data Structure

An important step of problem solving is to select the data structure wherein the data will be stored. The
program or algorithm would then work on this data structure and give the results.
A data structure can either be selected from the *built-in* data structures offered by the language or by designing a new one. Built-in data structures in 'C' are arrays, structures, unions, files, pointers, etc.
Let us consider a situation where it is required to create a merit list of 60 students. These students appeared in a class examination. The data of a student consists of his *roll number*, name, *marks*, *percentage*,
and *grade*. Out of many possibilities, the following two choices of data structures are worth considering:
**Choice I:** The various data items related to a student are: name, roll number, marks, percentage, and grade. The total number of students is 60. Therefore, the simplest choice would be to have as many parallel lists as there are data items to be represented.

An equivalent declaration for data structure is given below:

```
int roll[60];
char names[60][15];
int marks[60];
float percent[60];
char grade[60];
```

Now in the above declaration, every list has to be considered as individual. The generation of merit list would require the list to be sorted. The sorting process would involve a large number of exchange operations. For each list, separate exchange operations would have to be written. For example, the exchange between data of *i*th and *j*th student would require the following operations:

```
char tempName[15],chTemp;
int i,j, temp;
float fTemp;

temp = roll[i];
roll[i] = roll[j];
roll[j] = temp;
strcpy (tempName, names[i]);
strcpy (names[i], names[j]);
strcpy (names[j], tempName);
temp = marks[i];
marks[i] = marks[j];
marks[j] = temp;
fTemp = percent[i];
percent[i] = percent[j];
percent[j] = fTemp;
chTemp = grade[i];
grade[i] = grade[j];
grade[j] = chTemp;
```

Though the above code may work, but it is not only a clumsy way of representing the data but would

require a lot of typing effort also. In fact, as many as ***fifteen statements*** are required to exchange the data
of *i*th student with that of *j*th student.

**Choice II:** From the built-in data structures, the `struct` can be selected to represent the data of a student as shown below:

```
struct student {
int roll;
char name[15];
int marks;
float percent;
char grade;
};
struct student studList [60];
```

Now in this case, exchange between data of *i*th and
*j*th student would require the following operations:

```
int i,j;
struct student temp;
temp = studList[i];
studList [i] = studList [j];
studList [j] = temp;
```

Thus, only ***three statements*** are required to exchange the data of *i*th student with that of *j*th student.

A comparison of the two choices discussed above establishes that Choice II is definitely better than Choice I as far as the selection of data structures for the given problem is concerned. The number of lines
of code of Choice II is definitely less than the Choice I. Moreover, the code of Choice II is comparatively
easy and elegant.

From the above discussion, it can be concluded that choice of right data structures is of paramount importance from the point of view of representing the data belonging to a problem.

## Operations on Data Structure

The following operations can be performed on data structure.

### Traversal

Accessing each element of a data structure exactly once so that these elements can be processed.

### Insertion

Adding a new element to data structure

### Deletion
Removing an existing element from data structure

### Searching
Finding the location of an element with some given key value

### Sorting
Arranging the elements of a data structure in either ascending or descending order

### Merging
Combining the elements of two similar data structure into a third data structure.

## Algorithm

An algorithm is the step by step instructions to solve a given problem .

## Analysis of algorithm

There can be more than one algorithm for solving a particular problem. Analysis of an algorithm means measuring the efficiency/complexity of an algorithm in terms of time & space so that we can choose the best algorithm.

Based on the space & time taken by an algorithm, complexity is of 2 types-

1. Space complexity
   It means how much memory is taken by an algorithm to solve a particular problem of size n.
   Space complexity of an algorithm is measured by counting the maximum memory needed by an algorithm.
2. Time complexity
   It means how much time is taken by an algorithm to solve a particular problem of size n.
   Time complexity of an algorithm is measured by counting the no. of key(basic) operations.
   For example, In searching and sorting algorithm, the basic operation is the no. of comparisions.

   The space and time complexity of an algorithm is represented by function f(n) where n is the size of problem(input size).

## What is rate of growth?

The rate at which the space/running time complexity f(n) of an algorithm increases as n increases is called rate of growth.

## Commonly used rate of growth

| Rate of growth | Name |
|---|---|
| 1 | Constant |
| n | Linear |
| logn | Logarithmic |
| nlogn | Linear Logarithmic |
| $n^2$ | Quadratic |
| $n^3$ | Cubic |
| $2^n$ | Exponential |

## Types of analysis

There are 3 types of analysis-

Worst case
Define the input for which the algorithm runs the slower.

Best case
Define the input for which the algorithm runs the fastest.

Average case

- Provides a prediction of running time of the algorithm.
- Assume that the input is random.
  Lower bound<=Average time<=upper bound

## Calculation of time complexity

Let f(n) be the actual running time and g(n) be the approximate running time.

**a)**
```
for(i = 1;i<=n;i++)
{
```

```
    printf("Hello\n");
}
```

**b)**

```
sum=0;
for(i=0;i<n;i++)
{
    sum=sum+a[i];
}
```

**c)**

```
for(i = 0;i<n;i++)
{
  for(j=0;j<n;j++)
     {
          c[i][j]=a[i][j]+b[i][j];
     }
}
```

**d)**

```
i=1;
while(i<=n)
{
  printf("A");
  i++;
}
printf("B");
```

**e)**

```
for(i=0;i<=n;i++)
{
    for(j=1;j<=m;j++)
    {
        printf("A");
    }
 printf("B");
}
```

**f)**
```
a=a+1
for(i=1;i<=n;i++)
   m=m+2;
for(i=1;i<=n;i++)
{
   for(j=1;j<=n;j++)
     k=k+1;
}
```

**g)**
```
for(i=0;i<=n;i++)
{
    If(a[i]<a[i+1])
    {
          printf("A");
          printf("B");
    }
    else
    {
      printf("C");
      printf("D");
      printf("E");
    }
}
```

**h)**
```
i=1;
while(i<=n)
{
  j=0;
  while(j<=m)
   {
     printf("A");
     j++;
   }
   printf("B");
   i++;
}
printf("C");
```

**i)**
```
for(i=1;i<=n;i++)
```

```
{
    for(j=i;j<=n-1;j++)
     {
            printf("A");
     }
     printf("B");
}
printf("C");
```

**j)**
```
for(i=1;i<=n;)
   i=i*2;
```

**k)**
```
for(i=n;i>=1;)
i=i/2;
```

## Asymptotic Analysis

Asymptotic analysis of an algorithm refers to the rate at which the required space or running time grows as the problem size grows.

For asymptotic analysis of an algorithm, asymptotic notations (shorthand way to represent space or time complexity) can be used.

## Asymptotic Notations

### O(Big-oh) notation

It provides asymptotic upper bound for a given function.

Let f(n) and g(n) be two functions.

If f(n)=O(g(n)) then there exists two positive integers/real no. constants c and $n_0$ such that

$0 \leq f(n) \leq cg(n)$ for every $n \geq n_0$

### O(Big-omega) notation

It provides asymptotic lower bound for a given function.

Let f(n) and g(n) be two functions.

If $f(n)=\Omega(g(n))$ then there exists two positive integers/real no. constants c and $n_0$ such that

$f(n)\geq cg(n)\geq 0$ for every $n\geq n_0$

## θ(Theta) notation

It provides asymptotic lower bound and asymptotic upper bound for a given function.

Let f(n) and g(n) be two functions.

If $f(n)=\theta(g(n))$ then there exists two positive constants c1,c2 and $n_0$ such that

$0\leq c1g(n)\leq f(n)\leq c2g(n)\geq 0$ for every $n\geq n_0$

**Q. Find O(Big-oh) notation for the functions given below-**

a) $f(n)=3n+5$
b) $f(n)=10n^2+7$
c) $f(n)=6n^2+4n+5$
d) $f(n)=2^n+6n^2+3n$
e) $f(n)=3n-8$
f) $f(n)=8n^2+3n-8$
g) $f(n)=2^n+3n^2-8n+6$

**Q. Find Ω(Big-omega) notation for the functions given below-**

a) $f(n)=3n+8$
b) $f(n)=3n-8$
c) $f(n)=6n^2+4n-5$
d) $f(n)=4*2^n+3n$

**Q. Find θ(Theta) notation for the functions given below-**

**a)** $f(n)=27n^2+16n+25$

**b)** $f(n)=2^n+6n^2+3n$

**c)** $f(n)=n/2(n-1)$

## Master Method

The master method solves recurrences of the following form-

$T(n) = aT(n/b)+f(n)$

The above recurrence gives the running time of an algorithm that divides the problem of size n into a subproblems(each of size n/b) where a and b are positive constatnts.

The a subproblems are solved recursively each of which takes T(n/b) time.

f(n) describes the cost of dividing the problem and combining the results of subproblems.

## Master Theorem

Let a≥1 and b>1 be two constants.

Let f(n) be a function and let T(n) be defined on the non-negative integers by the recurrence-

T(n)=aT(n/b)+f(n)

Then T(n) can be bounded asymptotically as-

a) if $f(n) = \theta(n^{\log_b a})$ then $T(n) = \theta(n^{\log_b a}\lg n)$
b) if $f(n) = O(n^{\log_b a-\varepsilon})$ for some constant $\varepsilon>0$ then $T(n) = \theta(n^{\log_b a})$
c) if $f(n) = \Omega(n^{\log_b a+\varepsilon})$ for some constant $\varepsilon>0$ and if $af(n/b)\leq cf(n)$ for some constant $c<1$ and all sufficiently large n then $T(n) = \theta(f(n))$

**Q. Find the time complexity for the recurrences given below-**

a) $T(n) = 4T(n/2)+n^2$

b) $T(n) = 7T(n/2)+n^2$

c) $T(n) = 2T(n/2)+n^3$

## Problems for which master theorem is not applicable

(a) $T(n) = 2^nT(n/2)+n^n$
(b) $T(n) = 0.5T(n/2)+1/n$
(c) $T(n) = 64T(n/8)-n^2\log n$

## Recursion Tree Method

In recursion tree method, we consider the 2nd term as the root and at each level nodes are expanded.

**Q. Find the time complexity for the recurrences given below-**

a) $T(n) = 2T(n/2)+n^2$

b) $T(n) = T(n/3)+T(2n/3)+n$

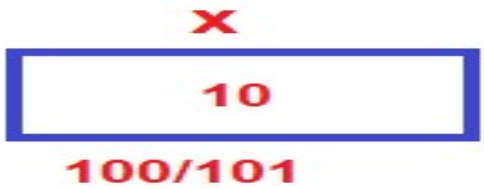
# Array

## Array Declaration and Initialization:-

Before understanding the array, let us first understand simple variables. Variables are supported by every programming language and the variables will have the same data type. For example, if we want to declare a variable of type integer, whose name is x and in this, if we want to store the value 10 then we need to write something like below in c language.

```
int x = 10;
```

Then we know very well that at runtime variable x will get some memory. If we assume that integer takes two bytes of memory, then two bytes of memory will be allocated for x. Let say the 1st-byte address is 100 and the next byte address is 101 and together in these two bytes the value 10 will be stored as shown in the below image. This type of variable is a single value variable i.e. they just store one single value.




The single value variable is also called a **scalar variable.** Then what is an array?

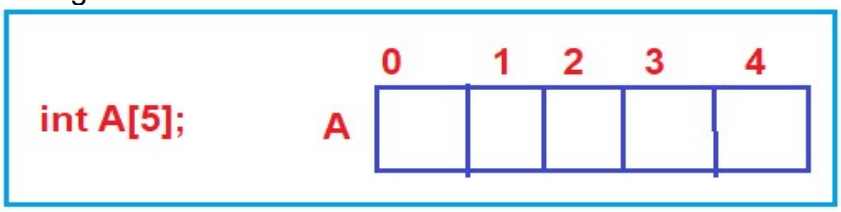
## What is an Array?

In the array variable, we can store multiple values that are a list of values or a set of values. So, we can define an array as a collection of elements and all the elements are of the same type. In simple words, the array is a collection of similar data elements grouped under one name.

Suppose instead of 1 value, if we want to store a list of values, then we can declare an array. Let us say we want to create an integer type array; whose name is A and the size is 5 and in C language the following line of code does the same.

**int A[5];**

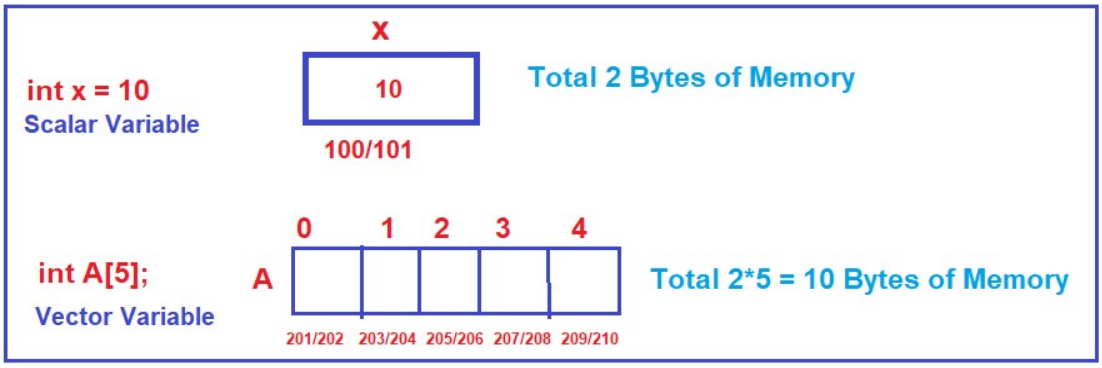
In this array, we can store 5 integers and the indices will be 0 1 2 3 4 as shown in the below image. And here we can store 5 values.




The array variable is also called **Vector variables** i.e. having some dimension.

## Difference between Scalar and Vector Variable:-

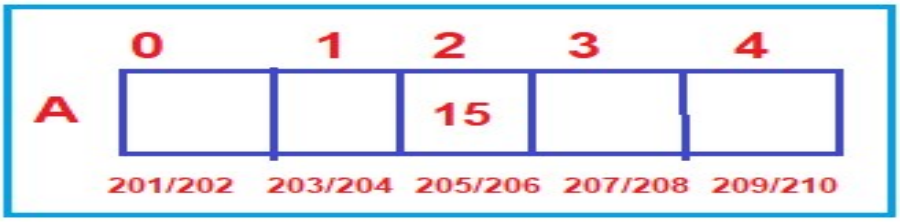
In the case of scalar variable i.e. x, we can store a single value in it and the memory is 2 bytes. On the other hand, A is a vector variable and in this variable, we can store 5 integer values and the memory it will occupy is 10 bytes (we are considering integer takes 2 bytes). For example, the first-byte address is 201 and then 202, 203, up to 210. The most important point that you need to remember is that the memory will be allocated continuously.




All the five integers will be having the same name i.e. A. Then how we can differentiate all those five integers. We can differentiate them with the index which will start from 0 up to 1. So, A[0] means the first integer, A[1] means the second integer and A[4] means the fifth integer. So, we can access them or differentiate them through indexes.

Suppose we want to store a value 15 at index 2 then we can do the same using the below line of code in most programming languages.

**A[2]=15;**
Once you write the above code, then it will store 15 in the index position 2 on the array as shown below.




Ordinary variable is known as Scalar variable where as Array is known as Vector variable.

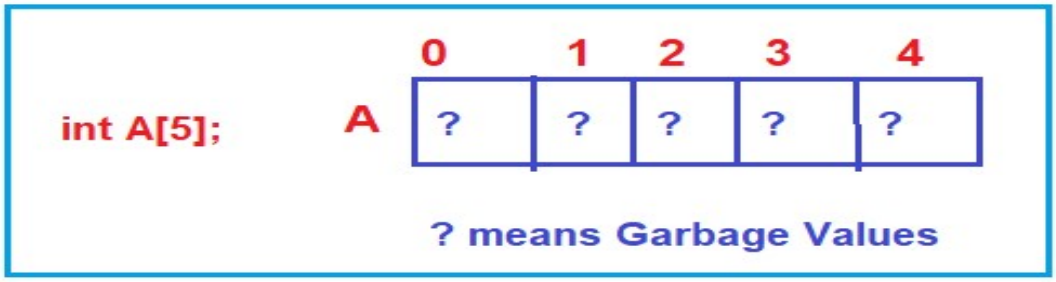
So, using the name and the index we can access any of those integers or any of those elements.
The array is a collection of elements or a list of elements that are of the same type and this concept is supported by every programming language as a basic feature. With this keep in mind let us proceed and understand what are the different ways to declare an array.

## Declarations and Initialization of Array:-

### First method:-

Declare an array of some size without Initializing. As you can see in the below image, we are declaring an integer array with the name A of size 5, then this will allocate the memory space for 5 integers starting from indexes 0 to 4. As we have

not initialized the array with any values, so by default it will store garbage values.

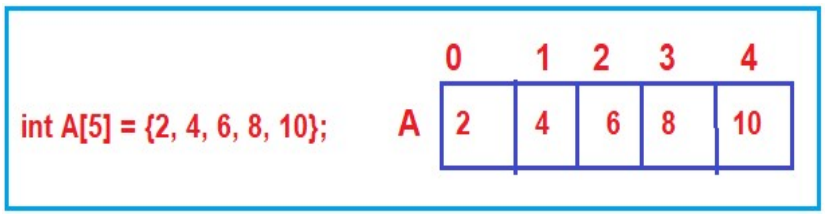



**Note:-**

When the array is not initialized with any values then by default it will store garbage or unknown or random values in it.

**Second method:-**

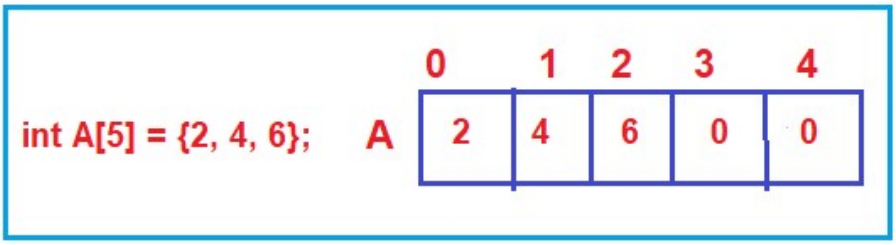
We will declare an array of some size and also, initialize with values. As you can see in the below image, we created an array with the name A and size 5 as well as we initialize the array with five elements. In this case, the array will be created during the runtime and all the values will be initialized at that time.




**Note:** The above example is the declaration + initialization

**Third method:**

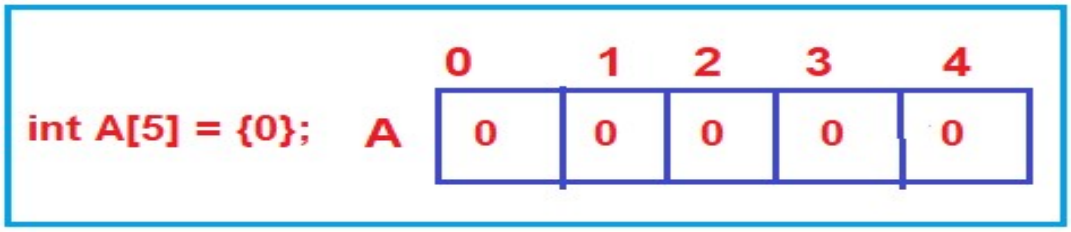
We will declare an array of some size but we will initialize only a few values. As you can see in the below image, we have created an array with the name A and size 5 and initialize it only with three elements. In this case, the rest values will be initialized with 0 by default.




Here, the array is initialized with three values, so the first three indexes are initialized with the values 2,4, and 6, and the last two index positions are initialized with 0. So, the point that you need to remember is, once the initialization process starts it will try to initialize all the elements and if values are not present then it will initialize with 0.

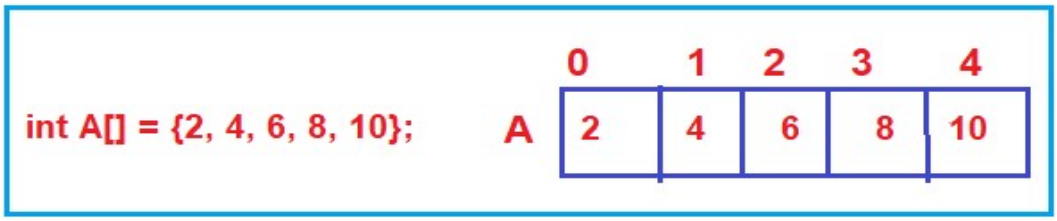
**Fourth method:-**

We will declare an array of some size and we will assign only 0 as the first element, the rest of the elements are also filled with 0. As you observe in the below image, array A is created with size 5 and initialized with only one value i.e. 0 and in this case, the rest four indexes are also initialized with 0.




**Fifth Method:**

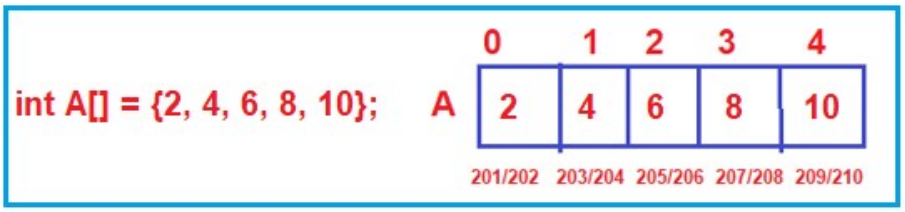
We will not mention any size but initialize with elements. As you can see in the below image, we created an array with the name A without mentioning the size but initializing the array with five elements. Depending on the number of elements we have mentioned in the initialization list, the array size will be the same as that one. So, an array with size 5 will be created in this case and it will be initialized with all those values.




So, these are the various ways of declaration and initialization of an array. This is there in both C and C++. Now I hope you understand how to declare and initialize an array. Let us proceed further and understand how to access the elements of an array.

## Accessing Array Elements:

Let say we have an integer array of size 5 and this array is initialized with five integers as shown in the below image.




Now, how to access those elements. Suppose we want to print the value that is present at index 2, then we can do the same using A[2] as shown below,

**printf("%d", A[2]);**

Suppose we want to print the value that is present at index 3, then we can do the same using A[3] as shown below,

**printf("%d", A[3]);**

**Note:-**

The point that you need to remember is, we can access all the array elements just by changing the index.

If we have to traverse through an array (traversing means visiting all elements once), then we can take the help of a loop which will help us to change the indexes from 0 to 4 and that will allow us to access all the elements by traversing through those elements as shown below. 'for' loop is used for accessing the elements in an array.

Ex:-

```
int A[5] = {2, 4, 6, 8, 10};
for(i=0;i<5;i++)
{
    printf("%d", A[i]);
}
```

**Note:-**

for loop is used for traversing through all the elements of an array. And the elements can be accessed with the name and index of an array.

Next let us see what are the different ways of accessing a particular element in an array,

1. **printf("%d", A[2]);**
   It will print the element which is present at index position 2.

2. **printf("%d", 2[A]);**
   We can also access the element using index outside and the name of an array inside subscript.

3. **printf("%d", *(A+2));**
   We can also access the element using pointer arithmetic.

**Note:-**

The array element can be accessed by using the subscript [] that is an index as well as they can be accessed with the help of pointer arithmetically.

## Demonstration of Array Declaration and Initialization:-

Let us see some programming examples to understand the Declaration and Initialization.

**Ex:**

In the below example, we have declared and initialize five different arrays.

1. In array A, we have only declared but not initialized, in this case, it will store some garbage values and that what you can see when you print the elements (Sometimes you may see the values are 0, but they are also garbage).
2. In array B, we have declared and initialized the array with five elements and those elements you can when you traverse the array (traverse means accessing each element).
3. In array C, we have mentioned the size as 5, but initialized with 2 elements, in this case when you access the array elements, the first two values you will get as expected but the last three values you will get the default values based on the data type. As the data type, we used here is int, so you will get the default value of int i.e. 0.
4. In array D, we declared the array size as 5 but initialize the array only with one element and that is too with the value 0. In this case, the rest four elements are also initialized with 0.
5. In array E, we have not mentioned the array size but initialized with five elements. In this case size of the array will same as the number of elements we initialized i.e. 5. When we access the array, we will get the elements as expected.

Ex:-

```
#include <stdio.h>
int main()
{
    //Declare array of some size without Initialization
        int A[5];
        printf(" A Array Elements : ");
        for(int i = 0; i <5; i++)
        {
                printf("%d ",A[i]);
        }

        //Declare an array of  size 5 and also Initialize the array with 5 values
            int B[5]={1, 2, 3, 4, 5};
            printf("\n B Array Elements : ");
            for(int i = 0; i <5; i++)
            {
                printf("%d ",B[i]);
            }

            //Declare an array of size 5 but Initialize only few values
            int C[5]={2, 4};
            printf("\n C Array Elements : ");
            for(int i = 0; i <5; i++)
            {
```

```
                printf("%d ",C[i]);
            }

            //Declare an array of size 5 and assign only 0 as the first element
            int D[5]={0};
            printf("\n D Array Elements : ");
            for(int i = 0; i <5; i++)
            {
                printf("%d ",D[i]);
            }

        // We will not mention any size but will initialize the array with some
elements
            int E[]={1,2,3,4,5,6};
            printf("\n E Array Elements : ");
            for(int i = 0; i <5; i++)
            {
                printf("%d ",E[i]);
            }
return 0;

}
```

## Ex:- Array address are in contiguous

As we already discussed earlier, the array stores the elements in the contiguous memory location. Let us see an example and proof that the addresses are contiguous.

```
#include <stdio.h>
int main()
{
    int A[5];
    printf(" Array Addresses: \n");
    for(int i = 0; i <5; i++)
    {
        printf(" %u \n",&A[i]);
    }
return 0;

  }
```

**Output:** As you can see in the below output each index takes 4 bytes and all indexes are contiguous and hence it proofs the array stores the elements in a contiguous memory location.

```
Array Addresses:
3061908784
3061908788
3061908792
3061908796
3061908800
```

## Insertion at specific position in array

## Algorithm

### INS_SP_POS(a,n,value,pos)

**Step 1:** **[Check for position]**

**If pos<0 OR pos>n then**

**Write "Invalid position"**

**Return n**

**[End of If structure]**

**Step 2:** **[Shifting values to the right by 1 position]**

**Repeat for i = n-1 down to pos**

**Set a[i+1] = a[i]**

**[End of loop]**

**Step 3:** **[Insert value at specified position]**

**Set a[pos] = value**

**Step 4:** **[Update no. of values in array]**

**Return n+1**

**/*A PROGRAME TO INSERT A VALUE AT SPECIFIC POSITION IN AN ARRAY*/**

```
#include<stdio.h>
int INS_SP_POS(int a[],int n,int value,int pos)
{
    int i;
    if(pos<0||pos>n)
    {
        printf("\nInvalid position. Value can't be inserted.");
        return n;
    }
    for(i=n-1;i>=pos;i--)
        a[i+1]=a[i];
    a[pos]=value;
    return n+1;
}

void DISPLAY(int a[],int n)
{
    int i;
    for(i=0;i<n;i++)
        printf("%d\n",a[i]);
}

int main()
{
    int a[10],n,value,pos,i;
```

```
    printf("How many values are there?:");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        printf("\nEnter value:");
        scanf("%d",&a[i]);
    }

    printf("\nEnter the value and its position to insert:");
    scanf("%d%d",&value,&pos);
    n=INS_SP_POS(a,n,value,pos);
    printf("\nValues of an array are\n");
    DISPLAY(a,n);

    return 0;
}
```

## Deletion at specific position in array

## Algorithm

### DEL_SP_POS(a,n,pos)

**Step 1:** **[Check for position]**

**If pos<0 OR pos≥n then**

**Write "Invalid position"**

Return n

[End of If structure]

Step 2: [Shifting values to the left by 1 position]

Repeat for i = pos+1 to n-1

Set a[i-1] = a[i]

[End of loop]

Step 4: [Update no. of values in array]

Return n-1

```
/*A PROGRAME TO DELETE A VALUE FROM SPECIFIC POSITION FROM AN
ARRAY*/

#include<stdio.h>


int DEL_SP_POS(int a[],int n,int pos)

{

   int i;

   if(pos<0||pos>=n)

   {

      printf("\nInvalid position.Item can't be deleted.");

      return n;

   }


   for(i=pos+1;i<=n-1;i++)

      a[i-1]=a[i];
```

```
    return n-1;
}
void DISPLAY(int a[],int n)
{
    int i;
    for(i=0;i<n;i++)
        printf("%d\n",a[i]);
}

int main()
{
    int a[10],n,pos,i;

    printf("How many values are there?:");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("\nEnter value:");
        scanf("%d",&a[i]);
    }
    printf("\nEnter the position to delete:");
    scanf("%d",&pos);

    n=DEL_SP_POS(a,n,pos);
    printf("\nValues of an array are\n");
    for(i=0;i<n;i++)
```

```
    printf("%d\n",a[i]);

  return 0;
}
```

# Searching

Searching is the process of finding the key(desired value) within a list of values. Searching is called successful if the key is found otherwise it is called unsuccessful.

The two important and simple searching techniques are shown below:

- Linear search
- Binary search

**a) Linear Search**

This search is also called sequential search since key is compared with every value of an array one by one sequentially.

In other words, we can say that key is compared with first value of an array. If the mismatch occurs then the key is compared with second value of an array and so on. This process is repeated until the key is compared with all values of an array.

## Linear Search in array

## Algorithm

### LINEAR_SEARCH(a,n,key)

**Step 1:** **[Search for value]**

```
        Repeat for i = 0 to n-1
                If key==a[i] then
                        Return i
                [End of If structure]
        [End of loop]
```

**Step 2:** **Return -1**

### Advantages of linear search

- Very simple approach
- Works well for small arrays
- Used to search when the elements are not sorted

### Disadvantages of linear search

- Less efficient if the array size is large
- If the elements are already sorted, linear search is not efficient.

```
/*A PROGRAM  TO SEARCH A VALUE IN AN ARRAY USING LINEAR SEARCH */

#include<stdio.h>

int LINEAR_SEARCH(int a[],int n,int key)

{

   int i;

   for(i=0;i<n;i++)

      {

         if(key==a[i])
```

```
            return i;
        }
    return -1;
}

int main()
{
    int a[10],n,value,pos,i;

    printf("How many values are there?:");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("\nEnter value:");
        scanf("%d",&a[i]);
    }
    printf("\nEnter value to be search:");
    scanf("%d",&value);

    pos=LINEAR_SEARCH(a,n,value);
    if(pos==-1)
        printf("\n%d is not found.",value);
    else
        printf("\n%d is found at position %d.",value,pos);
    return 0;
}
```

**b) Binary Search**

Binary search is an extremely efficient searching algorithm in comparision to linear search since it finds the key(desired value) within a list of values in minimum possible comparisions especially when the list is large enough.

To do binary search, an array must be sorted.

The logic behind binary search is as follows:

a) First find the middle element of array

b) Compare the key with middle element

   There are 3 cases

   1. If the key is the middle element then the search is successful
   2. If the key is less than middle element then search in first half of array
   3. If the key is greater than middle element then search in second half of array

   We repeat the same process until the key is found or search area is exhausted.

   In this, search area is reduced by half each time when the comparision is made.

## Non recursive binary search in an array

## Algorithm

### BINARY_SEARCH(a,n,key)

**Step 1: [Initialize segment variables]**

**Set beg = 0, end = n-1 and mid = int((beg+end)/2)**

**Step 2: Repeat while beg≤end**

```
        [Search successful?]
        If key==a[mid] then
                Return mid
        Else if key<a[mid] then
                Set end = mid-1
        Else
                Set beg = mid+1
        [End of If structure]
```

Set mid = int((beg+end)/2)

[End of loop]

**Step 3:** Return -1

```
/*A PROGRAM  TO SEARCH A VALUE IN AN ARRAY USING BINARY SEARCH */
#include<stdio.h>

int BINARY_SEARCH(int a[],int n,int key)
{
   int beg,end,mid;
   beg=0;
   end=n-1;
   mid=(beg+end)/2;
   while(beg<=end)
   {
      if(key==a[mid])
         return mid;
      else if(key<a[mid])
         end=mid-1;
      else
         beg=mid+1;
      mid=(beg+end)/2;
   }
   return -1;
}
```

```
int main()
{
    int a[10],n,value,pos,i;

    printf("How many values are there?:");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("\nEnter value:");
        scanf("%d",&a[i]);
    }

    printf("\nEnter value to search:");
    scanf("%d",&value);
    pos=BINARY_SEARCH(a,n,value);
    if(pos==-1)
        printf("%d is not found.",value);
    else
        printf("%d is found at position %d.",value,pos);
    return 0;
}
```

## Recursive binary search in an array

## Algorithm

**BINARY_SEARCH(a,beg,end,key)**

**Step 1:** **[Search unsuccessful?]**

**If beg>end then**

**Return -1**

**[End of If structure]**

**Step 2:** **[Finding middle]**

**Set mid = int((beg+end)/2)**

**Step 3:** **If key==a[mid] then**

**Return mid**

**Else if key<a[mid] then**

**Call BINARY_SEARCH(a,beg,mid-1,key)**

**Else**

**Call BINARY_SEARCH(a,mid+1,end,key)**

**[End of If structure]**

**Step 4:** **Return**

```
/*A PROGRAM  TO SEARCH A VALUE IN AN ARRAY RECURSIVELY USING BINARY
SEARCH */

int BINARY_SEARCH(int a[],int beg,int end,int key)

{

   int mid;

   if(beg>end)

      return -1;
```

```
    mid=(beg+end)/2;
    if(key==a[mid])
        return mid;
    else if(key<a[mid])
        BINARY_SEARCH(a,beg,mid-1,key);
    else
        BINARY_SEARCH(a,mid+1,end,key);
}

int main()
{
    int a[10],n,value,pos,i;

    printf("How many values are there?:");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("\nEnter value:");
        scanf("%d",&a[i]);
    }

    printf("\nEnter value to search:");
    scanf("%d",&value);
    pos=BINARY_SEARCH(a,0,n-1,value);
    if(pos==-1)
        printf("%d is not found.",value);
```

```
    else
        printf("%d is found at position %d.",value,pos);
    return 0;
}
```

## Sorting

The process of arranging an array in **ascending** or **descending** order is called sorting.

If the elements are numbers then they can be sorted in ascending or descending order.

In Case the elements are characters or Strings then you can sort them in alphabetical order.

## Bubble Sort

Bubble sort is the simplest sorting algorithm thar sorts an array of of items in multiple passes.

In the first pass, first and second item are compared and swapped(exchanged) if they are in the wrong order. Then the second and third items are compared and swapped if they are in the wrong order.

This process continues until the last two items of an array are compared and swapped if they are in the wrong order.

In the first pass, n-1 comparisions occur. After the first pass, the first largest item is placed at the first last position.

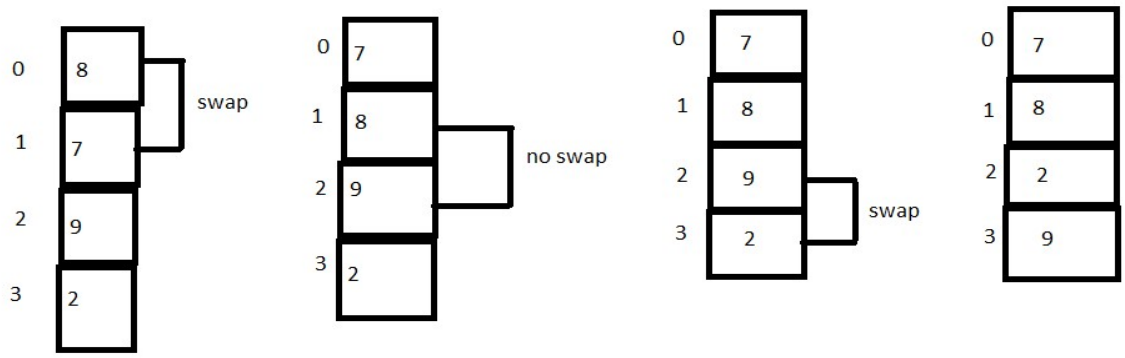
In the second pass, n-2 comparisions occur. After the second pass, the second largest item is placed at the second last position.

After n-1 passes, entire array is sorted and hence no more items are left for comparision.

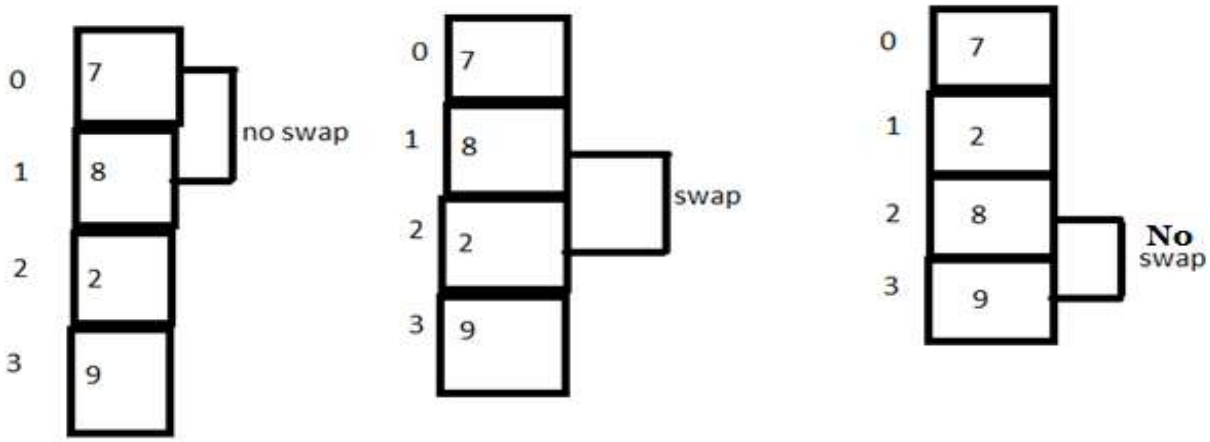
### Implementation of Bubble Sort

**Illustrates how the following numbers, get sorted in ascending order using bubble sort:- {8,7,9,2}**

# First Pass:-

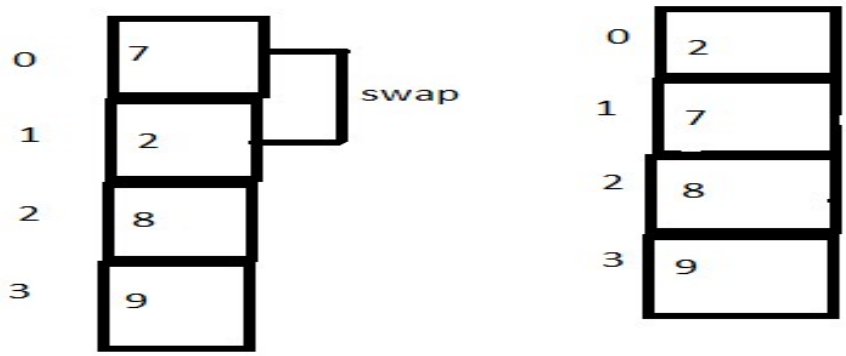



## Second Pass:-




After 2nd pass second largest number 8 bubbles to second last position.

## Third3rd pass:-




**This array is sorted after three passes.**

## Bubble Sort

### Algorithm

**BUBBLE_SORT(a,n)**

**Step 1:** Repeat for i =0 to less than n-1

**Step 2:** Repeat for j= 0 to less than n-1-i

```
            if a[j]>a[j+1]  then
                    Set temp = a[j]
                    Set a[j] = a[j+1]
                    Set a[j+1] = temp
            [End of if structure]
    [End of Step 2 loop]
    [End of Step1 loop]
```

```
/*A PROGRAM TO SORT an ARRAY IN ASCENDING ORDER USING BUBBLE
SORT*/

#include<stdio.h>


void BUBBLE_SORT(int a[],int n)

{

   int i,j,temp;

   for(i=0;i<n-1;i++)

   {

      for(j=0;j<n-1-i;j++)

      {

         if(a[j]>a[j+1])

         {

            temp=a[j];

            a[j]=a[j+1];

            a[j+1]=temp;

         }

      }

   }

}


void DISPLAY(int a[], int n)
```

```
{
   int i;
   for (i=0; i<n; i++)
         printf("%d\t ", a[i]);
}



int main()
{
   int a[15],n,i;

   printf("How many values to sort?:");
   scanf("%d",&n);
   for(i=0;i<n;i++)
   {
      printf("\nEnter value:");
      scanf("%d",&a[i]);
   }

   BUBBLE_SORT(a,n);
   printf("\nAfter sorting,array is\n");
   DISPLAY(a,n);
```

```
    return 0;
}
```

## Selection Sort

If the array is to be arranged in ascending order,the smallest elements need to be selected and exchange/interchange with the first position element in the array. Again, the smallest element from the remaining elements is selected and exchange/interchange with the second array element. The process continues till all elements are arranged.

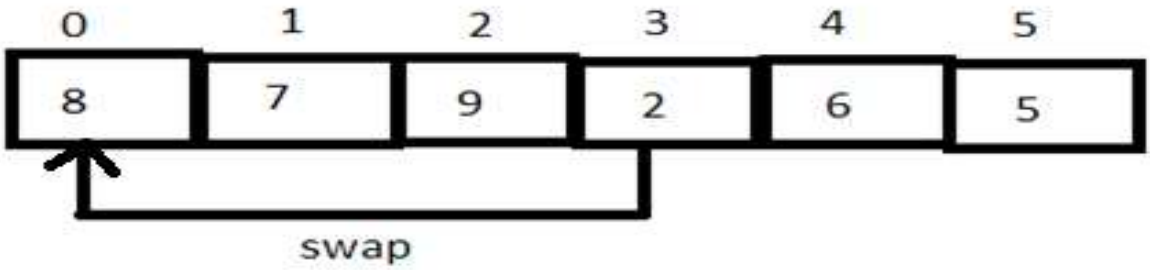
If the array needs to be arranged in descending order, select the largest element.

**Q.** Illustrates how the following array, get sorted in ascending

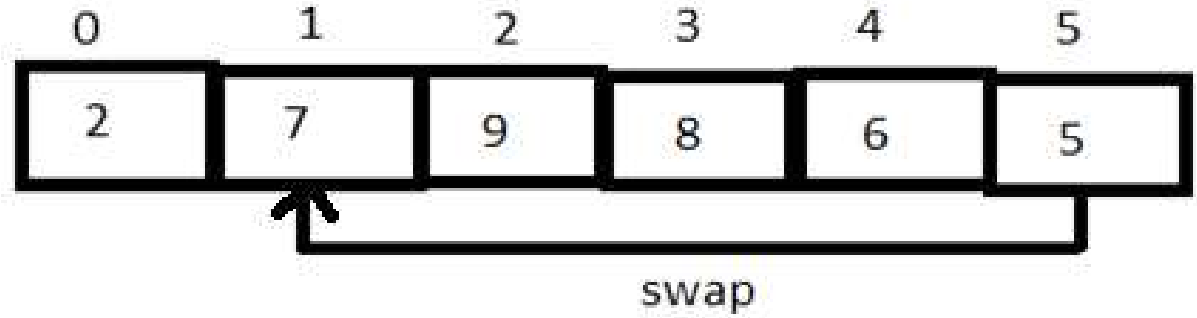
order using Selection sort: {8,7,9,2,6,5}.

# Sol.

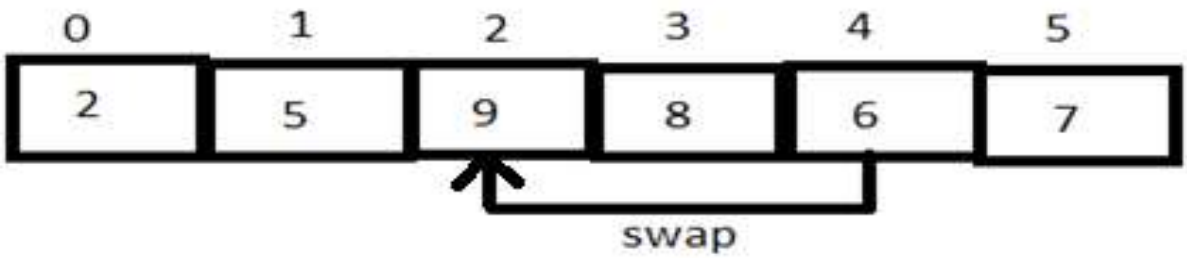
**1st pass**



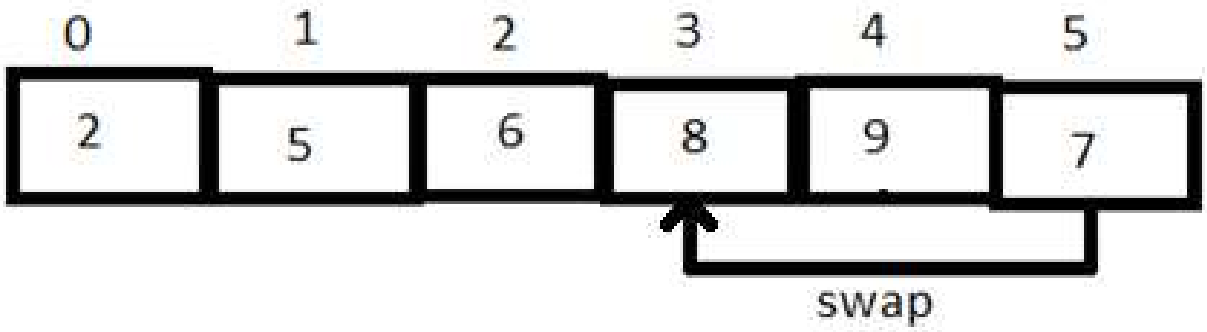

**2nd pass**



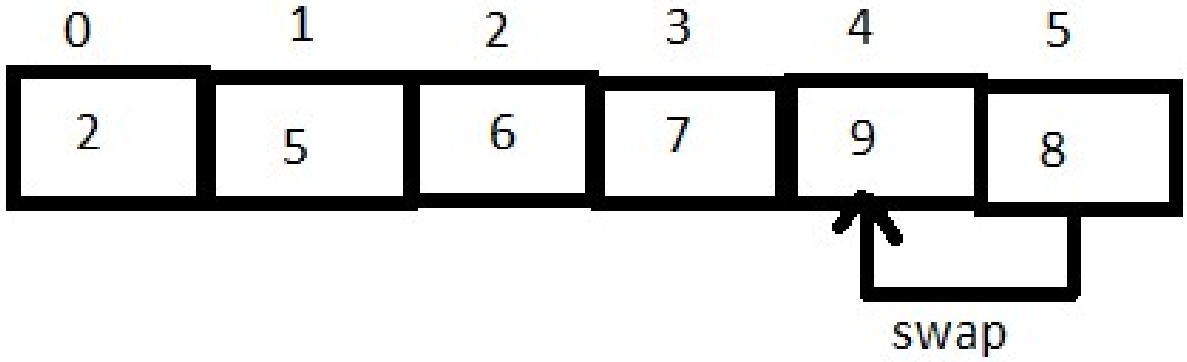
**3rd pass**




**4th pass**




**5th pass**



**After fifth pass entire array gets sorted.**

## Selection Sort

## Algorithm

### SELECTION_SORT(a,n)

```
Step 1: Repeat for i =0 to less than n-1
Step 2: Set min=a[i]
        Set pos=i
Step 3: Repeat for j= i+1 to n-1
            if a[j]<min  then
                Set min=a[j]
                Set pos=j
            [End of if structure]
        [End of Step 3 loop]
Step 4: if pos≠i  then
            Set temp=a[pos]
            Set a[pos]=a[i]
            Set a[i]=temp
        [End of if structure]
[End of Step1 loop]
```

/*A PROGRAM TO SORT an ARRAY IN ASCENDING ORDER USING SELECTION SORT*/

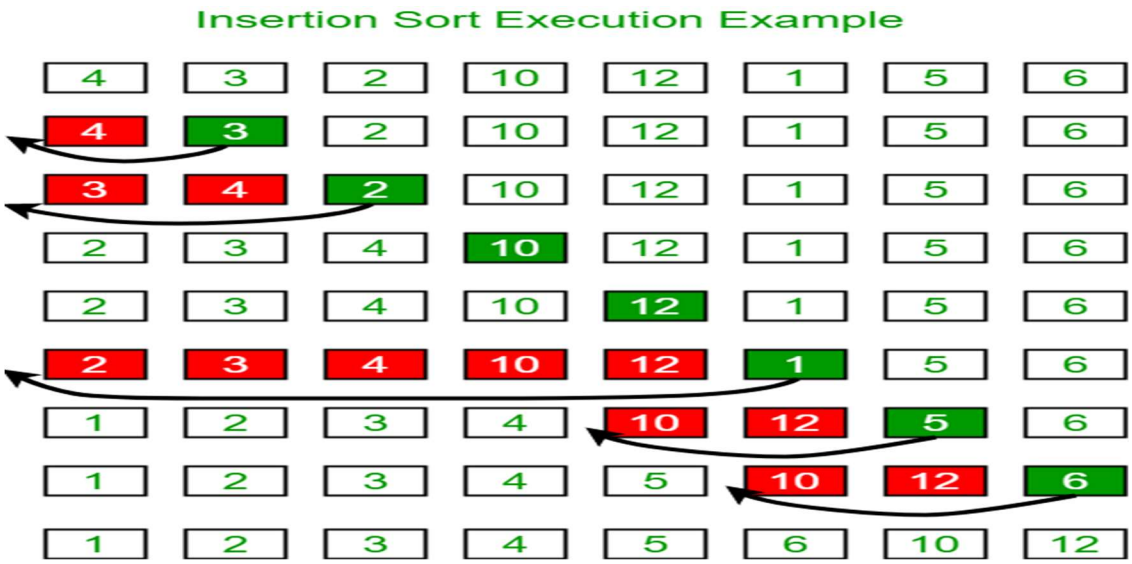
```
#include <stdio.h>

void SELECTION_SORT(int a[],int n)
{
        int i,j,temp,min,pos;
        for(i=0;i<n-1;i++)
        {
                min=a[i];
                pos=i;
      for(j=i+1;j<n;j++)
      {
         if(a[j]<min)
         {
            min=a[j];
            pos=j;
         }
      }
      if(pos!=i)
      {
       temp=a[pos];
       a[pos]=a[i];
       a[i]=temp;
      }
```

```
}

}

void DISPLAY(int a[], int n)
{
    int i;
    for (i=0; i<n; i++)
            printf("%d\t ", a[i]);
}


int main()
{
    int a[15],n,i;

    printf("How many values to sort?:");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("\nEnter value:");
        scanf("%d",&a[i]);
    }

    SELECTION_SORT(a,n);
    printf("\nAfter sorting,array is\n");
```

```
    DISPLAY(a,n);

    return 0;

}
```

# Insertion Sort

Insertion sort is a simple sorting algorithm that works similar to the way you sort playing cards in your hands. The array is virtually split into a sorted and an unsorted part. Values from the unsorted part are picked and placed at the correct position in the sorted part.




## Insertion Sort

## Algorithm

### INSERTION_SORT(a,n)

**Step 1:** **Repeat for i = 1 to n-1**

**Set key = a[i]**

**Set j = i-1**

**Step 2:** **Repeat while j>=0 and key<a[j]**

**Set a[j+1]=a[j]**

**Set j=j-1**

**[End of Step 2 loop]**

**Step 3:** Set a[j+1] = key

[End of Step1 loop]

**/*A PROGRAM TO SORT AN ARRAY IN ASCENDING ORDER USING INSERTION SORT*/**

```
#include <stdio.h>
void INSERTION_SORT(int a[],int n)
{
        int i, j,key;

        for (i=1;i<n;i++)
{
                key = a[i];
                j = i - 1;

                while(j >= 0 &&key<a[j])
{
                        a[j + 1] = a[j];
                        j = j - 1;
                }
                a[j + 1] = key;
```

```
        }



}


void DISPLAY(int a[], int n)
{
   int i;
   for (i=0; i<n; i++)
        printf("%d\t ", a[i]);
}



int main()
{
   int a[15],n,i;


   printf("How many values to sort?:");
   scanf("%d",&n);
   for(i=0;i<n;i++)
   {
      printf("\nEnter value:");
```

```
    scanf("%d",&a[i]);
  }

  INSERTION_SORT(a,n);
  printf("\nAfter sorting,array is\n");
  DISPLAY(a,n);
  return 0;
}
```

## Merging

Merging means combining the elements of two similar data structure into a third data structure.

**MERGING**

**Algorithm**

**MERGE(a,n1,b,n2,c)**

**Let a and b be sorted arrays having n1 and n2 elements respectively.**

**This algorithm merge a and b in an array c with n=n1+n2 elements.**

**Step 1:** Set i=0,j=0 and k=0

**Step 2:** Repeat while i<n1 and j<n2
[Compare]
If a[i]<=b[j] then
[Assign element from a to c]
Set c[k]=a[i]
Set i=i+1
Else
[Assign element from b to c]
Set c[k]=b[j]

Set j=j+1
[End of if structure]
Set k=k+1
[End of Step 2 loop]

**Step 3:** [Assign remaining elements from a to c]
Repeat while i<n1
Set c[k]=a[i]
Set i=i+1
Set k=k+1
[End of Step 3 loop]

**Step 4:** [Assign remaining elements from b to c]
Repeat while j<n2
Set c[k]=b[j]
Set j=j+1
Set k=k+1
[End of Step 4 loop]

**Step 5:** return k

```
/*A PROGRAM TO MERGE TWO SORTED ARRAYS*/

#include<stdio.h>

int MERGE(int a[],int n1,int b[],int n2,int c[])

{

   int i,j,k;

   i=j=k=0;


   while(i<n1&&j<n2)

   {

      if(a[i]<=b[j])

      {

         c[k]=a[i];

         i++;
```

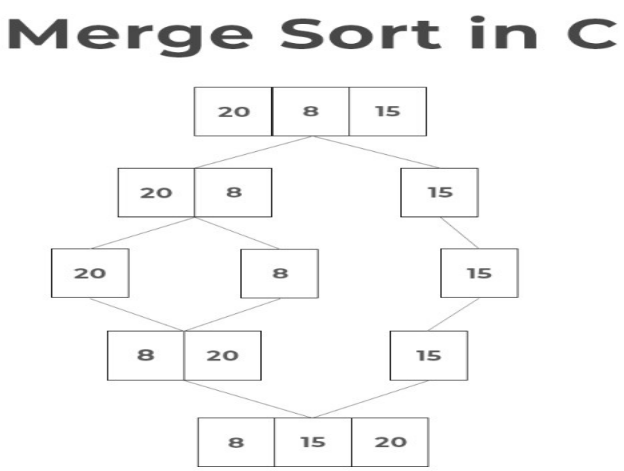
```
        }
        else
        {
            c[k]=b[j];
            j++;
        }
        k++;
    }

    while(i<n1)
    {
        c[k]=a[i];
        i++;
        k++;
    }



    while(j<n2)
    {
        c[k]=b[j];
        j++;
        k++;
    }
    return k;
}
```

```
void DISPLAY(int a[],int n)
{
    int i;
    for(i=0;i<n;i++)
        printf("%d\t",a[i]);
}
int main()
{
    int a[10],b[10],c[20],n1,n2,n,i,j;

    printf("How many values in first array?:");
    scanf("%d",&n1);

    for(i=0;i<n1;i++)
    {
        printf("\nEnter value:");
        scanf("%d",&a[i]);
    }


    printf("\nHow many values in second array?:");
    scanf("%d",&n2);

    for(j=0;j<n2;j++)
    {
        printf("\nEnter value:");
```

```
        scanf("%d",&b[j]);
    }
    n=MERGE(a,n1,b,n2,c);
    printf("\nAfter merging,array is\n");
    DISPLAY(c,n);
    return 0;
}
```

# Merge Sort in C

Merge sort is also based on divide and conquer approach**.** It divides the input array into two sub-arrays and then recursively divide each of the two sub-array until we have sub-arrays of length one. Finally these sub-arrays are merged into one sorted array.




In Merge sort the unsorted array is divided into n sub-arrays, each having one element.

## Merge Sort

## Algorithm

**Merge_Sort(a,beg,end)**

**Step1:** if beg<end then

[Find the middle point to divide the array into two sub-arrays]

Set mid = (beg+end)/2

[Call merge sort for first sub-array]

Call Merge_Sort(a,beg,mid)

[Call merge sort for second sub-array]

Call Merge_Sort(a,mid+1,end)

[Merge the two sorted sub-arrays]

Call Merge(a,beg,mid,end)

[End of if structure]

**Step 2:** return

## Merge

## Algorithm

**Merge(a,beg,mid,end)**

**Step1:** [Finding the size of first and second sorted array]

Set n1 = mid-beg+1

Set n2 = end-mid

**Step 2:** Set i=beg,j=mid+1 and k=beg

**Step 3:** Repeat while i<=mid and j<=end

If a[i]<=a[j] then

Set temp[k]=a[i]

Set i=i+1

Else

Set temp[k]=a[j]

Set j=j+1

[End of if structure]

Set k=k+1
[End of Step 3 loop]

**Step 4:** Repeat while i<=mid
Set temp[k]=a[i]
Set i=i+1
Set k=k+1
[End of Step 4 loop]

**Step 5:** Repeat while j<=end
Set temp[k]=a[j]
Set j=j+1
Set k=k+1
[End of Step 5 loop]

**Step 6:** [Copy elements of temp into a]
Repeat for i = beg to end
Set a[i] = temp[i]
[End of Step 6 loop]

**/*A PROGRAM TO SORTS AN ARRAY IN ASCENDING ORDER USING MERGE SORT*/**

```
#include <stdio.h>

void MERGE(int a[],int beg,int mid,int end)

{

   int i,j,k;

   int n1=mid-beg+1,n2=end-mid;

   int temp[n1+n2];



   i=beg;

   j=mid+1;
```

```
k=beg;


while(i<=mid&&j<=end)
{
   if(a[i]<=a[j])
   {
      temp[k]=a[i];
      i++;
   }
   else
   {
      temp[k]=a[j];
      j++;
   }
   k++;
}
while(i<=mid)
{
   temp[k]=a[i];
   i++;
   k++;
}
```

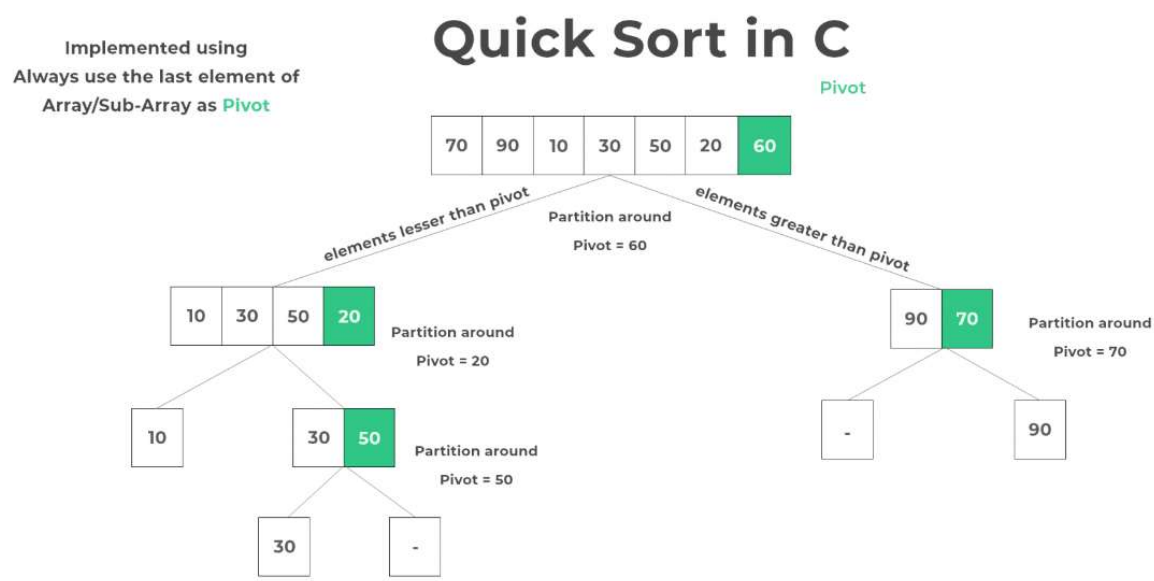
```
    while(j<=end)
    {
        temp[k]=a[j];
        j++;
        k++;
    }
    for(i=beg;i<=end;i++)
        a[i]=temp[i];
}
void MERGE_SORT(int a[],int beg,int end)
{
    int mid;
    if(beg<end)
    {
        mid=(beg+end)/2;
        MERGE_SORT(a,beg,mid);
        MERGE_SORT(a,mid+1,end);
        MERGE(a,beg,mid,end);
    }
}
```

```
void DISPLAY(int a[], int n)
{
   int i;
   for (i=0; i<n; i++)
         printf("%d\t ", a[i]);
}



int main()
{
   int a[15],n,i;

   printf("How many values to sort?:");
   scanf("%d",&n);
   for(i=0;i<n;i++)
   {
      printf("\nEnter value:");
      scanf("%d",&a[i]);
   }

   MERGE_SORT(a,0,n-1);
   printf("\nAfter sorting,array is\n");
```

```
    DISPLAY(a,n);

    return 0;

}
```

# Quick Sort in C

Quick sort is based on divide and conquer approach. On the basis of a selected element (pivot) from the list, it partitions the rest of the list into two parts—a `sub-list` that contains elements less than the pivot and other `sub-list` containing elements greater than the pivot. The pivot is inserted between the two `sub-lists`. The algorithm is recursively applied to the `sub-lists` until the size of each `sub-list` becomes 1, indicating that the whole list has become sorted.




## Quick Sort

## Algorithm

### QUICK_SORT(a,beg,end)

**Step1:** **if beg<end then**

**Set mid = call Partition(a,beg,end)**

Call Quick_Sort(a,beg,mid-1)

Call Quick_Sort(a,mid+1,end)

[End of if structure]

Step 2: return

## Partition

## Algorithm

Partition(a,beg,end)

Step 1: Set pivot = a[end]

Set i=beg-1

Step 2: Repeat for j=beg to end-1

If a[j]<=pivot then

Set i=i+1

temp=a[i]

a[i]=a[j]

a[j]=temp

[End of if structure]

[End of step 2 loop]

Step 3: set temp=a[i+1]

Set a[i+1]=a[end]

Set a[end]=temp

Step 4: return i+1

```
/*A PROGRAM TO SORT an ARRAY IN ASCENDING ORDER USING QUICK SORT*/

#include <stdio.h>

int PARTITION(int a[],int beg,int end)
{
   int pivot,i,j,temp;
   pivot=a[end];
   i=beg-1;
   for(j=beg;j<=end-1;j++)
   {
      if(a[j]<=pivot)
      {
        i++;
        temp=a[i];
        a[i]=a[j];
        a[j]=temp;
      }
   }
      temp=a[i+1];
      a[i+1]=a[end];
```

```
        a[end]=temp;
    return i+1;
}
void QUICK_SORT(int a[],int beg,int end)
{
    int mid;
    if(beg<end)
    {
        mid=PARTITION(a,beg,end);
        QUICK_SORT(a,beg,mid-1);
        QUICK_SORT(a,mid+1,end);
    }
}



void DISPLAY(int a[], int n)
{
    int i;
    for (i=0; i<n; i++)
            printf("%d\t ", a[i]);
}
```

```
int main()
{
   int a[15],n,i;

   printf("How many values to sort?:");
   scanf("%d",&n);
   for(i=0;i<n;i++)
   {
      printf("\nEnter value:");
      scanf("%d",&a[i]);
   }

   QUICK_SORT(a,0,n-1);
   printf("\nAfter sorting,array is\n");
   DISPLAY(a,n);
   return 0;
}
```

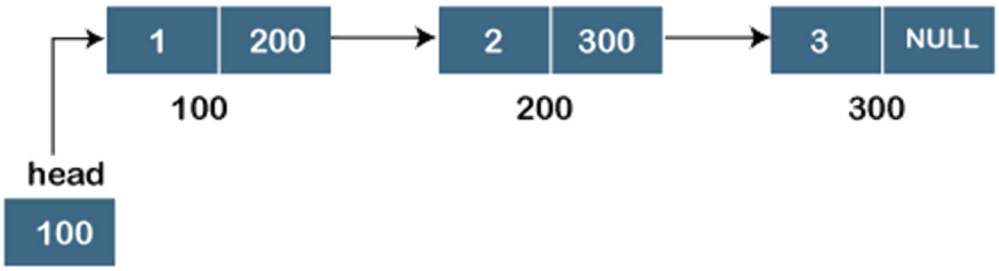
## What is Linked List in Data Structure?

**A linked list is a collection of elements called nodes. Each node is divided into 2 parts as follows-**

**Node**

| Data | *Next |
|---|---|

a) **Data or Info or Item part**
   **This part contains the information**
b) **Next or Link or Address part**
   **This part contains the address of next node**




**Where head is a pointer that contains the address of first node.**
Linked List is a linear data structure like an array.However, the elements or data storage of linked list elements are not in a contiguous manner and we use pointers to create linearity in linked lists.

# Why we use Linked List in Data Structure?

Generally for storage arrays are used. But, there are many disadvantages with arrays –

## Fixed Size

Array is a static data structure i.e memory is pre-allocated to all the elements of an array at compile time. Thus the size of an array is fixed and needs to be pre-defined.

Ex:- `int a[10];` or `int a[] = {1, 2, 3, 4, 5}`

We can't declare an array without declaring the size.

```
int a[];  //Not Ok
```

## Memory wastage (Not Optimized)

- If you declare an array of size 10 and just assign values to first two elements
- The memory is allocated to all of them even though, we may not end up using them.

### Need for Contiguous Memory

- Arrays need Contiguous memory, which sometimes is bad.

### Insertion in Array is expensive

- Let us say you have an array a[] = {10, 20, 40, 60, 80} and we need to add 50 in a same sorted format.
- Then, we have to move all elements after 40 to one additional position

  Also, same goes for deletion if we want to delete 20. Then all elements after it have to be moved to one position in the opposite direction.

# Advantages of using Linked List in C

1. Linked list is a dynamic data structure i.e memory is allocated to the elements of a linked list at run time. Thus the size of a linked list is not fixed and hence it can grow or shrink during the execution of a program.
2. There is no memory wastage since here the memory is not pre-allocated for the elements. Memory is allocated whenever it is required and deallocated when it is no longer needed.
3. Linked List does not need contiguous memory, i.e; if one node has a address of 1000 than the next node may have the address 2000.

4. Insertion and Deletion Operation in Linked Lists are pretty easy and less complicated.

## Disadvantages of using Linked List in C

**1)** We have to search elements linearly, and can't do random access.

2) We can't do efficient searches like binary search

3) Extra space for pointers is needed

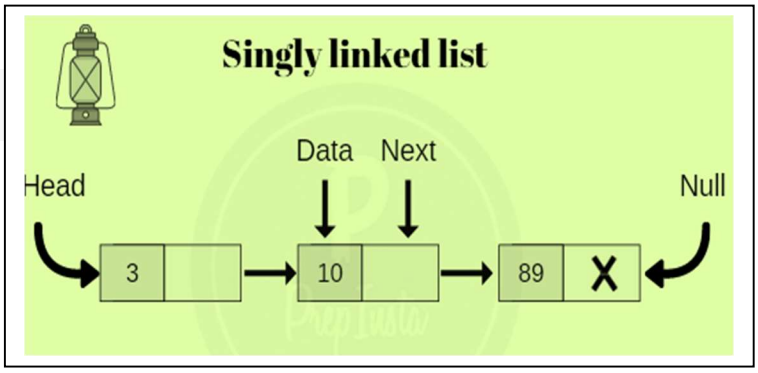
4) Loss of data threat is there, if we lose one pointer location the rest of linked list can't be accessed.

### Types of Linked Lists

There are four types of linked list, that are mentioned in following ways-

1. **Singly linked list**
2. **Circular linked list**
3. **Doubly linked list**
4. **Circular doubly linked list**

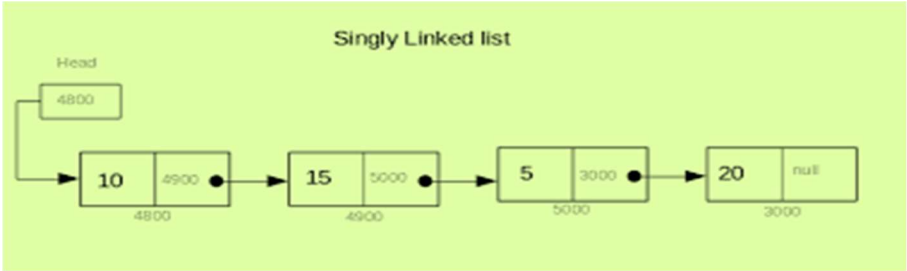
## Singly linked list:-




In a singly list there is the two field on the list.

- **Data:** This part contains some information.
- **Next (pointer):** This part contains the address of next node.
- The Next part of last node of linked list contains NULL which means end of the list

In a Singly linked list, each node only contains the address of the next node and not the previous node, so we can only traverse it in forward direction but not in backward direction.




## Representation of node in singly linked lists

```
struct node {
int data;
struct node* next;
};
```

This code will create a new node in a Linked List which will be storing integer type of data.

# Program to Create a Singly List

```
//We are creating program for linked list creation
#include<stdio.h>
#include<stdlib.h>
//stdlib is used to provide a declaration of 'malloc()' func

// structure of linked list
struct node
{
    int data;
    struct node* next;  // Pointer pointing towards next node
};


struct node* CREATE_LIST(struct node *head)
{
    struct node *node,*last;
    int item;
```

```
    char ans;

    do
    {
        printf("\nEnter the item:");
        scanf("%d",&item);
        //Allocating memory for a node in heap
        node=(struct node*)malloc(sizeof(struct node));
        if(node==NULL)//Check for overflow error

        {
            printf("Insufficient Memory");
            return head;
        }
        node->data=item;
        node->next=NULL;
        if(head==NULL)
            head=node;
        else
            last->next=node;
        last=node;
        printf("\nWant to add another node(y/n)?:");
        getchar();//To clear the buffer
        scanf("%c",&ans);
    }while(ans=='Y'||ans=='y');
    return head;
}

//function to print the linked list
void DISPLAY(struct node *head)
{
    struct node *q;

    if(head==NULL)
        printf("\nList is empty.");
    else
    {
        printf("\nData in nodes are as follows:-\n\n");
        q=head;
        do
        {
            printf("%d\t",q->data);
            q=q->next;
        }while(q!=NULL);
    }
}

int main()
{
    struct node *head=NULL;

    head=CREATE_LIST(head);
    DISPLAY(head);
    return 0;
```

```
}
```

# Insertion

The insertion into a singly linked list can be performed at different positions. Based on the position of the new node being inserted, the insertion is categorized into the following categories.

## Insertion in singly linked list at begining

### Algorithm

**INS_BEG(head,item)**

**Step 1:** [Allocate memory for the new node]
Set node = (struct node*)malloc(sizeof(struct node))
**Step 2:** [Check for overflow error?]
If node = NULL then
Write "Overflow Error"
Return head
[End of If structure]
**Step 3:** [Insert an item into new node]
Set node->data = item
**Step 4:** [Insert new node at the front of list]
Set node->next = head
**Step 5:** [Make the new node as the first node of the list]
Set head = node
**Step 6:** Return head

## Insertion in singly linked list at end

### Algorithm

**INS_LAST(head,item)**

**Step 1:** [Allocate memory for the new node]

Set node = (struct node*)malloc(sizeof(struct node))

**Step 2:** [Check for overflow error?]

If node = NULL then

Write "Overflow Error"

Return head

[End of If structure]

**Step 3:** [Insert an item into new node]

Set node->data = item

**Step 4:** Set node->next = NULL

**Step 5:** [Check for empty list?]

If head==NULL then

Set head = node

Return head

[End of If structure]

**Step 6:** [Traverse the list till the last node]

Set q = head

Repeat while q->next≠Null

Set q = q->next

**Step 7:** [Insert the node at the end]

Set q->next = node

**Step 8:** Return head

## Insertion in singly linked list at specific position

## Algorithm

**INS_SP_POS(head,item)**

**Step 1:** [Allocate memory for the new node]

Set node = (struct node*)malloc(sizeof(struct node))

**Step 2:** [Check for overflow error?]

If node = NULL then

Write "Overflow Error"

Return head

[End of If structure]

**Step 3:** [Insert an item into new node]

Set node->data = item

```
Step 4:  [Check for empty list?]
         If head==NULL then
                  Set node->next=NULL
                  Set head = node
                  Return head
         [End of If structure]
Step 5:  [Ask for position to insert]
         Write "Enter position to insert node"
         Read pos
Step 6:  [Check for invalid position]
         If pos<0 then
                  Write "Invalid position"
                  Return head
         [End of If structure]
Step 7:  If pos<0 then
                  Set head = Call INS_BEG(head,item)
                  Return head
         [End of If structure]

Step 8:  [Traverse the list for specific position]
         Set q = head
Step 9:  Repeat for i = 0 to less than pos-1
                  Set q = q->next
                  [Check for invalid position]
                  If q = NULL then
                           Write "Invalid position"
                           Return head
                  [End of If structure]
         [End of Step 9 loop]
Step 10: Set node->next=q->next
Step 11: Set q->next=node
Step 12: Return head
```

```
/*A PROGRAM TO PERFORM INSERTION OPERATIONS ON SINGLY LINKED LIST*/

#include<stdio.h>
#include<stdlib.h>

struct node
{
   int data;
   struct node *next;
};

struct node* INS_BEG(struct node *head,int item)
{
   struct node *node;

   node=(struct node*)malloc(sizeof(struct node));
   if(node==NULL)
   {
      printf("\nOverflow error");
      return head;
   }

   node->data=item;
   node->next=head;
   head=node;
   return head;
}

struct node* INS_LAST(struct node *head,int item)
{
   struct node *node,*q;
   int i;

   node=(struct node*)malloc(sizeof(struct node));
   if(node==NULL)
   {
      printf("\nOverflow error");
```

```
        return head;
    }

    node->data=item;
    node->next=NULL;

    if(head==NULL)
    {
        head=node;
        return head;
    }

    q=head;
    while(q->next!=NULL)
        q=q->next;
    q->next=node;
    return head;
}


struct node* INS_SP_POS(struct node* head,int item)
{
    struct node *node,*q;
    int pos,i;

    node=(struct node*)malloc(sizeof(struct node));
    if(node==NULL)
    {
        printf("\nOverflow error");
        return head;
    }

    node->data=item;
    if(head==NULL)
    {
        node->next=NULL;
        head=node;
```

```
        return head;
    }
    printf("\nEnter position to insert node:");
    scanf("%d",&pos);

    if(pos<0)
    {
        printf("\nInvalid position. Can't insert");
        return head;
    }

    if(pos==0)
    {
        head=INS_BEG(head,item);
        return head;
    }
        q=head;
        for(i=0;i<pos-1;i++)
        {
            q=q->next;
            if(q==NULL)
            {
                printf("\nInvalid position. Can't insert");
                return head;
            }
        }

        node->next=q->next;
        q->next=node;
    return head;
}


void DISPLAY(struct node *head)
{
    struct node *q;
```

```
   if(head==NULL)
      printf("\nList is empty.");
   else
   {
      printf("\nData in nodes are as follows:-\n\n");
      q=head;
      do
      {
         printf("%d\t",q->data);
         q=q->next;
      }while(q!=NULL);
   }
}

int main()
{
   struct node *head=NULL;
   int item,choice;

   while(1)
   {
      printf("\n\t\t\t*****OPERATION ON  LINKED LIST*****");
      printf("\n\t\t1.Insertion of node at begining.");
      printf("\n\t\t2.Insertion of node at last.");
      printf("\n\t\t3.Insertion of node at specific position");
      printf("\n\t\t4.Display");
      printf("\n\t\t5.Exit");
      printf("\n\nEnter your choice:");
      scanf("%d",&choice);
      if(choice>=1&&choice<=3)
      {
         printf("\nEnter the item:");
         scanf("%d",&item);
      }
      switch(choice)
      {
         case 1:
```

```
            head=INS_BEG(head,item);
        break;

        case 2:
            head=INS_LAST(head,item);
        break;

        case 3:
            head=INS_SP_POS(head,item);
        break;

        case 4:
            DISPLAY(head);
        break;

        case 5:
            printf("\n\t\t***GOOD BYE***");
            exit(0);

        default:
            printf("\n\nYou entered a wrong choice.");
    }
  }
  return 0;
}
```

## Deletion in singly linked list from begining

### Algorithm

**DEL_BEG(head)**

**Step 1:** **[**Check for empty list?]
If head=NULL then
Write "List is empty"
return head
[End of If structure]

**Step 2:** **[**Getting address of first node]

Set q=head

**Step 3:** [Assigning address of second node to head]

Set head=head->next

**Step 4:** [Delete the node at the front of list]

free(q)

**Step 5:** return head

## Deletion in singly linked list from end

## Algorithm

**DEL_LAST(head)**

**Step 1:** [Check for empty list?]

If head=NULL then

Write "List is empty"

Return head

[End of If structure]

**Step 2:** [Getting address of last node and its previous node]

Set q=head

Repeat while q->next≠NULL

Set p=q

Set q=q->next

[End of Step 2 loop]

**Step 3:** [Check for only one node]

If q=head then

Set head=NULL

Else

Set p->next=q->next

[End of If structure]

**Step 4:** [Delete the node at the rear of list]

free(q)

**Step 5:** Return head

## Deletion in singly linked list from specific position

## Algorithm

**DEL_SP_POS(head)**

**Step 1:** [Check for empty list?]
If head=NULL then
Write "List is empty"
Return head
[End of If structure]

**Step 2:** [Ask for position to delete]
Write "Enter position to delete node"
Read pos
**Step 3:** [Check for invalid position]
If pos<0 then
Write "Invalid position."
Return head
[End of If structure]
**Step 4:** If pos==0 then
Set head = Call DEL_BEG(head,item)
Return head
[End of If structure]

**Step 5:** [Traverse the list for specific position]
Set q = head
**Step 6:** Repeat for i = 0 to less than pos
Set p = q
Set q = q->next
[Check for invalid position]
If q = NULL then
Write "Invalid position."
Return head
[End of If structure]
[End of Step 6 loop]
**Step 7:** Set p->next=q->next
**Step 8:** [Delete the node from specific position of list]

free(q)

**Step 9:** Return head

```
/*A PROGRAM TO PERFORM DELETION OPERATIONS ON SINGLY LINKED LIST*/
#include<stdio.h>
#include<stdlib.h>
struct node
{
   int data;
   struct node *next;
};
struct node* CREATE_LIST(struct node *head)
{
   struct node *node,*q;
   int item;
   char ans;

   do
   {
      printf("\nEnter item:");
      scanf("%d",&item);
      node=(struct node*)malloc(sizeof(struct node));
      node->data=item;
      node->next=NULL;
      if(head==NULL)
         head=node;
      else
      {
         q=head;
         while(q->next!=NULL)
            q=q->next;
         q->next=node;
      }
      printf("\nWant to add another[y/n]:");
      getchar();
      scanf("%c",&ans);
   }while(ans=='y'||ans=='Y');
```

```
    return head;
}

struct node* DEL_BEG(struct node *head)
{
    struct node *q;

    if(head==NULL)
    {
        printf("\nList is empty.");
        return head;
    }
    q=head;
    head=head->next;
    free(q);
    return head;
}

struct node* DEL_LAST(struct node *head)
{
    struct node *p,*q;

    if(head==NULL)
    {
        printf("\nList is empty.");
        return head;
    }
    q=head;
    while(q->next!=NULL)
    {
        p=q;
        q=q->next;
    }
    if(q==head)
        head=NULL;
    else
        p->next=q->next;
```

```
    free(q);
    return head;
}


struct node* DEL_SP_POS(struct node *head)
{
    struct node *p,*q;
    int pos,i;

    if(head==NULL)
    {
      printf("\nList is empty.");
      return head;
    }


        printf("\nEnter position to delete:");
        scanf("%d",&pos);
        if(pos<0)
        {
            printf("\nCan't delete");
            return head;
        }
        if(pos==0)
        {
            head=DEL_BEG(head);
            return head;
        }

        q=head;
        for(i=0;i<pos;i++)
        {
            p=q;
            q=q->next;
            if(q==NULL)
            {
```

```
            printf("\nCan't delete");
            return head;
         }

      }
         p->next=q->next;
      free(q);
   return head;
}
void DISPLAY(struct node *head)
{
   struct node *q;

   if(head==NULL)
      printf("\nList is empty.");
   else
   {
      printf("\nData in nodes are as follows:-\n\n");
      q=head;
      do
      {
         printf("%d\t",q->data);
         q=q->next;
      }while(q!=NULL);
   }
}

int main()
{
   struct node *head=NULL;
   int choice;
   while(1)
   {
      printf("\n\t\t\t*****OPERATION ON LINEAR LINKED LIST*****");
      printf("\n\t\t1.Insertion of nodes in linked list.");
      printf("\n\t\t2.Deletion of node from begining.");
      printf("\n\t\t3.Deletion of node from end.");
```

```
    printf("\n\t\t4.Deletion of node from specific position.");
    printf("\n\t\t5.Display");
    printf("\n\t\t6.Exit");
    printf("\n\nEnter your choice:");
    scanf("%d",&choice);
    switch(choice)
    {
      case 1:
        head=CREATE_LIST(head);
      break;

      case 2:
        head=DEL_BEG(head);
      break;
      case 3:
        head=DEL_LAST(head);
      break;
      case 4:
        head=DEL_SP_POS(head);
      break;
      case 5:
        DISPLAY(head);
      break;
      case 6:
        printf("\n\t\t***GOOD BYE***");
        exit(0);
      default:
        printf("\nWrong choice.");
    }
  }
  return 0;
}
```

## Merging of two singly linked list

## Algorithm

**MERGE_LIST(P,Q,R)**

**Step 1:** **[**Assigning address of first node of both list to p and q respectively]
Set p = P and q = Q
**Step 2:** Repeat while p≠NULL and q≠NULL
**[**Allocating memory for a node**]**
Set node=(struct node*)malloc(sizeof(struct node))
[Compare]
If p->data<=q->data then
[Assigning data from first list]
Set node->data=p->data
Set p=p->next
Else
[Assigning data from second list]
Set node->data=q->data
Set q=q->next
**[**End of If structure**]**
**Step 3:** Set node->next=NULL
**Step 4:** **[**Check for empty list?**]**
If R=NULL then
Set R=node
Else
Set last->next=node
**[**End of If structure**]**
**Step 5:** Set last=node
**[**End of Step 2 loop**]**
**Step 6:** **[**Assign remaining data from first list**]**
Repeat while p≠NULL
**[**Allocating memory for a node**]**
Set node=(struct node*)malloc(sizeof(struct node))
Set node->data=p->data
Set node->next=NULL
Set last->next=node
Set last=node

Set p=p->next
[End of Step 6 loop]

**Step 7:** [Assign remaining data from second list]
Repeat while q≠NULL
[Allocating memory for a node]
Set node=(struct node*)malloc(sizeof(struct node))
Set node->data=q->data
Set node->next=NULL
Set last->next=node
Set last=node
Set q=q->next
[End of Step 7 loop]

**Step 8:** Return R

```
/*A PROGRAM TO MERGE TWO SINGLY LINKED LIST*/
#include<stdio.h>
#include<stdlib.h>

struct node
{
   int data;
   struct node *next;
};


struct node* CREATE_LIST(struct node *head)
{
   struct node *node,*last;
   int item;
   char ans;

   do
   {
      printf("\nEnter item:");
      scanf("%d",&item);
      node=(struct node*)malloc(sizeof(struct node));
```

```
      node->data=item;
      node->next=NULL;
      if(head==NULL)
         head=node;
      else
         last->next=node;
      last=node;
      printf("\nDo you want to add another node(y/n):");
      getchar();
      scanf("%c",&ans);
   }while(ans=='Y'||ans=='y');
   return head ;
}

struct node* MERGE_LIST(struct node *P,struct node *Q,struct node *R)
{
   struct node *node,*last;
   struct node *p,*q;

   p=P;
   q=Q;
   while(p!=NULL&&q!=NULL)
   {
      node=(struct node*)malloc(sizeof(struct node));
      if(p->data<=q->data)
      {
         node->data=p->data;
         p=p->next;
      }
      else
      {
         node->data=q->data;
         q=q->next;
      }
      node->next=NULL;
      if(R==NULL)
         R=node;
```

```
      else
         last->next=node;
      last=node;
   }

   while(p!=NULL)
   {
      node=(struct node*)malloc(sizeof(struct node));
      node->data=p->data;
      node->next=NULL;
      last->next=node;
      last=node;
      p=p->next;
   }

   while(q!=NULL)
   {
      node=(struct node*)malloc(sizeof(struct node));
      node->data=q->data;
      node->next=NULL;
      last->next=node;
      last=node;
      q=q->next;
   }
   return(R);
}

void DISPLAY(struct node *head)
{
   struct node *q;

   if(head==NULL)
      printf("\nList is empty");
   else
   {
      q=head;
      while(q!=NULL)
```

```
      {
         printf("%d\t",q->data);
         q=q->next;
      }
   }
}

int main()
{
   struct node *P=NULL,*Q=NULL,*R=NULL;

   printf("\n\n\t\t***Creating First List***\n");
   P=CREATE_LIST(P);
   printf("\n\n\t\t***Creating Second List***\n");
   Q=CREATE_LIST(Q);
   R=MERGE_LIST(P,Q,R);
   printf("\nFirst list is\n");
   DISPLAY(P);
   printf("\nSecond list is\n");
   DISPLAY(Q);
   printf("\nResultant list is\n");
   DISPLAY(R);
   return 0;
}


/*A PROGRAM TO COPY ONE SINGLY LINKED LIST INTO ANOTHER*/
#include<stdio.h>
#include<stdlib.h>

struct node
{
   int data;
   struct node *next;
};

struct node* CREATE_LIST(struct node *P)
```

```
{
   struct node *node,*last;
   int item;
   char ans;

   do
   {
      printf("\nEnter the item:");
      scanf("%d",&item);
      node=(struct node*)malloc(sizeof(struct node));
      node->data=item;
      node->next=NULL;
      if(P==NULL)
         P=node;
      else
         last->next=node;
      last=node;
      printf("\nDo you want to add another node(y/n):");
      getchar();
      scanf("%c",&ans);
   }while(ans=='Y'||ans=='y');
   return(P);
}

struct node* COPY_LIST(struct node *Q,struct node *P)
{
   struct node *node,*last,*p;

   p=P;
   while(p!=NULL)
   {
      node=(struct node*)malloc(sizeof(struct node));
      node->data=p->data;
      node->next=NULL;
      if(Q==NULL)
         Q=node;
      else
```

```
        last->next=node;
      last=node;
      p=p->next;
   }
   return(Q);
}

void DISPLAY(struct node *start)
{
   struct node *q;

   q=start;
   while(q!=NULL)
   {
   printf("%d\t",q->data);
   q=q->next;
   }
}
int main()
{
   struct node *P=NULL,*Q=NULL;
   struct node* CREATE_LIST(struct node *P);
   struct node* COPY_LIST(struct node *Q,struct node *P);
   void DISPLAY(struct node *start);

   P=CREATE_LIST(P);
   Q=COPY_LIST(Q,P);
   printf("\nOriginal list is\n");
   DISPLAY(P);
   printf("\nCopy of original list is\n");
   DISPLAY(Q);
   return 0;
}

/*A PROGRAM TO DISPLAY SINGLY LINKED LIST IN REVERSE ORDER*/

#include<stdio.h>
```

```
#include<stdlib.h>

struct node
{
   int data;
   struct node *next;
};

struct node* CREATE_LIST(struct node *head)
{
   struct node *node,*q;
   int item;
   char ans;

   do
   {
      printf("\nEnter item:");
      scanf("%d",&item);
      node=(struct node*)malloc(sizeof(struct node));
      node->data=item;
      node->next=NULL;
      if(head==NULL)
         head=node;
      else
      {
         q=head;
         while(q->next!=NULL)
            q=q->next;
         q->next=node;
      }
      printf("\nWant to add another[y/n]:");
      getchar();
      scanf("%c",&ans);
   }while(ans=='y'||ans=='Y');
   return(head);
}
```

```
void DISPLAY_REVERSE(struct node *head,struct node *current)
{
   if(head==NULL)
   {
      printf("\nList is empty.");
      return;
   }

   if(current==NULL)
      return;
   DISPLAY_REVERSE(head,current->next);
   printf("%d\t",current->data);

}
void DISPLAY(struct node *head)
{
   struct node *q;

   if(head==NULL)
      printf("\nList is empty.");
   else
   {
      printf("\nData in nodes are as follows:-\n\n");
      q=head;
      while(q!=NULL)
      {
         printf("%d\t",q->data);
         q=q->next;
      }
   }
}


int main()
{
   struct node *head=NULL;
```

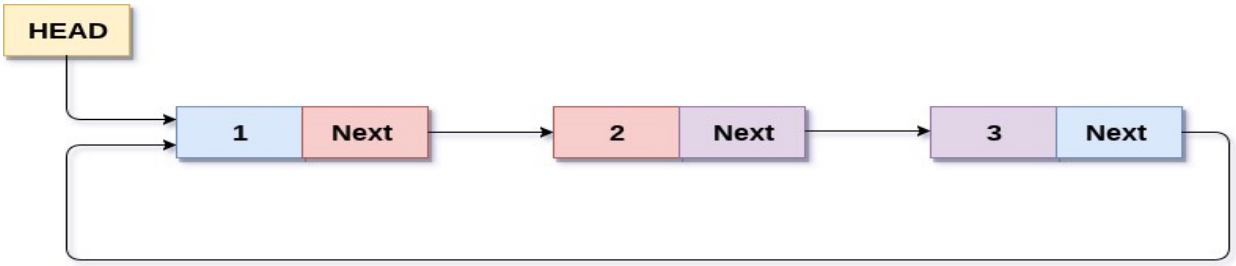
```
    int choice;


    while(1)
    {
        printf("\n\t\t\t*****OPERATION ON LINEAR LINKED LIST*****\n\n\t\t\t");
        printf("1.Insertion of nodes in linked list.\n\n\t\t\t");
        printf("2.Display reverse of the list.\n\n\t\t\t");
        printf("3.Display\n\n\t\t\t");
        printf("4.Exit\n\n");
        printf("Enter your choice:");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1:
                head=CREATE_LIST(head);
            break;

            case 2:
                DISPLAY_REVERSE(head,head);
            break;

            case 3:
                DISPLAY(head);
                break;

            case 4:
                printf("\n\t\t***GOOD BYE***");
                exit(0);
            default:
                printf("\nWrong choice.");
        }
    }
    return 0;
}
```

# Circular Singly Linked List

In a circular singly linked list, the last node of the list contains a pointer to the first node of the list.

We traverse a circular singly linked list until we reach the same node where we started. The circular singly liked list has no beginning and no ending. There is no null value present in the next part of any of the nodes.

The following image shows a circular singly linked list.




Circular Singly Linked List

## Insertion in circular linked list at begining

### Algorithm

**INS_BEG(head,item)**

**Step 1:** [Allocate memory for the new node]
Set node = (struct node*)malloc(sizeof(struct node))
**Step 2:** [Check for overflow error?]
If node = NULL then
Write "Overflow Error"
Return head
[End of If structure]
**Step 3:** [Insert an item into new node]
Set node->data = item
**Step 4:** [Check for empty list?]
If head==NULL then

```
                Set head = node
                Set node->next=head
                Return head
        [End of If structure]
Step 5: [Traverse the list]
        Set q = head
        Repeat while q->next≠head
                Set q = q->next
Step 6: [Insert the node at the front of list]
        Set q->next = node
        Set node->next=head
        Set head=node
Step 7: Return head
```

## Insertion in circular linked list at last

## Algorithm

**INS_LAST(head,item)**

```
Step 1: [Allocate memory for the new node]
        Set node = (struct node*)malloc(sizeof(struct node))
Step 2: [Check for overflow error?]
        If node = NULL then
                Write "Overflow Error"
                Return head
        [End of If structure]
Step 3: [Insert an item into new node]
        Set node->data = item
Step 4: [Check for empty list?]
        If head==NULL then
                Set head = node
                Set node->next=head
                Return head
        [End of If structure]
Step 5: [Traverse the list]
        Set q = head
        Repeat while q->next≠head
```

Set q = q->next

**Step 6:** [Insert the node at the end of list]

Set q->next = node

Set node->next=head

**Step 7:** Return head

## Insertion in circular linked list at specific position

## Algorithm

**INS_SP_POS(head,item)**

**Step 1:** [Allocate memory for the new node]

Set node = (struct node*)malloc(sizeof(struct node))

**Step 2:** [Check for overflow error?]

If node = NULL then

Write "Overflow Error"

Return head

[End of If structure]

**Step 3:** [Insert an item into new node]

Set node->data = item

**Step 4:** [Check for empty list?]

If head==NULL then

Set node->next=NULL

Set head = node

Return head

[End of If structure]

**Step 5:** [Ask for position to insert]

Write "Enter position to insert node"

Read pos

**Step 6:** [Check for invalid position]

If pos<0 then

Write "Invalid position"

Return head

[End of If structure]

**Step 7:** If pos<0 then

Set head = Call INS_BEG(head,item)
Return head
[End of If structure]

**Step 8:** [Traverse the list for specific position]
Set q = head
**Step 9:** Repeat for i = 0 to less than pos-1
Set q = q->next
[Check for invalid position]
If q = NULL then
Write "Invalid position"
Return head
[End of If structure]
[End of Step 9 loop]
**Step 10:** Set node->next=q->next
**Step 11:** Set q->next=node
**Step 12:** Return head

```
/*A PROGRAM TO PERFORM INSERTION OPERATIONS ON CIRCULAR LINKED
LIST*/

#include<stdio.h>
#include<stdlib.h>

struct node
{
   int data;
   struct node *next;
};

struct node* INS_BEG(struct node *head,int item)
{
   struct node *node,*q;

   node=(struct node*)malloc(sizeof(struct node));
   if(node==NULL)
   {
```

```
        printf("\nOverflow error");
        return head;
    }

    node->data=item;
    if(head==NULL)
    {
        head=node;
        node->next=head;
        return head;
    }
    q=head;
    while(q->next!=head)
        q=q->next;
    q->next=node;
    node->next=head;
    head=node;
    return(head);
}


struct node* INS_LAST(struct node *head,int item)
{
    struct node *node,*q;

    node=(struct node*)malloc(sizeof(struct node));

    if(node==NULL)
    {
        printf("\nOverflow error");
        return head;
    }

    node->data=item;
    if(head==NULL)
    {
        head=node;
```

```
        node->next=head;
        return head;

    }
    q=head;
    while(q->next!=head)
        q=q->next;
    q->next=node;
    node->next=head;
    return head;
}



struct node* INS_SP_POS(struct node* head,int item)
{
    struct node *node,*q;
    int pos,i;

    node=(struct node*)malloc(sizeof(struct node));
    if(node==NULL)
    {
        printf("\nOverflow error");
        return head;
    }

    node->data=item;
    if(head==NULL)
    {
       head=node;
       node->next=head;
       return head;
    }
    printf("\nEnter position to insert node:");
    scanf("%d",&pos);

    if(pos<0)
```

```
    {
        printf("\nInvalid position. Can't insert");
        return head;
    }

    if(pos==0)
    {
        head=INS_BEG(head,item);
        return head;
    }
        q=head;
        for(i=0;i<pos-1;i++)
        {
            q=q->next;
            if(q==head)
            {
                printf("\nInvalid position. Can't insert");
                return head;
            }
        }

        node->next=q->next;
        q->next=node;

        return head;
}


void DISPLAY(struct node *head)
{
    struct node *q;

    if(head==NULL)
        printf("\nList is empty.");
    else
    {
        printf("\nData in nodes are as follows:-\n\n");
```

```
        q=head;
        do
        {
            printf("%d\t",q->data);
            q=q->next;
        }while(q!=head);
    }
}


int main()
{
    struct node *head=NULL;
    int item,choice;

    while(1)
    {
        printf("\n\t\t\t*****OPERATION ON  LINKED LIST*****");
        printf("\n\t\t1.Insertion of node at begining.");
        printf("\n\t\t2.Insertion of node at last.");
        printf("\n\t\t3.Insertion of node at specific position");
        printf("\n\t\t4.Display");
        printf("\n\t\t5.Exit");
        printf("\n\nEnter your choice:");
        scanf("%d",&choice);
        if(choice>=1&&choice<=3)
        {
            printf("\nEnter the item:");
            scanf("%d",&item);
        }
        switch(choice)
        {
            case 1:
                head=INS_BEG(head,item);
            break;

            case 2:
```

```
            head=INS_LAST(head,item);
        break;

        case 3:
            head=INS_SP_POS(head,item);
        break;

        case 4:
            DISPLAY(head);
        break;

        case 5:
            printf("\n\t\t***GOOD BYE***");
            exit(0);

        default:
            printf("\n\nYou entered a wrong choice.");
    }
  }
  return 0;
}
```

## Deletion in circular linked list from begining

## Algorithm

**DEL_BEG(head)**

**Step 1:** **[**Check for empty list?]
If head=NULL then
Write "List is empty"
Return head
[End of If structure]

**Step 2:** **[**Getting address of first and last node]
Set q=head
Set p=q
Repeat while p->next≠head
Set p=p->next

[End of Step 2 loop]

**Step 3:** [Check for only one node]
If p=head then
Set head=NULL
Else
Set p->next=q->next
Set head=head->next
[End of If structure]

**Step 4:** [Delete the node from the front of list]
free(q)

**Step 5:** Return head

## Deletion in circular linked list from last

## Algorithm

**DEL_LAST(head)**

**Step 1:** [Check for empty list?]
If head=NULL then
    Write "List is empty"
    Return head
[End of If structure]

**Step 2:** [Getting address of last node and its previous node]
Set q=head
Repeat while q->next≠head
Set p=q
Set q=q->next
[End of Step 2 loop]

**Step 3:** [Check for only one node]
If q=head then
Set head=NULL
Else
Set p->next=q->next
[End of If structure]

**Step 4:** [Delete the node from the rear of list]
free(q)

**Step 5:** Return head

## Deletion in circular linked list from specific position

## Algorithm

**DEL_SP_POS(head)**

**Step 1:** **[**Check for empty list?]
If head=NULL then
Write "List is empty"
Return head
[End of If structure]

**Step 2:** **[**Ask for position to delete**]**
Write "Enter position to delete node"
Read pos
**Step 3:** **[**Check for invalid position**]**
If pos<0 then
Write "Invalid position."
Return head
**[**End of If structure**]**
**Step 4:** If pos==0 then
Set head = Call DEL_BEG(head,item)
Return head
**[**End of If structure**]**

**Step 5:** **[**Traverse the list for specific position]
Set q = head
**Step 6:** Repeat for i = 0 to less than pos
Set p = q
Set q = q->next
**[**Check for invalid position**]**
If q = head then
Write "Invalid position."
Return head
**[**End of If structure**]**
**[**End of Step 6 loop**]**

**Step 7:** Set p->next=q->next
**Step 8:** [Delete the node from specific position of list]
free(q)
**Step 9:** Return head

```
/*A program to perform deletion operations on circular linked list*/
#include<stdio.h>
#include<stdlib.h>
struct node
{
   int data;
   struct node *next;
};
struct node* CREATE_LIST(struct node *head)
{
   struct node *node,*q;
   int item;
   char ans;

   do
   {
      printf("\nEnter item:");
      scanf("%d",&item);
      node=(struct node*)malloc(sizeof(struct node));

      node->data=item;
      if(head==NULL)
      {
        head=node;
        node->next=head;
      }
      else
      {
         q=head;
```

```
        while(q->next!=head)
            q=q->next;
        q->next=node;
        node->next=head;
    }
    printf("\nWant to add another[y/n]:");
    getchar();
    scanf("%c",&ans);
    }while(ans=='y'||ans=='Y');
  return head;
}
struct node* DEL_BEG(struct node *head)
{
  struct node *p,*q;

  if(head==NULL)
  {
    printf("\nList is empty.");
    return head;
  }
  q=head;
  p=q;
  while(p->next!=head)
    p=p->next;
  if(p==head)
    head=NULL;
  else
  {
    p->next=q->next;
    head=head->next;
  }

  free(q);
return head;
}
```

```
struct node* DEL_LAST(struct node *head)
{
   struct node *p,*q;

   if(head==NULL)
   {
      printf("\nList is empty.");
      return head;
   }
   q=head;
   while(q->next!=head)
   {
      p=q;
      q=q->next;
   }

   if(q==head)
      head=NULL;
   else
      p->next=q->next;

   free(q);
return head;
}

struct node* DEL_SP_POS(struct node *head)
{
   struct node *p,*q;
   int pos,i;

   if(head==NULL)
   {
     printf("\nList is empty.");
     return head;
   }
```

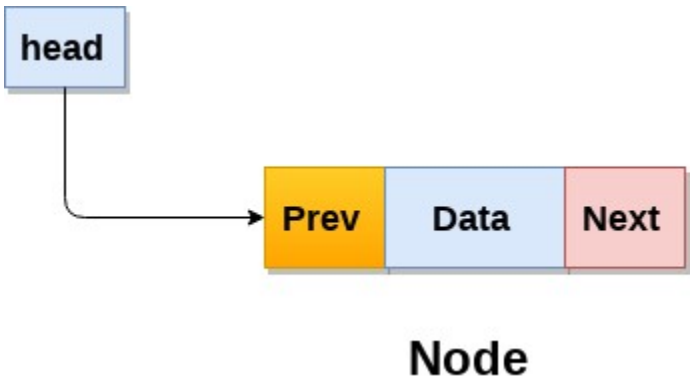
```
    printf("\nEnter position to delete:");
    scanf("%d",&pos);
    if(pos<0)
    {
      printf("\nCan't delete");
      return head;
    }

    if(pos==0)
    {
      head=DEL_BEG(head);
      return head;
    }

    q=head;
    for(i=0;i<pos;i++)
    {
      p=q;
      q=q->next;
      if(q==head)
      {
        printf("\nCan't delete");
        return head;
      }

    }
    p->next=q->next;
    free(q);

    return head;
}

void DISPLAY(struct node *head)
{
  struct node *q;

  if(head==NULL)
```

```
        printf("\nList is empty.");
    else
    {
        printf("\nData in nodes are as follows:-\n\n");
        q=head;
        do
        {
            printf("%d\t",q->data);
            q=q->next;
        }while(q!=head);
    }
}
int main()
{
    struct node *head=NULL;
    int choice;
    while(1)
    {
        printf("\n\t\t\t*****DELETION OPERATION ON CIRCULAR LINKED
LIST*****");
        printf("\n\t\t1.Insertion of nodes in linked list.");
        printf("\n\t\t2.Deletion of node from begining.");
        printf("\n\t\t3.Deletion of node from last.");
        printf("\n\t\t4.Deletion of node from specific position.");
        printf("\n\t\t5.Display");
        printf("\n\t\t6.Exit");
        printf("\n\nEnter your choice:");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1:
                head=CREATE_LIST(head);
            break;

            case 2:
                head=DEL_BEG(head);
            break;
```

```
        case 3:
           head=DEL_LAST(head);
        break;

        case 4:
           head=DEL_SP_POS(head);
        break;
        case 5:
           DISPLAY(head);
        break;
        case 6:
           printf("\n\t\t***GOOD BYE***");
           exit(0);
        default:
           printf("\nWrong choice.");
     }
  }
  return 0;
}
```

# Doubly linked list

A doubly linked list is a collection of nodes in which each node is divided into 3 parts as follows-

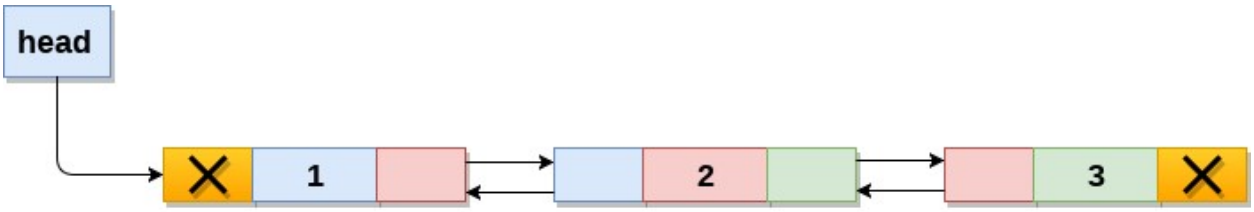



Where

*Prev part stores the address of its previous(predecessor) node

Data part stores some information and

Next part stores the address of next(Successor) node




**Doubly Linked List**

In a singly linked list, we could traverse only in one direction, because each node contains address of the next node and it doesn't have any record of its previous nodes. However, doubly linked list overcome this limitation of singly linked list. Due to the fact that, each node of the list contains the address of its previous node, we can find all the details about the previous node as well by using the previous address stored inside the previous part of each node.

**Representation of node in doubly linked list**

```
struct node
{
    int data;
    struct node *prev;
    struct node *next;
}
```

## Insertion in doubly linked list at begining

### Algorithm

**INS_BEG(head,item)**

**Step 1:** [Allocate memory for the new node]
Set node = (struct node*)malloc(sizeof(struct node))
**Step 2:** [Check for overflow error?]
If node = NULL then
Write "Overflow Error"
Return head
[End of If structure]
**Step 3:** [Insert an item into new node]
Set node->data = item
**Step 4:** [Insert new node at the front of list]
Set node->prev=NULL
Set node->next = head
**Step 5:** If head≠NULL then
Set head->prev=node
**Step 6:** [Make the new node as the first node of the list]
Set head = node
**Step 7:** Return head

## Insertion in doubly linked list at end

### Algorithm

**INS_LAST(head,item)**

**Step 1:** [Allocate memory for the new node]
Set node = (struct node*)malloc(sizeof(struct node))

**Step 2:** [Check for overflow error?]
If node = NULL then
Write "Overflow Error"
Return head
[End of If structure]
**Step 3:** [Insert an item into new node]
Set node->data = item
**Step 4:** Set node->next = NULL
**Step 5:** [Check for empty list?]
If head=NULL then
Set node->prev = NULL
Set head=node
Return head
[End of If structure]
**Step 6:** [Traverse the list till the last node]
Set q = head
Repeat while q->next≠NULL
Set q = q->next
**Step 7:** [Insert the node at the end]
Set q->next = node
Set node->prev=q
**Step 8:** Return head

## Insertion in doubly linked list at specific position

## Algorithm

**INS_SP_POS(head,item)**

**Step 1:** [Allocate memory for the new node]
Set node = (struct node*)malloc(sizeof(struct node))
**Step 2:** [Check for overflow error?]
If node = NULL then
Write "Overflow Error"
Return head
[End of If structure]
**Step 3:** [Insert an item into new node]

```
        Set node->data = item
Step 4: [Check for empty list?]
        If head=NULL then
                Set node->prev=NULL
                Set node->next=NULL
                Set head = node
                Return head
        [End of If structure]
Step 5: [Ask for position to insert]
        Write "Enter position to insert node"
        Read pos
Step 6: [Check for invalid position]
        If pos<0 then
                Write "Invalid position"
                Return head
        [End of If structure]
Step 7: If pos=0 then
                Set head = Call INS_BEG(head,item)
                Return head
        [End of If structure]

Step 8: [Traverse the list for specific position]
        Set q = head
Step 9: Repeat for i = 0 to less than pos-1
                Set q = q->next
                [Check for invalid position]
                If q = NULL then
                        Write "Invalid position"
                        Return head
                [End of If structure]
        [End of Step 9 loop]
Step 10: Set node->next=q->next
Step 11: Set node->prev=q
Step 12: Set q->next=node
Step 13: If node->next≠NULL then
           Set node->next->prev=q->next
        [End of If structure]
```

**Step 14:** Return head

```
/*A PROGRAM TO PERFORM INSERTION OPERATIONS ON DOUBLY LINKED LIST*/
#include<stdio.h>
#include<stdlib.h>
struct node
{
   int data;
   struct node *prev;
   struct node *next;
};
struct node* INS_BEG(struct node *head, int item)
{
   struct node *node;
   node=(struct node*)malloc(sizeof(struct node));
   if(node==NULL)
   {
      printf("\nOverflow error");
      return head;
   }
   node->data=item;
   node->prev=NULL;
   node->next=head;
   if(head!=NULL)
      head->prev=node;
   head=node;
   return head;
}

struct node* INS_LAST(struct node *head, int item)
{
   struct node *node,*q;
   node=(struct node*)malloc(sizeof(struct node));
   if(node==NULL)
   {
      printf("\nOverflow error");
      return head;
```

```
    }
    node->data=item;
    node->next=NULL;
    if(head==NULL)
    {
        node->prev=NULL;
        head=node;
    }
    else
    {
        q=head;
        while(q->next!=NULL)
            q=q->next;
        q->next=node;
        node->prev=q;
    }
return head;
}


struct node* INS_SP_POS(struct node *head, int item)
{
    struct node *node,*q;
    int pos,i;
node=(struct node*)malloc(sizeof(struct node));
    if(node==NULL)
    {
        printf("\nOverflow error");
        return head;
    }
    node->data=item;
    if(head==NULL)
    {
        node->prev=NULL;
        node->next= NULL;
        head=node;
        return head;
```

```
    }
    printf("\nEnter position to insert:");
    scanf("%d",&pos);
    if(pos<0)
    {
        printf("\nInvalid position. Can't insert");
        return head;
    }

    if(pos==0)
    {
        head=INS_BEG(head,item);
        return head;
    }
        q=head;
        for(i=0;i<pos-1;i++)
        {
            q=q->next;
            if(q==NULL)
            {
                printf("\nInvalid position. Can't insert");
                return head;
            }
        }
        node->next=q->next;
        node->prev=q;
        q->next=node;
        if(node->next!=NULL)
        node->next->prev=q->next;


return head;
}


void DISPLAY(struct node *head)
{
```

```
    struct node *q;

    if(head==NULL)
        printf("\nList is empty.");
    else
    {
        printf("\nData in nodes are as follows:-\n\n");
        q=head;
        do
        {
            printf("%d\t",q->data);
            q=q->next;
        }while(q!=NULL);
    }
}

int main()
{
    struct node *head=NULL;
    int item,choice;

    while(1)
    {
        printf("\n\t\t\t*****INSERTION OPERATIONS ON  DOUBLY LINKED
LIST*****");
        printf("\n\t\t1.Insertion of node at begining.");
        printf("\n\t\t2.Insertion of node at last.");
        printf("\n\t\t3.Insertion of node at specific position");
        printf("\n\t\t4.Display");
        printf("\n\t\t5.Exit");
        printf("\n\nEnter your choice:");
        scanf("%d",&choice);
        if(choice>=1&&choice<=3)
        {
            printf("\nEnter the item:");
            scanf("%d",&item);
        }
```

```
    switch(choice)
    {
      case 1:
        head=INS_BEG(head,item);
      break;

      case 2:
        head=INS_LAST(head,item);
      break;

      case 3:
        head=INS_SP_POS(head,item);
      break;

      case 4:
        DISPLAY(head);
      break;

      case 5:
        printf("\n\t\t***GOOD BYE***");
        exit(0);

      default:
        printf("\n\nYou entered a wrong choice.");
    }
  }
  return 0;
}
```

## Deletion in doubly linked list from begining

## Algorithm

**DEL_BEG(head)**

**Step 1:** **[**Check for empty list?]
If head=NULL then
Write “List is empty”

return head
[End of If structure]

**Step 2:** [Getting address of first node]
Set q=head

**Step 3:** [Assigning address of second node to head]
Set head=head->next

**Step 4:** If head≠NULL then
head->prev=NULL

**Step 5:** [Delete the node from the front of list]
free(q)

**Step 6:** return head

## Deletion in doubly linked list from end

## Algorithm

**DEL_LAST(head)**

**Step 1:** [Check for empty list?]
If head=NULL then
Write "List is empty"
Return head
[End of If structure]

**Step 2:** [Getting address of last node]
Set q=head
Repeat while q->next≠NULL
Set q=q->next
[End of Step 2 loop]

**Step 3:** [Check for only one node]
If q=head then
Set head=NULL
Else
Set q->prev->next=q->next
[End of If structure]

**Step 4:** [Delete the node from the rear of list]
free(q)

**Step 5:** return head

## Deletion in singly linked list from specific position

## Algorithm

**DEL_SP_POS(head)**

```
Step 1:  [Check for empty list?]
         If head=NULL then
                  Write "List is empty"
                  Return head
         [End of If structure]

Step 2:  [Ask for position to delete]
         Write "Enter position to delete node"
         Read pos
Step 3: [Check for invalid position]
         If pos<0 then
                  Write "Invalid position."
                  Return head
         [End of If structure]
Step 4: If pos=0 then
                  Set head = Call DEL_BEG(head,item)
                  Return head
         [End of If structure]

Step 5:  [Traverse the list for specific position]
         Set q = head
Step 6:  Repeat for i = 0 to less than pos
                  Set p = q
                  Set q = q->next
                  [Check for invalid position]
                  If q = NULL then
                           Write "Invalid position."
                           Return head
                  [End of If structure]
         [End of Step 6 loop]
Step 7: Set p->next=q->next
Step 8:  If q->next≠NULL then
```

Set q->next->prev=p
[End of If structure]

**Step 9:** [Delete the node from specific position of list]
free(q)
**Step 10:** Return head

```
/*A PROGRAM TO PERFORM DELETION OPERATIONS ON DOUBLY LINKED LIST*/
#include<stdio.h>
#include<stdlib.h>
struct node
{
   int data;
   struct node *prev;
   struct node *next;
};

struct node* CREATE_LIST(struct node *head)
{
   struct node *node,*q;
   int item;
   char ans;

   do
   {
      printf("\nEnter item:");
      scanf("%d",&item);
      node=(struct node*)malloc(sizeof(struct node));
      node->data=item;
      node->next=NULL;

      if(head==NULL)
      {
         node->prev=NULL;
         head=node;
      }
```

```
      else
      {
         q=head;
         while(q->next!=NULL)
            q=q->next;
         q->next=node;
         node->prev=q;
      }

      printf("\nWant to add another[y/n]:");
      getchar();
      scanf("%c",&ans);
   }while(ans=='y'||ans=='Y');

return head;
}

struct node* DEL_BEG(struct node *head)
{
   struct node *q;

   if(head==NULL)
   {
      printf("\nList is empty.");
      return head;
   }

   q=head;
   head=head->next;
   if(head!=NULL)
         head->prev=NULL;

   free(q);
   return(head);
}

struct node* DEL_LAST(struct node *head)
```

```
{
   struct node *q;

   if(head==NULL)
   {
      printf("\nList is empty.");
      return head;
   }
   q=head;
   while(q->next!=NULL)
      q=q->next;
   if(q==head)
      head=NULL;
   else
      q->prev->next=q->next;
   free(q);
   return head;
}

struct node* DEL_SP_POS(struct node *head)
{
   struct node *p,*q;
   int pos,i;

   if(head==NULL)
   {
     printf("\nList is empty.");
     return head;
   }


      printf("\nEnter position to delete:");
      scanf("%d",&pos);
      if(pos<0)
      {
         printf("\nCan't delete");
         return head;
```

```
        }
        if(pos==0)
        {
            head=DEL_BEG(head);
            return head;
        }

        q=head;
        for(i=0;i<pos;i++)
        {
            p=q;
            q=q->next;
            if(q==NULL)
            {
                printf("\nCan't delete");
                return head;
            }

        }
            p->next=q->next;
            if(q->next!=NULL)
                q->next->prev=p;
        free(q);
    return head;
}


void DISPLAY(struct node *head)
{
    struct node *q;

    if(head==NULL)
        printf("\nList is empty.");
    else
    {
        printf("\nData in nodes are as follows:-\n\n");
        q=head;
```

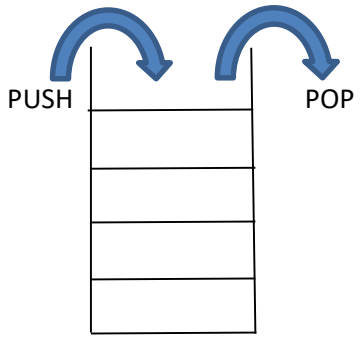
```
        while(q!=NULL)
        {
            printf("%d\t",q->data);
            q=q->next;
        }
    }
}

int main()
{
    struct node *head=NULL;
    int choice;
    while(1)
    {
        printf("\n\t\t\t*****DELETION OPERATION ON DOUBLY LINKED
LIST*****");
        printf("\n\t\t1.Insertion of nodes in linked list.");
        printf("\n\t\t2.Deletion of node from begining.");
        printf("\n\t\t3.Deletion of node from last.");
        printf("\n\t\t4.Deletion of node from specific position.");
        printf("\n\t\t5.Display");
        printf("\n\t\t6.Exit");
        printf("\n\nEnter your choice:");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1:
                head=CREATE_LIST(head);
            break;

            case 2:
                head=DEL_BEG(head);
            break;

            case 3:
                head=DEL_LAST(head);
            break;
```

```
        case 4:
           head=DEL_SP_POS(head);
        break;

        case 5:
           DISPLAY(head);
        break;
        case 6:
           printf("\n\t\t***GOOD BYE***");
           exit(0);
        default:
           printf("\nWrong choice.");
     }
  }
  return 0;
}
```

## STACK

A stack is a LIFO(Last In First Out) or FILO(First In Last Out) data structure in which items are inserted and deleted from only one end called Top of stack. In other words, a ***stack can be defined as a container in which insertion and deletion can be done from the one end known as the top of the stack.***

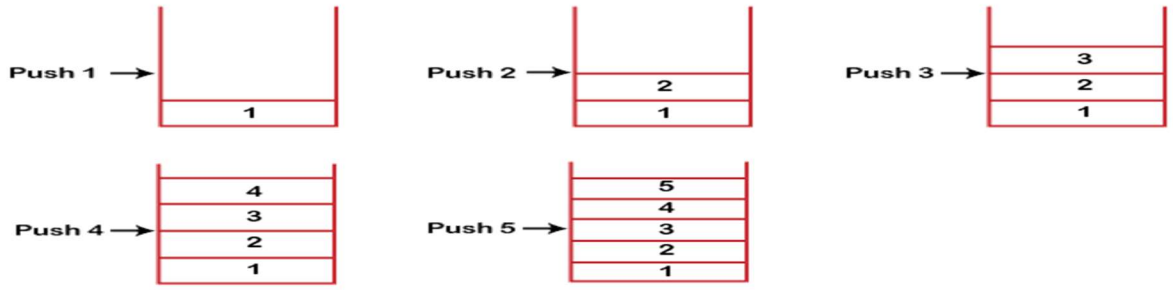
A stack is used in such a situation when we want to retrieve items in the reverse order of their storage.



## Working of Stack

Stack works on the LIFO principle. As we can observe in the below figure, the size of the stack is 5.

Suppose we want to store the items in a stack and let's assume that stack is empty. We have taken the stack of size 5 as shown below in which we are pushing the elements one by one until the stack becomes full.




When we perform the delete operation on the stack, It follows the LIFO pattern, which means that the value entered first will be removed last.

## Standard Stack Operations

**The following are some common operations implemented on the stack:**

- **Push():-** When we insert an item into the stack then the operation is known as a push. If the stack is full then the overflow error occurs.
- **Pop():-** When we delete an item from the stack, the operation is known as a pop. If the stack is empty the underflow error occurs.
- **isFull():-** It determines whether the stack is full or not.
- **isEmpty():-** It determines whether the stack is empty or not.
- **Peek():-** It returns the top item of stack without removing it.
- **Display():** It prints all the items available in the stack.

## Application of Stack

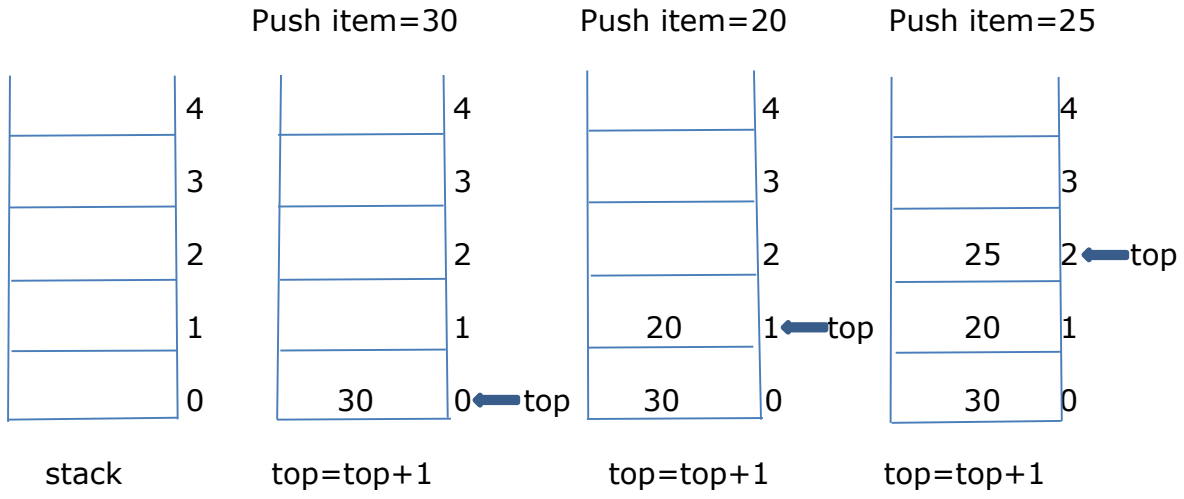
- Reversing a string

- Undo sequence in a text editor
- Balancing of symbols
- Infix-to-postfix conversion
- Evaluation of postfix expression
- Function calling
- Large number arithmetic

## PUSH operation

**The steps involved in the PUSH operation is given below:**

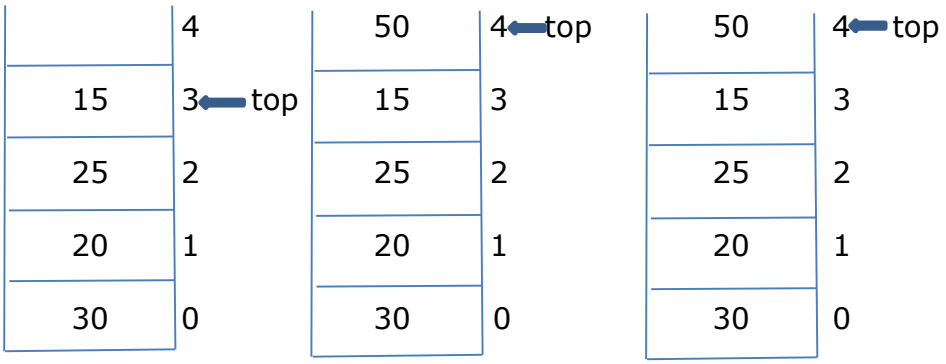
- Before inserting an item in a stack, we check whether the stack is full.
- If we try to insert the item in a stack which is full then the ***overflow*** error occurs.
- When we initialize a stack, we set the value of top as -1 to show that the stack is empty.
- When the new item is pushed in a stack, first the value of the top gets increased, i.e., **top=top+1,** and then the item will be placed at the new position of the **top**.
- The items will be inserted until we reach the ***MAXSIZE*** of the stack.



MAXSIZE=5 stack[top]=item stack[top]=item stack[top]=item

top=-1

Push item=15 Push item=50 Push item=60




top=top+1 top=MAXSIZE-1 top=MAXSIZE-1 so stack is full

stack[top]=item stack[top]=item and hence Overflow error occurs

## Push

## Algorithm

**PUSH(stack,top,item)**

**Step1:** **[**Is stack already full?**]**
If top=MAXSIZE-1 then
Write "Stack overflow error"
return
[End of if structure]
**Step 2:** **[**Increase top by 1**]**

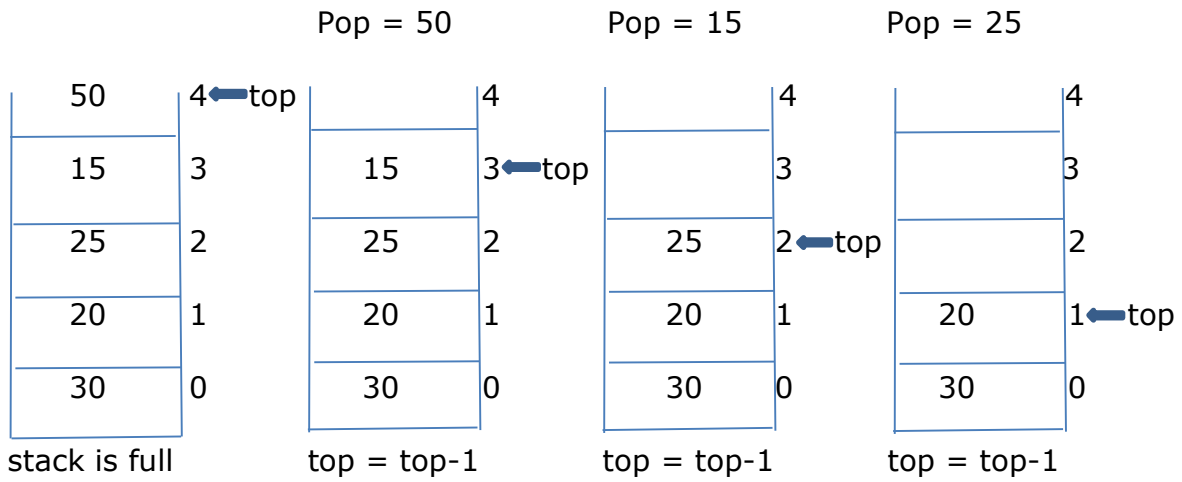
Set top = top+1

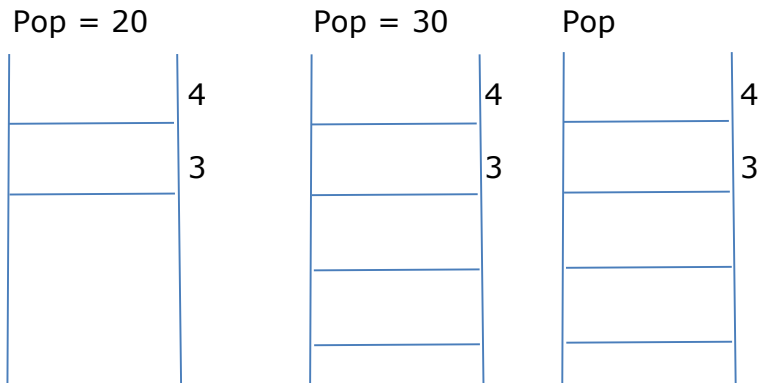
**Step 3:** **[**Insert item at top of stack**]**
Set stack[top]=item
**Step 4:** return

# POP operation

**The steps involved in the POP operation is given below**

- Before deleting the element from the stack, we check whether the stack is empty.
- If we try to delete the item from the stack which is empty then the ***underflow*** error occurs.
- If the stack is not empty, we first access the element which is pointed by ***top***
- Once the pop operation is performed, the top is decreased by 1 i.e. **top=top-1**.



| | | | |
|---|---|---|---|
| | 2 | 2 | 2 |
| | 1 | 1 | 1 |
| 30 | 0 ← top | 0 | 0 |
| top = top-1 | top=-1 | top=-1 so stack is empty and hence underflow error occurs | |

## Pop

## Algorithm

**POP(stack,top)**

**Step1:** **[**Is stack already empty?**]**
If top=-1 then
Write "Stack underflow error"
return
[End of if structure]

**Step 2:** **[**Store top most item of stack**]**

Set delitem = stack[top]

**Step 3:** **[**Decrease top by 1**]**
Set top = top-1

**Step 4:** return delitem

```
/*A PROGRAM TO PERFORM OPERATIONS ON STACK USING ARRAY(Local
variable)*/
#include<stdio.h>
#include<stdlib.h>
#define MAXSIZE 5

void PUSH(int stack[],int *top,int item)
{
   if(*top==MAXSIZE-1)
      printf("Stack overflow error.");
```

```
    else
    {
        *top=*top+1;
        stack[*top]=item;
        printf("\n%d is pushed in stack.",item);
    }
}

int POP(int stack[],int *top)
{
    int delitem;

    if(*top==-1)
    {
        printf("Stack underflow error.");
        return -1;
    }

        delitem=stack[*top];
        *top=*top-1;
        return delitem;
}

int PEEK(int stack[],int top)
{
    if(top==-1)
    {
        printf("Stack underflow error.");
        return -1;
    }
    return stack[top];
}

void DISPLAY(int stack[],int top)
{
    int i;
```

```
    if(top==-1)
        printf("Stack is empty.");
    else
    {
        printf("Items of Stack are\n");
        for(i=top;i>=0;i--)
            printf("%d\n",stack[i]);
    }
}

int main()
{
    int stack[MAXSIZE];
    int top=-1;
    int item,choice;

    while(1)
    {
        printf("\n\t\t\t_ _ _ _ OPERATIONS ON STACK _ _ _ _\n");
        printf("\n\n\t\t\t1. PUSH");
        printf("\n\n\t\t\t2. POP");
        printf("\n\n\t\t\t3. PEEK");
        printf("\n\n\t\t\t4. DISPLAY");
        printf("\n\n\t\t\t5. EXIT");
        printf("\n\nEnter your choice:");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1:
                printf("\nEnter the item to be pushed:");
                scanf("%d",&item);
                PUSH(stack,&top,item);
            break;

            case 2:
                item=POP(stack,&top);
                if(item!=-1)
```

```
            printf("\n%d is popped from stack",item);
        break;

        case 3:
            item=PEEK(stack,top);
            if(item!=-1)
            printf("Top item of stack = %d",item);
        break;

        case 4:
            DISPLAY(stack,top);
        break;

        case 5:
            printf("\n\n\t\t\t_ _ _ Good Bye _ _ _");
            exit(0);

        default:
            printf("\nWrong choice");
      }
   }
   return 0;
}

/*A PROGRAM TO PERFORM OPERATIONS ON STACK USING ARRAY(Global
variable)*/
#include<stdio.h>
#include<stdlib.h>
#define MAXSIZE 5

int stack[MAXSIZE];
int top=-1;

int isFULL(void)
{
   if(top==MAXSIZE-1)
      return 1;
```

```
   return 0;
}

int isEMPTY(void)
{
   if(top==-1)
      return 1;
   return 0;
}

void PUSH(int item)
{
   if(isFULL())
      printf("Stack overflow error.");
   else
   {
      top=top+1;
      stack[top]=item;
      printf("\n%d is pushed in stack.",item);
   }
}

int POP(void)
{
   int delitem;

   if(isEMPTY())
   {
      printf("Stack underflow error.");
      return -1;
   }
      delitem=stack[top];
      top=top-1;
      return delitem;
}

int PEEK(void)
```

```
{
   if(isEMPTY())
   {
      printf("Stack underflow error.");
      return -1;
   }

   return stack[top];
}

void DISPLAY(void)
{
   int i;

   if(isEMPTY())
      printf("Stack is empty.");
   else
   {
      printf("Items of Stack are\n");
      for(i=top;i>=0;i--)
         printf("%d\n",stack[i]);
   }
}

int main()
{
   int item,choice;

   while(1)
   {

      printf("\n\t\t\t_ _ _ _ OPERATIONS ON STACK _ _ _ _\n");
      printf("\n\n\t\t\t1. PUSH");
      printf("\n\n\t\t\t2. POP");
      printf("\n\n\t\t\t3. PEEK");
      printf("\n\n\t\t\t4. DISPLAY");
      printf("\n\n\t\t\t5. EXIT");
```

```
    printf("\n\nEnter your choice:");
    scanf("%d",&choice);
    switch(choice)
    {
      case 1:
        printf("\nEnter the item to be pushed:");
        scanf("%d",&item);
        PUSH(item);
      break;

      case 2:
       item=POP();
        if(item!=-1)
        printf("\n%d is popped from stack",item);
      break;

      case 3:
         item=PEEK();
        if(item!=-1)
        printf("Top item of stack = %d",item);
      break;

      case 4:
        DISPLAY();
      break;

      case 5:
        printf("\n\n\t\t\t_ _ _ Good Bye _ _ _");
        exit(0);

      default:
        printf("\nWrong choice");
    }
  }
  return 0;
}
```

## Stack Using Linked List

We know that stack is a special type of data structure where elements are inserted and deleted from the same end. That is, if an element is inserted at front end, an element has to be deleted from front end. If an element is inserted at rear end, an element has to be deleted from the rear end. Thus, a stack can be implemented using the following functions:

- INS_BEG()
- DEL_BEG()
- DISPLAY()

OR

- INST_LAST()
- DEL_LAST()
- DISPLAY()

```
/*A PROGRAM TO PERFORM OPERATIONS ON STACK USING LINKED LIST*/
#include<stdio.h>
#include<stdlib.h>

struct node
{
int data;
struct node *next;
};
struct node *top=NULL;

void PUSH(int item)
{
   struct node *node;
   node=(struct node*)malloc(sizeof(struct node));
   if(node==NULL)
      printf("\nStack overflow error");
   else
      {
         node->data=item;
         node->next=top;
         top=node;
```

```
        printf("\n%d is pushed in stack",item);
    }
}

void POP(void)
{
struct node *q;
if(top==NULL)
  printf("\nstack underflow error.");
else
  {
    q=top;
    top=top->next;
    printf("\n%d is popped from stack",q->data);
    free(q);
  }
}


void DISPLAY(void)
{
struct node *q;
if(top==NULL)
  printf("\nstack is empty.");
else
  {
    printf("\nStack items are as follows:-\n\n");
    q=top;
    while(q!=NULL)
      {
        printf("%d\n",q->data);
        q=q->next;
      }
  }
}

int main()
```

```
{
int item,choice;

while(1)
{
printf("\n\t\t\t*****OPERATION ON STACK*****\n\n\t\t\t\t");
printf("1.PUSH\n\n\t\t\t\t");
printf("2.POP\n\n\t\t\t\t");
printf("3.DISPLAY\n\n\t\t\t\t");
printf("4.EXIT\n\n");
printf("Enter your choice:");
scanf("%d",&choice);
switch(choice)
{
case 1:
printf("\nEnter the item:");
scanf("%d",&item);
PUSH(item);
break;
case 2:
POP();
break;
case 3:
DISPLAY();
break;
case 4:
exit(0);
default:
printf("\n\nYou entered a wrong choice.");
}
}
return 0;
}
```

## Representaion of arithmetic expression

An arithmetic expression can be represented in one of the following notation-

**(i) Infix notation**

In an expression, if an operator is in between two operands, the expression is called an infix expression. The infix expression can be parenthesized or un-parenthesized.

For example,

**a + b** is an un-parenthesized infix expression

(**a + b**) is a parenthesized infix expression

**(ii) Prefix notation**

In an expression, if an operator precedes the two operands (i.e., operator comes before the two operands), the expression is called prefix expression. No, parenthesis is allowed in prefix expressions. The prefix expression is always un-parenthesized.

It is also called polish expression.

For example,

**+ a b**

**(iii) Postfix notation**

In an expression, if an operator follows the two operands (i.e., operator comes after the two operands), the expression is called postfix expression. No, parenthesis is allowed in postfix expressions. The postfix expression is always un-parenthesized.

It is also called suffix expression or reverse polish expression.

For example,

**a b +**

## Conversion from infix to postfix

**Q.** Convert the infix expression A+B*C into postfix expression.

**Solution:**

**Given**

A+B*C (Infix)

A+(B*C)

A+(BC*)

A+D Where D = (BC*)

AD+

Now expanding AD+

ABC*+ (Postfix)

**Q.** Convert the infix expression (A+B)*C into postfix expression.

**Solution:**

**Given**

(A+B)*C (Infix)

(AB+)*C

D*C Where D = (AB+)

DC*

Now expanding DC*

AB+C* (Postfix)

**Q.** Convert the infix expression A*B+C/D into postfix expression.

**Solution:**
**Given**
A*B+C/D (Infix)
(A*B)+C/D
(AB*)+C/D

E+C/D Where E=(AB*)
E+(C/D)
E+(CD/)
E+F Where F=(CD/)
EF+
Now expanding EF+
AB*F+
AB*CD/+ (Postfix)

**Q.** Convert the infix expression (A+B)*C/D+E^F/G into postfix expression.

**Solution:**
**Given**
(A+B)*C/D+E^F/G (Infix)
(AB+)*C/D+E^F/G
H*C/D+E^F/G Where H=(AB+)
H*C/D+(E^F)/G
H*C/D+(EF^)/G
H*C/D+I/G Where I=(EF^)
(H*C)/D+I/G
(HC*)/D+I/G
J/D+I/G Where J=(HC*)
(J/D)+I/G
(JD/)+I/G
K+I/G Where K=(JD/)
K+(I/G)
K+(IG/)
K+L Where L=(IG/)
KL+
Now expanding KL+
JD/L+
HC*D/L+
AB+C*D/L+
AB+C*D/IG/+
AB+C*D/EF^G/+ (Postfix)

**Q.** Convert the infix expression A+(B*C-(D/E^F)*G)*H into postfix expression.

**Solution:**
**Given**
A+(B*C-(D/E^F)*G)*H (Infix)

A+(B*C-(D/(E^F))*G)*H
A+(B*C-(D/(EF^))*G)*H
A+(B*C-(D/I)*G)*H Where I=(EF^)
A+(B*C-(DI/)*G)*H
A+(B*C-J*G)*H Where J=(DI/)
A+((B*C)-J*G)*H
A+((BC*)-J*G)*H
A+(K-J*G)*H Where K=(BC*)
A+(K-(J*G))*H
A+(K-(JG*))*H
A+(K-L)*H Where L=(JG*)
A+(KL-)*H
A+M*H Where M=(KL-)
A+(M*H)
A+(MH*)
A+N Where N=(MH*)
AN+
Now expanding AN+
AMH*
AKL-H*
ABC*L-H*
ABC*JG*-H*
ABC*DI/G*-H*
ABC*DEF^/G*-H* (Postfix)

## Converting Infix Expression into Postfix Expression

## Algorithm

**POLISH(Q,P)**

**Suppose Q is an arithmetic expression written in infix notation. This algorithm finds the equivalent postfix expression P.**

**Step1:** Push "(" onto STACK and add ")" to the end of Q.
**Step 2:** Scan Q from left to right and repeat Steps 3 to 6 for each element of Q until the STACK is empty:

**Step 3:** If an operand is encountered, add it to P.
**Step 4:** If a left parenthesis is encountered, push it onto STACK.
**Step 5:** If an operator × is encountered then:

(a) Repeatedly pop from STACK and add to P each operator(on the top of STACK) which has the same precedence as or higher precedence than ×.

(b) Add × to STACK

[End of If structure]

**Step 6:** If a right parenthesis is encountered then:

(a) Repeatedly pop from STACK and add to P each operator(on the top of STACK) until a left parenthesis is encountered.

(b) Remove the left parenthesis.

[End of If structure]

[End of Step 2 loop]

**Step 7:** return

**Q.** Convert the infix expression A+(B*C-(D/E^F)*G)*H into postfix expression.

**Solution:** Suppose Q is an arithmetic expression written in infix notation.

Q: A+(B*C-(D/E^F)*G)*H

We add right parenthesis "(" at the end of Q and push left parenthesis "(" onto STACK.

Q: A+(B*C-(D/E^F)*G)*H)

| Symbol Scanned | STACK | Expression P |
|---|---|---|
| | ( | |
| A | ( | A |
| + | (+ | A |
| ( | (+( | A |
| B | (+( | AB |
| * | (+(* | AB |
| C | (+(* | ABC |
| - | (+(- | ABC* |
| ( | (+(-( | ABC* |
| D | (+(-( | ABC*D |
| / | (+(-(/ | ABC*D |
| E | (+(-(/ | ABC*DE |
| ^ | (+(-(/^ | ABC*DE |
| F | (+(-(/^ | ABC*DEF |
| ) | (+(- | ABC*DEF^/ |
| * | (+(-* | ABC*DEF^/ |
| G | (+(-* | ABC*DEF^/G |
| ) | (+ | ABC*DEF^/G*- |

| | | |
|---|---|---|
| * | (+* | ABC*DEF^/G*- |
| H | (+* | ABC*DEF^/G*-H |
| ) | | ABC*DEF^/G*-*+ |

**/*A PROGRAM TO INPUT INFIX EXPRESSION AND CONVERT IT INTO POSTFIX EXPRESSION*/**

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#define MAXSIZE 20

char stack[MAXSIZE];
int top=-1;

int isFULL(void)
{
   if(top==MAXSIZE-1)
      return 1;
   return 0;
}

int isEMPTY(void)
{
   if(top==-1)
      return 1;
   return 0;
}

void PUSH(char sym)
{
   if(isFULL())
   {
      printf("\nStack overflow error.");
      exit(1);
   }
   top=top+1;
   stack[top]=sym;
}

char POP(void)
{
   char delitem;
```

```
    if(isEMPTY())
    {
        printf("Stack underflow error.");
        exit(1);
    }
    delitem=stack[top];
    top=top-1;
    return delitem;
}

int PRECED(char ch)
{
    if(ch=='+'||ch=='-')
        return 1;
    if(ch=='*'||ch=='/')
        return 2;
    if(ch=='^')
        return 3;
    return 0;
}


void INFIX_POST(char infix[])
{
    char postfix[25];
    char symbol,temp;
    int i=0,j=0;

    while(top!=-1)
    {
        symbol=infix[i];
        switch(symbol)
        {
            case '(':
                PUSH(symbol);
            break;

            case ')':
            temp=POP();
            while(temp!='(')
            {
                postfix[j]=temp;
                j++;
                temp=POP();
            }
```

```
            break;

            case '+':
            case '-':
            case '*':
            case '/':
            case '^':
            while(PRECED(stack[top])>=PRECED(symbol))
            {
               temp=POP();
               postfix[j]=temp;
               j++;
            }
            PUSH(symbol);
            break;

            default:
            postfix[j]=symbol;
            j++;
         }
         i++;
      }
      postfix[j]='\0';
      printf("\nPostfix expression = %s",postfix);
}

int main()
{
   char infix[25];
   int len;

   printf("Enter any valid infix expression:");
   scanf("%s",infix);
   len=strlen(infix);
   PUSH('(');
   infix[len]=')';
   infix[len+1]='\0';
   INFIX_POST(infix);
   return 0;
}
```

## Evaluation of a Postfix Expression

### Algorithm

**Suppose P is an arithmetic expression written in postfix notation. This algorithm finds the value of P.**

**Step1:** Add a right parenthesis “)” at the end of P.
**Step 2:** Scan P from left to right and repeat Steps 3 and 4 for each element of P until the sentinel “)” is encountered.

**Step 3:** If an operand is encountered, put it on STACK.
**Step 4:** If an operator × is encountered then:

(a) Remove the two top elements of STACK where A is the top element and B is the next-to-top element.
(b) Evaluate B×A.
(c) Place the result of (b) back on STACK.

[End of If structure]
[End of Step 2 loop]

**Step 5:** Set VALUE equal to the top element on STACK.

**Q.** Convert the infix expression 5*(6+2)-12/4 into postfix and evaluate it.

**Solution:**
**Given**
5*(6+2)-12/4
5*(6 2 +)-12/4
5*A-12/4 where A = (6 2 +)
(5*A)-12/4
(5A*)-12/4
B-12/4 where B = (5A*)
B-(12/4)
B-(12 4 /)
B-C where C = (12 4 /)
BC-

Now expanding BC-
5A*C-
5 6 2 + * C –
5 6 2 + * 12 4 / - (Postfix)

**Evaluation of Postfix Expression**

Suppose P is an arithmetic expression written in infix notation.

P: 5 6 2 + * 12 4 / -

We add right parenthesis “)” at the end of P.

P: 5 6 2 + * 12 4 / -)

| Symbol Scanned | STACK |
|---|---|
| 5 | 5 |
| 6 | 5,6 |
| 2 | 5,6,2 |
| + | 5,8 |
| * | 40 |
| 12 | 40,12 |
| 4 | 40,12,4 |
| / | 40,3 |
| - | 37 |
| ) | |

**/*A PROGRAM TO INPUT POSTFIX EXPRESSION AND FIND ITS VALUE*/**

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<ctype.h>
#include<math.h>
#define MAXSIZE 20

float stack[MAXSIZE];
int top=-1;

int isFULL(void)
{
```

```
   if(top==MAXSIZE-1)
      return 1;
   return 0;
}

int isEMPTY(void)
{
   if(top==-1)
      return 1;
   return 0;
}

void PUSH(float val)
{
   if(isFULL())
      {
         printf("Stack overflow error.");
         exit(1);
      }
      top=top+1;
      stack[top]=val;
}

float POP(void)
{
   float delitem;

   if(isEMPTY())
   {
      printf("Stack underflow error.");
      exit(1);
   }
   delitem=stack[top];
   top=top-1;
   return delitem;
}

float POST_EVAL(char postfix[])
{
   float value=0;
   float A,B,res;
   char symbol;
   int i=0;
   while(postfix[i]!=')')
   {
```

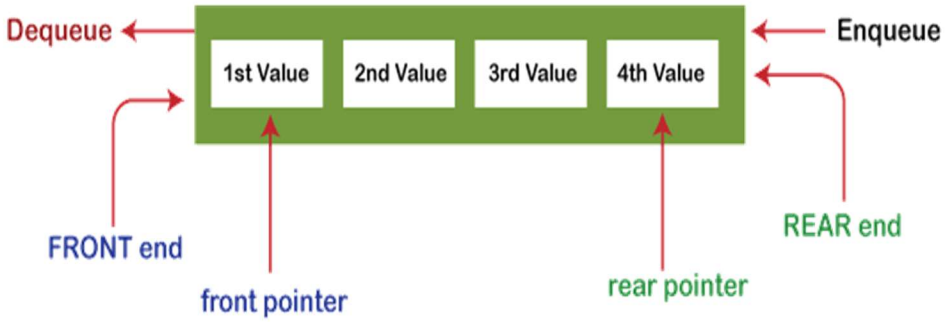
```
        symbol=postfix[i];
        switch(symbol)
        {
        case '+':
        case '-':
        case '*':
        case '/':
        case '^':
        A=POP();
        B=POP();
        if(symbol=='+')
           res=B+A;
        else if(symbol=='-')
           res=B-A;
        else if(symbol=='*')
           res=B*A;
        else if(symbol=='/')
           res=(float)B/A;
        else
           res=pow(B,A);
        PUSH(res);
        break;
default:
if(isdigit(symbol))
    {
        do
        {
        value=value*10+(symbol-48);
        i++;
        symbol=postfix[i];
        }while(symbol!=' ');
        PUSH(value);
        value=0;
    }
}
i++;
}
value=POP();
return value;
}

int main()
{
char postfix[25];
int len;
```

```
printf("Enter any valid postfix expression:");
fgets(postfix,25,stdin);
len=strlen(postfix);
postfix[len]=')';
postfix[len+1]='\0';
printf("\nPostfix value = %f",POST_EVAL(postfix));
return 0;
}
```

# Queue

A queue is a FIFO(First In First Out) or LILO(Last In Last Out) data structure in which items are inserted at one end called REAR end or the tail of the queue whereas items are deleted from another end called FRONT end or the head of the queue. In other words, it can be defined as a list or a collection with a constraint that the insertion can be performed at one end called the rear end or tail of the queue and deletion is performed on another end called the front end or the head of the queue.

A queue is used in such a situation when we want to retrieve items in the same order as they were inserted.




## Standard Queue Operations

**The following are some common operations implemented on the queue:**

- **Enqueue():-** When we insert an item into the queue then the operation is known as enqueue. If the queue is full then the overflow error occurs.
- **Dequeue():-** When we delete an item from the queue, the operation is known as dequeue. If the queue is empty than the underflow error occurs.
- **isFull():-** It determines whether the queue is full or not.
- **isEmpty():-** It determines whether the queue is empty or not.
- **Peek():-** It returns the front item of queue without removing it.
- **Display():** It prints all the items available in the queue.

A Queue can be represented as a container opened from both the sides in which the element can be enqueued from one side and dequeued from another side

# Applications of Queue

Due to the fact that queue performs actions on first in first out basis which is quite fair for the ordering of actions. There are various applications of queues discussed as below.

- Lines at a ticket counter or any other first come first served scenario requires a queue.
- Queues are widely used as waiting lists for a single shared resource like printer, disk, CPU.
- Queues are used in asynchronous transfer of data (where data is not being transferred at the same rate between two processes) for ex. pipes, file IO, sockets.

# Types of Queue

**There are four types of Queues**

1. Linear or Ordinary Queue
2. Circular Queue
3. Double Ended Queue(DEQue)
4. Priority Queue

## Insertion in Linear Queue

## Algorithm

**ENQUEUE(queue,front,rear,item,MAXSIZE)**

**Step1:** **[**Is queue already full?**]**

```
If rear=MAXSIZE-1 then
  Write "Queue overflow error"
  return
[End of if structure]
```

**Step 2:** **[**Finding value of rear**]**

If front = -1 then

Set front = 0 and rear = 0

Else

Set rear = rear+1

[End of If structure]

**Step 3:** **[**Insert item at rear end of queue**]**
Set queue[rear]=item
**Step 4:** return

## Deletion from Linear Queue

## Algorithm

**DEQUEUE(queue,front,rear)**

**Step1:** **[**Is queue already empty?**]**
If front = -1 then
Write "Queue underflow error"
return
[End of if structure]
**Step 2:** **[**Store front item of queue**]**

Set delitem = queue[front]

**Step 3:** **[**Find new value of front**]**
If front=rear then
Set front = -1 and rear = -1
Else
Set front = front+1
[End of If structure]
**Step 4:** return delitem

```
            /*A PROGRAM TO PERFORM OPERATIONS ON LINEAR QUEUE*/
#include<stdio.h>
#include<stdlib.h>
#define MAXSIZE 5

int queue[MAXSIZE];
int front=-1,rear=-1;

int isFULL(void)
{
   if(rear==MAXSIZE-1)
      return 1;
   return 0;
}

int isEMPTY(void)
{
   if(front==-1)
      return 1;
   return 0;
}


void ENQUEUE(int item)
{
   if(isFULL())
      printf("\nQueue Overflow error.");
   else
   {
      if(isEMPTY())
         front=rear=0;
      else
         rear=rear+1;
      queue[rear]=item;
      printf("\n%d is inserted in Queue",item);
   }
```

```
}

int DEQUEUE(void)
{
   int delitem;

   if(isEMPTY())
   {
      printf("Queue underflow error.");
      return -1;
   }

   delitem=queue[front];
   if(front==rear)
      front=rear=-1;
   else
      front=front+1;

   return delitem;
}

int PEEK(void)
{
   if(isEMPTY())
   {
      printf("Queue underflow error.");
      return -1;
   }
   return queue[front];
}

void DISPLAY(void)
{
   int i;

   if(isEMPTY())
      printf("Queue is empty.");
```

```
    else
    {
        printf("Items of Queue are\n");
        for(i=front;i<=rear;i++)
            printf("%d\t",queue[i]);
    }
}


int main()
{
    int item,choice;

    while(1)
    {

        printf("\n\t\t\t_ _ _ _ OPERATIONS ON LINEAR QUEUE _ _ _ _\n");
        printf("\n\n\t\t\t1. INSERTION");
        printf("\n\n\t\t\t2. DELETION");
        printf("\n\n\t\t\t3. PEEK");
        printf("\n\n\t\t\t4. DISPLAY");
        printf("\n\n\t\t\t5. EXIT");
        printf("\n\nEnter your choice:");
        scanf("%d",&choice);

        switch(choice)
        {
        case 1:
            printf("\nEnter the item:");
            scanf("%d",&item);
            ENQUEUE(item);
            break;

        case 2:
            item=DEQUEUE();
            if(item!=-1)
                printf("\n%d is deleted from Queue",item);
```

```
        break;

      case 3:
        item=PEEK();
        if(item!=-1)
          printf("\nItem at Front of Queue = %d",item);
        break;

      case 4:
        DISPLAY();
      break;
      case 5:
        printf("\n\n\t\t\t_ _ _ Good Bye _ _ _");
        exit(0);

      default:
        printf("\nWrong choice.");
      }
    }
    return 0;
}
```

## Disadvantage of Linear Queue

When we insert an item into a linear queue, rear is incremented by 1. Once the rear is set to MAXSIZE-1, we say that queue is full and hence no more items can be inserted.

If we delete some items from queue then rear still remains set at MAXSIZE-1 so we can’t insert more items even though the space is available in linear queue.

## Insertion in Circular Queue

## Algorithm

**ENQUEUE(queue,front,rear,item,MAXSIZE)**

**Step1:** **[**Is queue already full?**]**

```
If front = 0 and rear = MAXSIZE-1 OR front = rear+1 then
  Write "Queue overflow error"
  return
[End of if structure]
```

**Step 2:** **[**Finding value of rear**]**

```
If front = -1 then

        Set front = 0 and rear = 0

Else

        Set rear = (rear+1)%MAXSIZE

[End of If structure]
```

**Step 3:** **[**Insert item at rear end of queue**]**

```
Set queue[rear]=item
```

**Step 4:** return

## Deletion from Circular Queue

## Algorithm

**DEQUEUE(queue,front,rear,MAXSIZE)**

**Step1:** **[**Is queue already empty?**]**

```
If front = -1 then
  Write "Queue underflow error"
  return
[End of if structure]
```

**Step 2:** **[**Store front item of queue**]**

Set delitem = queue[front]

**Step 3:** [Find new value of front]
If front=rear then
Set front = -1 and rear = -1
Else
Set front = (front+1)%MAXSIZE
[End of If structure]

**Step 4:** return delitem

```
/*A PROGRAM TO PERFORM OPERATIONS ON CIRCULAR QUEUE*/

#include<stdio.h>

#include<stdlib.h>

#define MAXSIZE 5


int queue[MAXSIZE];

int front=-1,rear=-1;


int isFULL(void)

{

   if((front==0&&rear==MAXSIZE-1)||(front==rear+1))

      return 1;

   return 0;

}


int isEMPTY(void)

{

   if(front==-1)
```

```
        return 1;
    return 0;
}

void ENQUEUE(int item)
{
    if(isFULL())
        printf("\nQueue Overflow error.");
    else
    {
        if(isEMPTY())
            front=rear=0;
        else
            rear=(rear+1)%MAXSIZE;
    queue[rear]=item;
    printf("\n%d is inserted in Queue",item);

    }
}

int DEQUEUE(void)
{
    int delitem;

    if(isEMPTY())
    {
```

```
        printf("Queue underflow error.");
        return -1;
    }
    delitem=queue[front];
    if(front==rear)
        front=rear=-1;
    else
        front=(front+1)%MAXSIZE;
    return delitem;
    }

int PEEK(void)
{
    if(isEMPTY())
    {
        printf("Queue underflow error.");
        return -1;
    }
    return queue[front];
}

void DISPLAY(void)
{
    int i;

    if(isEMPTY())
```

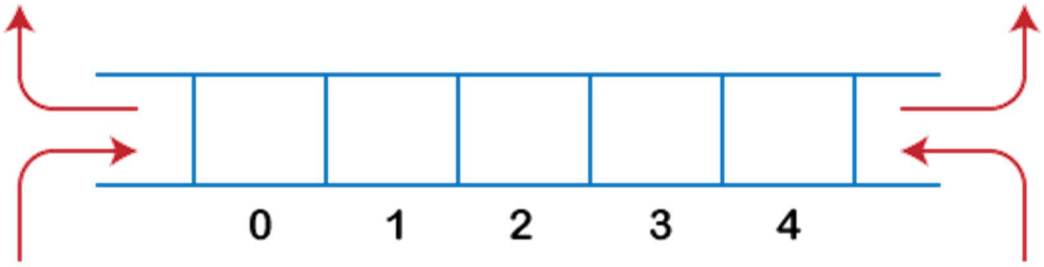
```
    {
        printf("\nQueue is empty.");
        return;
    }

    printf("Items of Queue are\n");
    if(front<=rear)
    {
        for(i=front;i<=rear;i++)
            printf("%d\t",queue[i]);
    }
    else
    {
        for(i=front;i<=MAXSIZE-1;i++)
            printf("%d\t",queue[i]);
        for(i=0;i<=rear;i++)
            printf("%d\t",queue[i]);
    }

}


int main()
{
    int item,choice;
```

```
while(1)
{
    printf("\n\t\t\t_ _ _ _ OPERATIONS ON CIRCULAR QUEUE _ _ _ _\n");
    printf("\n\n\t\t\t1. INSERTION");
    printf("\n\n\t\t\t2. DELETION");
    printf("\n\n\t\t\t3. PEEK");
    printf("\n\n\t\t\t4. DISPLAY");
    printf("\n\n\t\t\t5. EXIT");
    printf("\n\nEnter your choice:");
    scanf("%d",&choice);

    switch(choice)
    {
    case 1:
        printf("\nEnter the item:");
        scanf("%d",&item);
        ENQUEUE(item);
        break;

    case 2:
        item=DEQUEUE();
        if(item!=-1)
            printf("\n%d is deleted from Queue",item);
        break;

    case 3:
```

```
        item=PEEK();
        if(item!=-1)
            printf("\nItem at Front of Queue = %d",item);
        break;

    case 4:
        DISPLAY();
    break;
    case 5:
        printf("\n\n\t\t\t_ _ _ Good Bye _ _ _");
        exit(0);

    default:
        printf("\nWrong choice.");
    }
  }
  return 0;
}
```

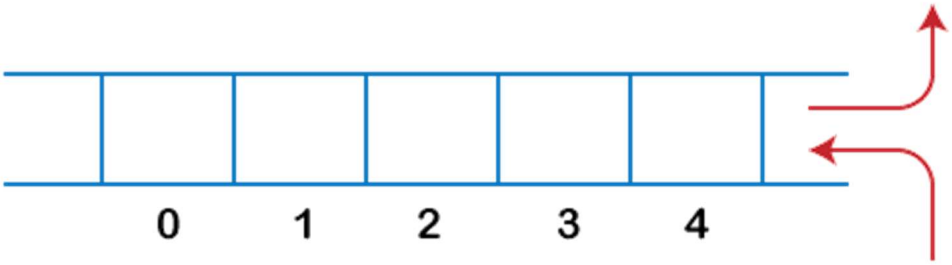
## Double Ended Queue(Deque)

The Deque stands for **Double Ended Queue** in which the items can be inserted and deleted from both ends i.e insertion and deletion of items can be performed from both front and rear end.

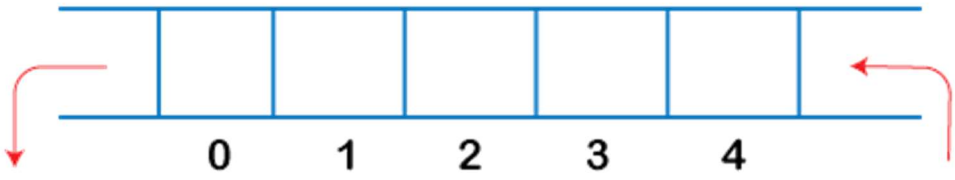



Deque can be used both as **stack** and **queue** as it allows the insertion and deletion operations on both ends.

The stack follows the LIFO rule in which both the insertion and deletion can be performed only from one end**.** In deque, the insertion and deletion operation can be performed from one side. Therefore, **we conclude that deque can be considered as a stack**.



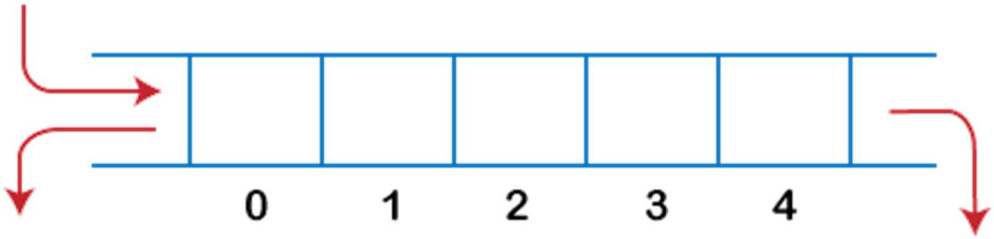

The queue follows the FIFO rule in which the element is inserted on one end and deleted from another end. In deque, the insertion can be performed on one end, and the deletion can be done on another end. Therefore, we conclude that the deque can also be considered as the queue.



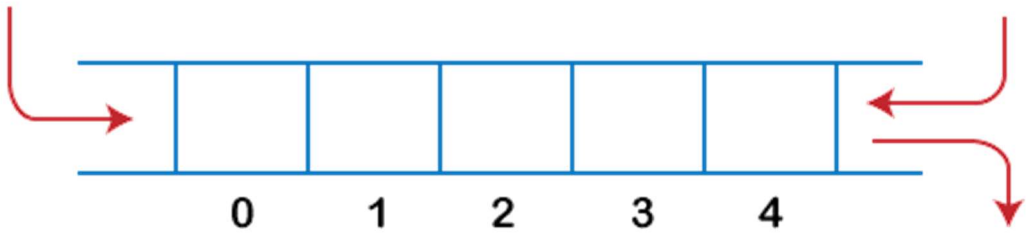

There are two types of Queues, **Input-restricted queue**, and **output-restricted queue**.

1. **<u>Input-restricted queue:-</u>**

   The input-restricted queue means that some restrictions are applied to the insertion. In input-restricted queue, the insertion is applied to one end while the deletion is applied from both the ends.




2. **<u>Output-restricted queue:-</u>**
   The output-restricted queue means that some restrictions are applied to the deletion operation. In an output-restricted queue, the deletion can be applied only from one end, whereas the insertion is possible from both ends.




## Operations on Deque:-

**The following are the operations applied on deque:**

- **Insert at front**
- **Delete from front**
- **Insert at rear**
- **Delete from rear**

Other than insertion and deletion, we can also perform **peek** operation in deque. Through **peek** operation, we can get the **front** and the **rear** element of the dequeue.

# Applications of Deque

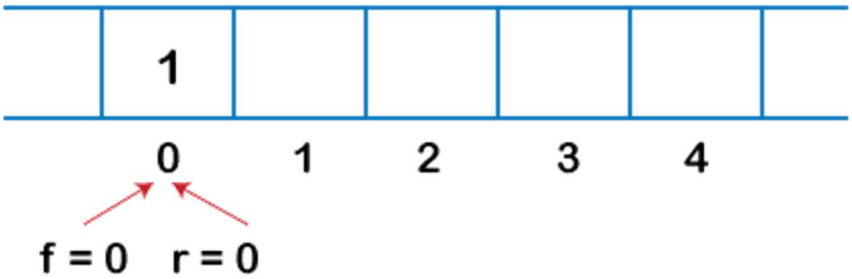
- The deque can be used as a **stack** and **queue**; therefore, it can perform both redo and undo operations.
- It can be used as a palindrome checker means that if we read the string from both ends, then the string would be the same.
- It can be used for multiprocessor scheduling. Suppose we have two processors, and each processor has one process to execute. Each processor is assigned with a process or a job, and each process contains multiple threads. Each processor maintains a deque that contains threads that are ready to execute. The processor executes a process, and if a process creates a child process then that process

will be inserted at the front of the deque of the parent process. Suppose the processor $P_2$ has completed the execution of all its threads then it steals the thread from the rear end of the processor $P_1$ and adds to the front end of the processor $P_2$. The processor $P_2$ will take the thread from the front end; therefore, the deletion takes from both the ends, i.e., front and rear end. This is known as the **A-steal algorithm** for scheduling.

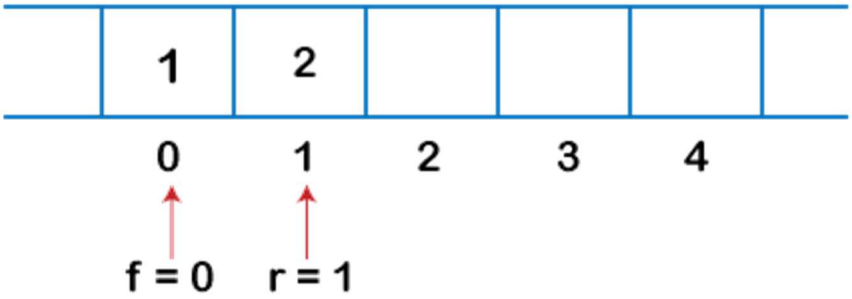
**The following are the steps to perform the operations on the Deque:**

## Enqueue operation

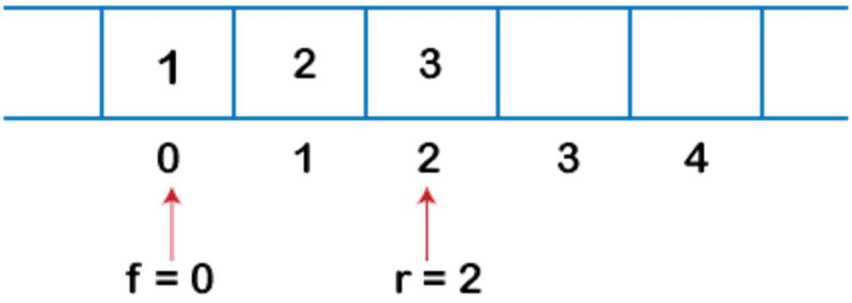
1. Initially, we are considering that the deque is empty, so both front and rear are set to -1, i.e., **f = -1** and **r = -1**.
2. As the deque is empty, so inserting an element either from the front or rear end would be the same thing. Suppose we have inserted element 1, then **front is equal to 0,** and the **rear is also equal to 0.**



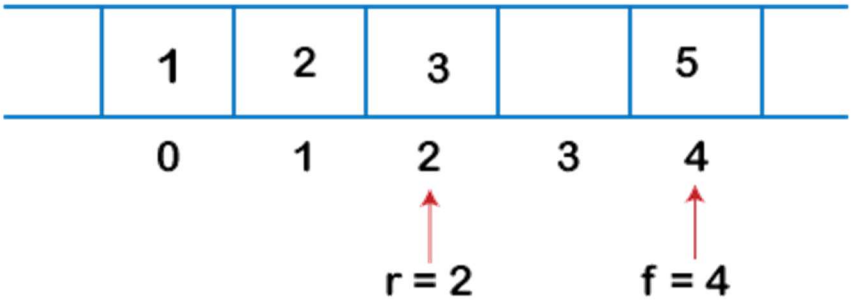

3. Suppose we want to insert the next element from the rear. To insert the element from the rear end, we first need to increment the rear, i.e., **rear=rear+1**. Now, the rear is pointing to the second element, and the front is pointing to the first element.




4. Suppose we are again inserting the element from the rear end. To insert the element, we will first increment the rear, and now rear points to the third element.

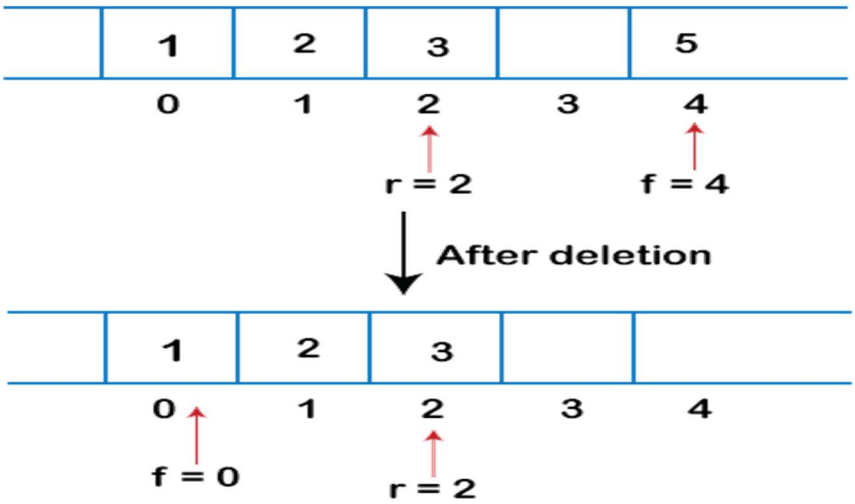



5.If we want to insert the element from the front end, and insert an element from the front, we have to decrement the value of front by 1. If we decrement the front by 1, then the front points to -1 location, which is not any valid location in an array. So, we set the front as **(n -1),** which is equal to 4 as n is 5. Once the front is set, we will insert the value as shown in the below figure:

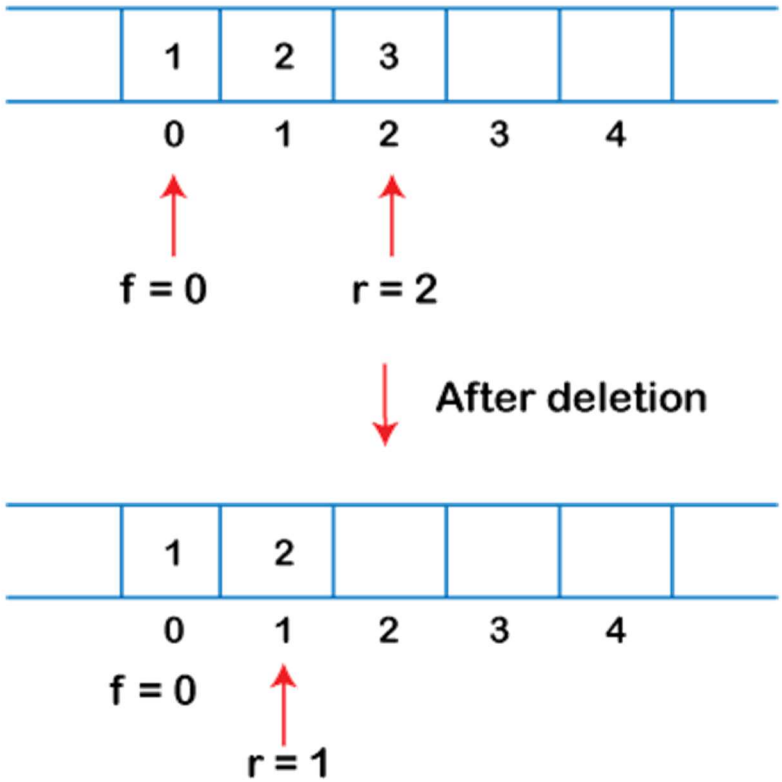



## Dequeue Operation

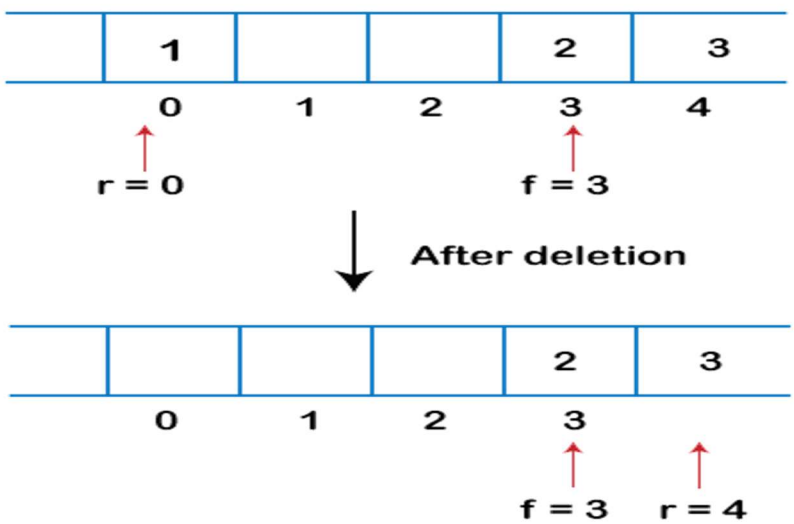
1. If the front is pointing to the last element of the array, and we want to perform the delete operation from the front. To delete any element from the front, we need to set **front=front+1**. Currently, the value of the front is equal to 4, and if we increment the value of front, it becomes 5 which is not a valid index. Therefore, we conclude that if front points to the last element, then front is set to 0 in case of delete operation.




If we want to delete the element from rear end then we need to decrement the rear value by 1, i.e., **rear=rear-1** as shown in the below figure:




If the rear is pointing to the first element, and we want to delete the element from the rear end then we have to set **rear=n-1** where **n** is the size of the array as shown in the below figure:



## Insertion in Double Ended Queue

## Algorithm

**ENQUEUE_REAR(queue,front,rear,item,MAXSIZE)**

**Step1:** **[**Is queue already full?**]**

```
If front = 0 and rear = MAXSIZE-1 OR front = rear+1 then
  Write "Queue overflow error"
  return
[End of if structure]
```

**Step 2:** **[**Finding value of rear**]**

```
If front = -1&&rear==-1 then

        Set front = 0 and rear = 0

Else

        Set rear = (rear+1)%MAXSIZE

[End of If structure]
```

**Step 3:** **[**Insert item at rear end of queue**]**

```
Set queue[rear]=item
```

**Step 4:** return

**ENQUEUE_FRONT(queue,front,rear,item,MAXSIZE)**

**Step1:** **[**Is queue already full?**]**

```
If front = 0 and rear = MAXSIZE-1 OR front = rear+1 then
  Write "Queue overflow error"
  return
[End of if structure]
```

**Step 2:** **[**Finding value of front**]**

```
If front = -1&&rear==-1 then

        Set front = 0 and rear = 0

Else
```

Set front = (front+MAXSIZE-1)%MAXSIZE

[End of If structure]

**Step 3:** **[**Insert item at front end of queue**]**
Set queue[front]=item
**Step 4:** return

## Deletion from Double Ended Queue

## Algorithm

**DEQUEUE_FRONT(queue,front,rear,MAXSIZE)**

**Step1:** **[**Is queue already empty?**]**
If front = -1&&rear==-1 then
Write "Queue underflow error"
return -1
[End of if structure]
**Step 2:** **[**Store front item of queue**]**

Set delitem = queue[front]

**Step 3:** **[**Find new value of front**]**
If front=rear then
Set front = -1 and rear = -1
Else
Set front = (front+1)%MAXSIZE
[End of If structure]
**Step 4:** return delitem

**DEQUEUE_REAR(queue,front,rear,MAXSIZE)**

**Step1:** **[**Is queue already empty?**]**
If front = -1&&rear==-1 then
Write "Queue underflow error"
return -1
[End of if structure]

**Step 2:** **[**Store rear item of queue**]**

Set delitem = queue[rear]

**Step 3:** **[**Find new value of rear**]**

If front=rear then

Set front = -1 and rear = -1

Else

Set rear = (rear+MAXSIZE-1)%MAXSIZE

[End of If structure]

**Step 4:** return delitem

```
/*A PROGRAM TO PERFORM OPERATIONS ON DOUBLE ENDED QUEUE*/
#include<stdio.h>
#include<stdlib.h>
#define MAXSIZE 5

int queue[MAXSIZE];
int front=-1,rear=-1;

int isFULL(void)
{
    if((front==0&&rear==MAXSIZE-1)||(front==rear+1))
        return 1;
    return 0;
}

int isEMPTY(void)
{
    if(front==-1&&rear==-1)
        return 1;
    return 0;
}

void ENQUEUE_REAR(int item)
{
    if(isFULL())
        printf("\nQueue Overflow error.");
```

```
    else
    {
        if(isEMPTY())
            front=rear=0;
        else
            rear=(rear+1)%MAXSIZE;

        queue[rear]=item;
        printf("\n%d is inserted in Queue",item);
    }
}

void ENQUEUE_FRONT(int item)
{
    if(isFULL())
        printf("\nQueue Overflow error.");
    else
    {
        if(isEMPTY())
            front=rear=0;
        else
            front=(front+MAXSIZE-1)%MAXSIZE;

        queue[front]=item;
        printf("\n%d is inserted in Queue",item);
    }
```

```
}

int DEQUEUE_FRONT(void)
{
    int delitem;

    if(isEMPTY())
    {
        printf("\nQueue underflow error.");
        return -1;
    }

    delitem=queue[front];
    if(front==rear)
        front=rear=-1;
    else
        front=(front+1)%MAXSIZE;

    return delitem;
}


int DEQUEUE_REAR(void)
{
    int delitem;
```

```
    if(isEMPTY())
    {
        printf("\nQueue underflow error.");
        return -1;
    }

    delitem=queue[rear];
    if(front==rear)
        front=rear=-1;
    else
        rear=(rear+MAXSIZE-1)%MAXSIZE;

    return delitem;
}




int PEEK_FRONT(void)
{
    if(isEMPTY())
    {
        printf("Queue underflow error.");
        return -1;
    }

    return queue[front];
```

```
}

int PEEK_REAR(void)
{
    if(isEMPTY())
    {
        printf("Queue underflow error.");
        return -1;
    }

    return queue[rear];
}

void DISPLAY(void)
{
    int i;

    if(isEMPTY())
    {
        printf("\nQueue is empty.");
        return;
    }

    printf("Items of Queue are\n");
    for(i=front;i!=rear;i=(i+1)%MAXSIZE)
        printf("%d\t",queue[i]);
```

```
    printf("%d",queue[i]);
}



int main()
{
    int item,choice;

    while(1)
    {
        printf("\n\t\t\t_ _ _ _ OPERATIONS ON DEQUE _ _ _ _\n");
        printf("\n\n\t\t\t1. Insertion at Rear end");
        printf("\n\n\t\t\t2. Insertion at Front end");
        printf("\n\n\t\t\t3. Deletion from Front end");
        printf("\n\n\t\t\t4. Deletion from Rear end");
        printf("\n\n\t\t\t5. PEEK(Front)");
        printf("\n\n\t\t\t6. PEEK(Rear)");
        printf("\n\n\t\t\t7. DISPLAY");
        printf("\n\n\t\t\t8. EXIT");
        printf("\n\nEnter your choice:");
        scanf("%d",&choice);

        switch(choice)
        {
        case 1:
            printf("\nEnter the item:");
```

```
    scanf("%d",&item);
    ENQUEUE_REAR(item);
break;

case 2:
    printf("\nEnter the item:");
    scanf("%d",&item);
    ENQUEUE_FRONT(item);
break;

case 3:
    item=DEQUEUE_FRONT();
    if(item!=-1)
        printf("\n%d is deleted from Queue",item);
break;

case 4:
    item=DEQUEUE_REAR();
     if(item!=-1)
        printf("\n%d is deleted from Queue",item);
break;

case 5:
    item=PEEK_FRONT();
     if(item!=-1)
        printf("\nItem at front of Queue = %d",item);
```

```
        break;
        case 6:
            item=PEEK_REAR();
             if(item!=-1)
                printf("\nItem at rear of Queue = %d",item);
        break;
        case 7:
            DISPLAY();
        break;

        case 8:
            printf("\n\t\t***GOOD BYE***");
            exit(0);

        default:
            printf("\nWrong choice.");
        }
    }
    return 0;
}
```

# Priority Queue

A priority queue is a data structure in which each element is assigned a priority. The priority of the element will be used to determine the order in which the elements will be processed. The general rules of processing the elements of a priority queue are

- An element with higher priority is processed before an element with a lower priority.
- Two elements with the same priority are processed on a first-come-first-served (FCFS) basis.

In real word example when a VIP comes for a service he will get the service before the other people standing in a Queue because he has the priority over other people.

# Types of Priority Queue

There are two types of priority queue-

## Ascending order priority queue

In an ascending priority queue elements can be inserted in any order. But, while deleting an element from the queue, only the smallest element is removed first..

## Descending order priority queue

In descending priority queue also elements can be inserted in any order. But, while deleting an element from the queue, only the largest element is deleted first.

## Insertion in Priority Queue

### Algorithm

**ENQUEUE(queue,front,rear,item,MAXSIZE)**

**Step1:** **[**Is queue already full?**]**

```
If rear=MAXSIZE-1 then
  Write "Queue overflow error"
  return
[End of if structure]
```

**Step 2:** **[**Finding value of rear and insert item at correct position**]**

```
If front = -1 and rear = -1 then
        Set front = 0 and rear = 0
        queue[rear] = item
Else
        Set j = rear
        Repeat while j>=front and item<queue[j]
                Set queue[j+1] = queue[j]
                Set j = j-1
        Set queue[j+1]=item
        Set rear = rear+1
[End of If structure]
```

**Step 3:** return

## Deletion from Priority Queue

## Algorithm

**DEQUEUE(queue,front,rear)**

**Step1:** **[**Is queue already empty?**]**

```
If front = -1 and rear = -1 then
  Write "Queue underflow error"
  return
[End of if structure]
```

**Step 2:** **[**Store front item of queue**]**

```
Set delitem = queue[front]
```

**Step 3:** **[**Find new value of front**]**

```
If front=rear then
 Set front = -1 and rear = -1
```

Else
Set front = front+1
[End of If structure]

**Step 4:** return delitem

```
/*A PROGRAM TO PERFORM OPERATIONS ON PRIORITY QUEUE*/
#include<stdio.h>
#include<stdlib.h>
#define MAXSIZE 5

int queue[MAXSIZE];
int front=-1,rear=-1;

int isFULL(void)
{
   if(rear==MAXSIZE-1)
      return 1;
   return 0;
}

int isEMPTY(void)
{
   if(front==-1&&rear==-1)
      return 1;
   return 0;
```

```
}



void ENQUEUE(int item)
{
    int j;
    if(isFULL())
        printf("\nQueue Overflow error.");
    else
    {
        if(isEMPTY())
        {
            front=rear=0;
            queue[rear]=item;
        }
        else
        {

            j=rear;
            while(j>=front&&item<queue[j])
            {
                queue[j+1]=queue[j];
                j=j-1;
            }
            queue[j+1]=item;
            rear=rear+1;
```

```
        }

        printf("\n%d is inserted in Queue",item);
    }

}

int DEQUEUE(void)
{
    int delitem;

    if(isEMPTY())
    {
        printf("Queue underflow error.");
        return -1;
    }

    delitem=queue[front];
    if(front==rear)
        front=rear=-1;
    else
        front=front+1;

    return delitem;
}
```

```
int PEEK(void)
{
    if(isEMPTY())
    {
        printf("Queue underflow error.");
        return -1;
    }
    return queue[front];
}

void DISPLAY(void)
{
    int i;

    if(isEMPTY())
        printf("Queue is empty.");
    else
    {
        printf("Items of Queue are\n");
        for(i=front;i<=rear;i++)
            printf("%d\t",queue[i]);
    }
}


int main()
```

```
{
    int item,choice;

    while(1)
    {

        printf("\n\t\t\t_ _ _ _ OPERATIONS ON PRIORITY QUEUE _ _ _ _\n");
        printf("\n\n\t\t\t1. INSERTION");
        printf("\n\n\t\t\t2. DELETION");
        printf("\n\n\t\t\t3. PEEK");
        printf("\n\n\t\t\t4. DISPLAY");
        printf("\n\n\t\t\t5. EXIT");
        printf("\n\nEnter your choice:");
        scanf("%d",&choice);

        switch(choice)
        {
        case 1:
            printf("\nEnter the item:");
            scanf("%d",&item);
            ENQUEUE(item);
            break;

        case 2:
            item=DEQUEUE();
            if(item!=-1)
```

```
            printf("\n%d is deleted from Queue",item);
        break;

    case 3:
        item=PEEK();
        if(item!=-1)
            printf("\nItem at Front of Queue = %d",item);
        break;

    case 4:
        DISPLAY();
    break;
    case 5:
        printf("\n\n\t\t\t_ _ _ Good Bye _ _ _");
        exit(0);

    default:
        printf("\nWrong choice.");
    }
  }
  return 0;
}
```

## Queue using linked lists

We know that queue is a special type of data structure where elements are inserted at one end and elements are deleted at the other end. It is a FIFO data structure.That is, if an element is inserted at front end, an element has to be deleted from rear end. If an element is inserted at rear end, an element has to be deleted from the front end. Thus, a queue can be implemented using the following functions:

- INS_BEG()
- DEL_LAST()
- DISPLAY()

OR

- INS_LAST()
- DEL_BEG()
- DISPLAY()

/*A PROGRAM TO PERFORM OPERATIONS ON QUEUE USING LINKED LIST*/

```
#include<stdio.h>
#include<stdlib.h>

struct node
{
   int data;
   struct node *next;
};
struct node *front=NULL,*rear=NULL;


void ENQUEUE(int item)
{
   struct node *node;
   node=(struct node*)malloc(sizeof(struct node));
   if(node==NULL)
      printf("\nQueue overflow error");
   else
   {
      node->data=item;
      node->next=NULL;

      if(front==NULL)
         front=rear=node;
```

```
        else
        {
            rear->next=node;
            rear=node;
        }
        printf("\n%d is inserted in Queue",item);
    }

}

int DEQUEUE(void)
{
    int delitem;
    struct node *q;
    if(front==NULL)
    {
     printf("\nQueue underflow error");
     return -1;
    }

    delitem=front->data;
    q=front;
    if(front==rear)
        front=rear=NULL;
    else
        front=front->next;

    free(q);
    return delitem;

}

int PEEK(void)
{
    if(front==NULL)
    {
         printf("Queue underflow error.");
         return -1;

    }

    return front->data;
}

void DISPLAY()
```

```
{
   struct node *q;
   if(front==NULL)
      printf("\nQueue is empty");
   else
   {
      printf("\nItems of queue are\n");
      for(q=front;q!=NULL;q=q->next)
      printf("%d\t",q->data);
   }
}

int main()
{
int choice,item;

while(1)
   {
      printf("\n\t\t\t*****OPERATION ON QUEUE*****\n\n\t\t\t\t");
      printf("1.INSERTION\n\n\t\t\t\t");
      printf("2.DELETION\n\n\t\t\t\t");
      printf("3.PEEK\n\n\t\t\t\t");
      printf("4.DISPLAY\n\n\t\t\t\t");
      printf("5.EXIT\n\n");
      printf("Enter your choice:");
      scanf("%d",&choice);
      switch(choice)
      {
      case 1:
      printf("\nEnter the item:");
      scanf("%d",&item);
      ENQUEUE(item);
      break;
      case 2:
      item=DEQUEUE();
      if(item!=-1)
         printf("\n%d is deleted from Queue",item);
      break;
      case 3:
      item=PEEK();
      if(item!=-1)
         printf("\nItem at front of Queue = %d",item);
         break;
      case 4:
      DISPLAY();
```

```
        break;
        case 5:
        exit(0);
        default:
        printf("\n\nYou entered a wrong choice.");
        }
    }
return 0;
}
```

# Tree

A tree is a non-linear data structure and consists of a non-empty finite collection of nodes and edges such that-

a) There is a distinguished node called the root of the tree.
b) The remaining nodes of the tree form an ordered collection of 0 or more subtrees.

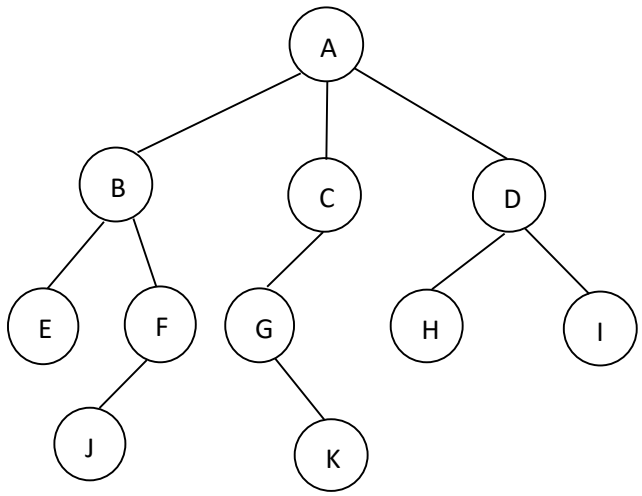



Figure: Tree

A tree is used to represent hierarchical (parent-child) relationship amongst nodes.

## Terminology used in tree

**a) Root**

First node in the hierarchical arrangement of nodes.

**OR**

Node with no predecessor

**OR**

Node with no parent

**Ex:** Node A

**b) Parent**

- Any node which has an edge directed downwards to the child node is known as parent node.
- Each parent can have one or more child node.

**Ex:** Node B is the parent of nodes E and F.

**c) Child**

- Any node which has an edge directed upwards to the parent node is known as child node.
- Each child node has a single parent node.

**Ex:** Nodes E and F are the child nodes of the node B.

**d) Sibling**

The nodes that have the same parent are known as siblings.

**Ex:** Nodes E and F are the sibling nodes.

**e) Leaf node(External node or Terminal node)**

A node with no successor

**OR**

A node that does not have any child node.

**Ex:** Nodes E,H,I,J and K are the leaf nodes.

**f) Branch node(Internal node)**

A node which has at least one child node.

**Ex:** Nodes A,B,C,D,F and G are internal nodes.

**g) Level**

The entire tree structure is levelled in such a way that the root node is always at level 0.
Then its immediate children are at level 1 and their immediate children are at level 2 and so on up to the terminal node.

**h) Ancestor of a node**

An ancestor of a node is any predecessor node on a path from the root to that node.
**Ex:** Nodes A,C and G are ancestors of node K.

**i) Descendent of a node**

The immediate successor of the given node is known as a descendant of a node.
**Ex:** K is the descendant of node G.

**j) Path**

It is a sequence of consecutive edges from the source node to the destination node.
**Ex:** The path between A and J is given by node pairs (A,B),(B,F),(F,J).

**k) Depth of a node**

It is the length of the path from the root to the node.
**Ex:** Depth of G is 2(A-C-G).

**l) Depth of a tree**

It is the maximum depth among all the nodes in the tree.

**m)** **Height of a node**

It is the length of the path from that node to the deepest node.

**Ex:** Height of B is 2(B-F-J).

**n)** **Height of a tree**

It is the maximum height among all the nodes in the tree.

**OR**

It is the maximum level of any node in the tree.

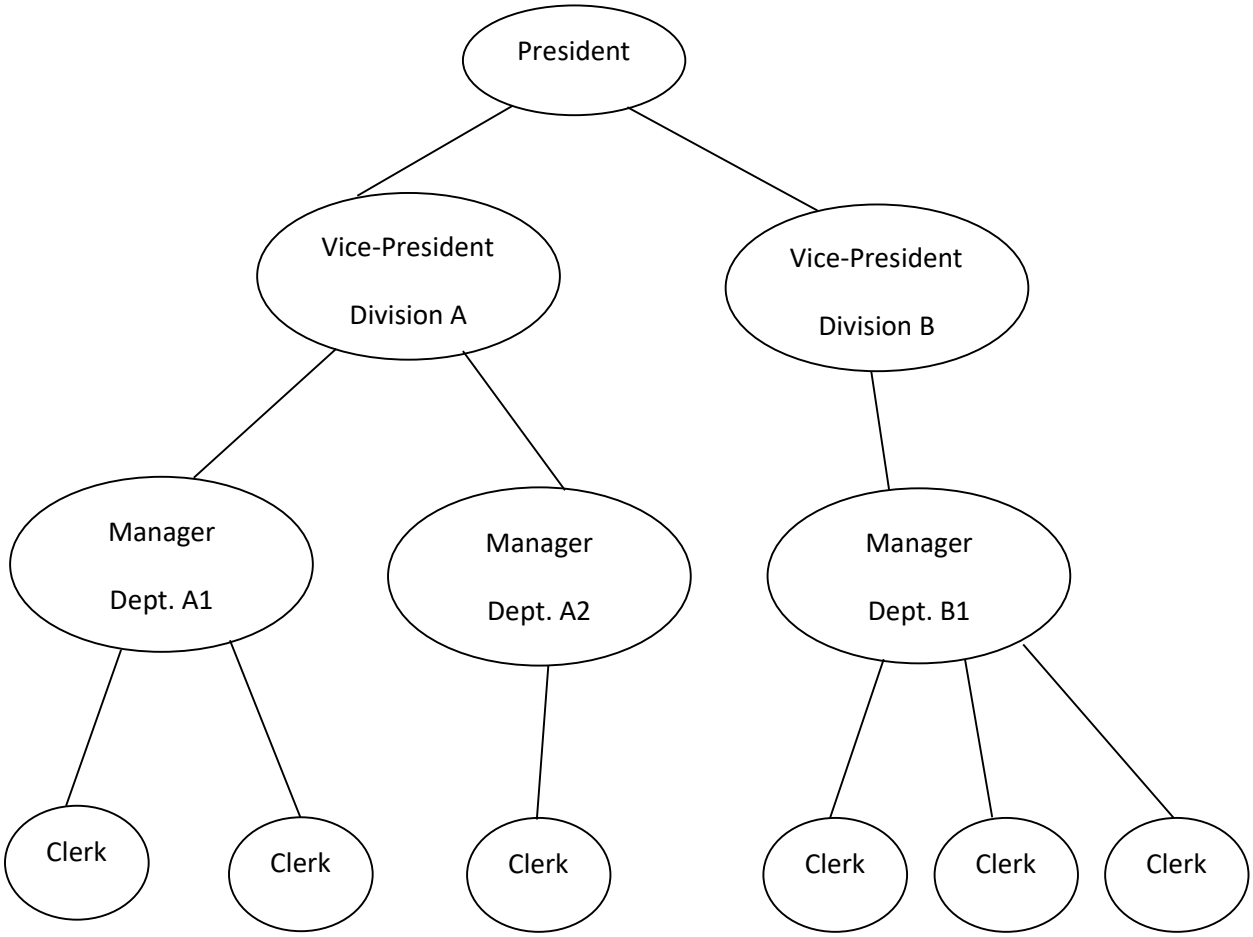
**Note:** For a given tree, depth and height returns the same value but for individual node, we may get different results.

**o)** **Forest**

Forest is a collection of n≥0 disjoint trees.

**Ex:** In the above tree, if we remove its root node then it becomes a forest with 3 trees.

A tree can be used to represent the lines of responsibilities in a business as shown in the figure.




We can use the above tree to determine the total salary paid to employees either by division or department.

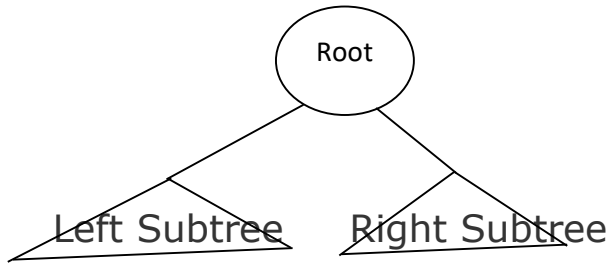
The total salary for the division A can be found by computing sum of the salary paid in department A1 and A2 plus the salary of vice-president of division A.

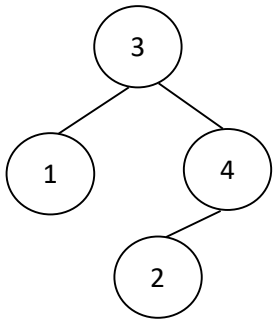
Similarly, the total salary paid in department A1 is the sum of salary of the manager of department A1 and of the 2 clerks.

## Binary Tree

A tree is called binary tree if each node has at most 2 children. In other words, a node can have 0,1 or 2 children. Thus, we can say that in a binary tree the maximum degree(the no. of subtrees) of any node is at most 2. That means, there may be a 0 degree node,1 degree node or 2 degree node.

We can visualize binary tree as consisting of a root and 2 binary trees, called the left and right subtree of the root.



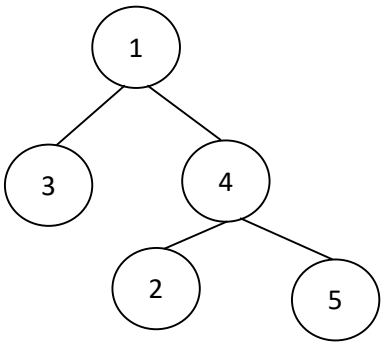




## Types of Binary Tree

### a) Strict Binary Tree

A binary tree is called strict binary tree if each node has exactly 0 or 2 children. In other words, degree of each node in a strict binary tree is either 0 or 2.

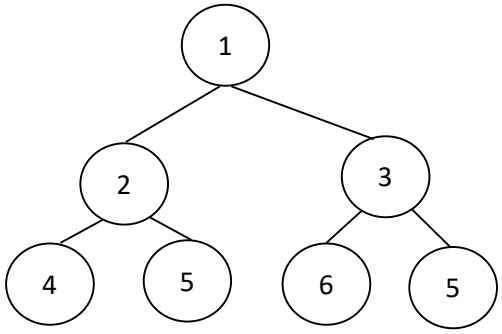
**Ex.**



**b) Full Binary Tree**

A binary tree is called full binary tree if each node has exactly 2 children and all leaf nodes are at the same level.

**Ex.**

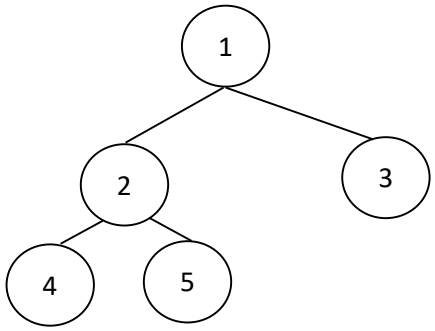



- The no. of nodes n in a full binary tree are $2^{h+1}-1$.
- The no. of leaf nodes in a full binary tree are $2^{h}$.

**c) Complete Binary Tree**

A binary tree is said to be a Complete binary tree if all the levels are completely filled except possibly the last level; and the last level has all the keys towards left.

**Ex.**




- The no. of nodes n in a complete binary tree is between 2(minimum) and $2^{h+1}-1$(maximum).

- The no. of NULL links(wasted pointers) in a complete binary tree of n nodes are n+1.

## Representation of Binary Tree

There are two types of representation of binary tree-

**(i)** Array Representation
**(ii)** Linked List Representation

## Array Representation

An array can be used to store the nodes of a binary tree. The root node is always at index 0. Then in successive memory locations the left child and the right child are stored.

**Consider the following binary tree.**

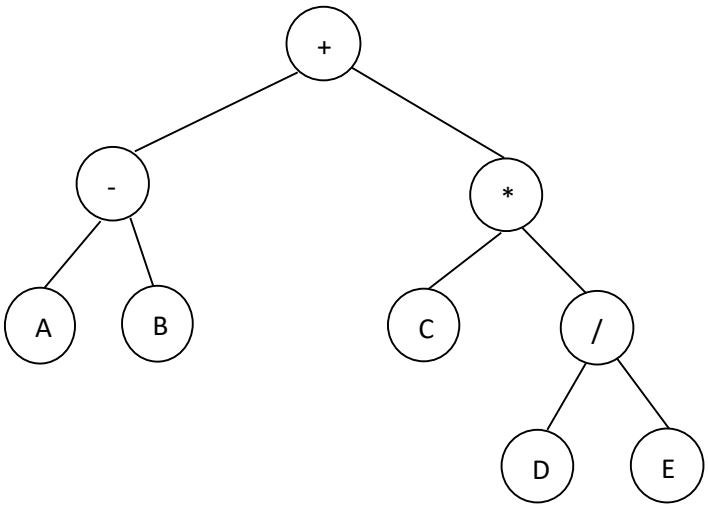



**Fig: Binary Tree**

The array representation of above binary tree is

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| + | - | * | A | B | C | / | | | | | | | D | E |

The following rules are used to decide the location of any node in the array-

(a) The root node is at index 0.
(b) For any node with index i
    (i) Parent(i) = floor((i-1)/2)
    (ii) Leftchild(i) = 2*i+1
    (iii) Rightchild(i) = 2*i+2

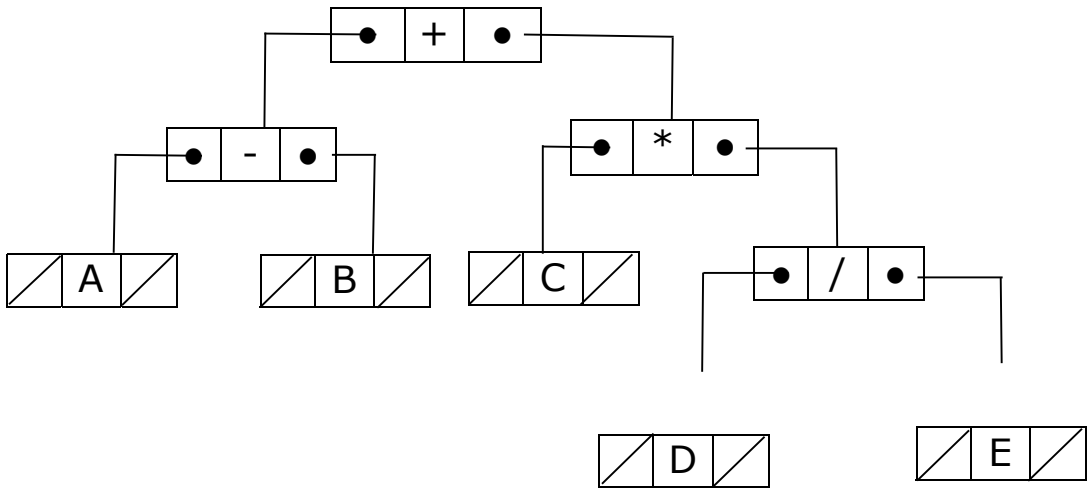
## Linked List Representation

In this representation, each node of linked list is divided into 3 parts as follows

| *lchild | data | *rchild |
|---|---|---|

Where

- data part contains the information
- *lchild contains the address of left children
- *rchild contains the address of right children




## Advantage of array representation of binary tree

**(a)** Node can be accessed directly with the help of index.
**(b)** Data are stored without pointer reference of successor or predecessor.
**(c)** This representation is very much useful where the language does not support dynamic memory allocation

## Disadvantages of array representation of binary tree

**(a)** Memory is wasted because it will be allocated for all the nodes. Absence of node will lead to empty entry in array.
**(b)** As the array size is limited, enhancement of tree structure is restricted.
**(c)** Inefficiency in processing time during insertion or deletion of node because considerable movement of up and down in array structure.

## Traversal of Binary Tree

Traversing is a method of visiting each node of a tree exactly once in a systematic order.

The various traversal techniques of a binary tree shown below-

**(a) Preorder Traversal(Root->Left->Right)**

- Visit the root node(Root)
- Traverse the left subtree in preorder(Left)
- Traverse the right subtree in postorder(Right)

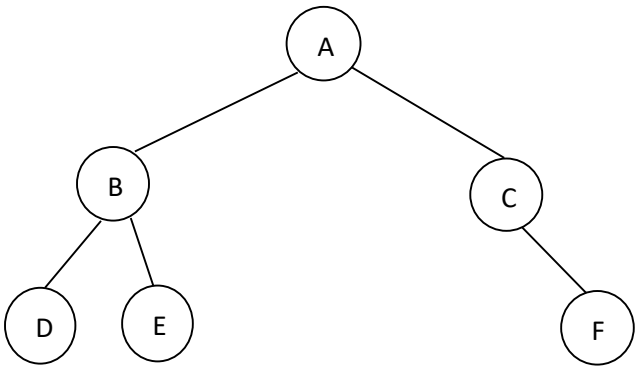
**(b) Inorder Traversal(Left->Root->Right)**

- Traverse the left subtree in inorder(Left)
- Visit the root node(Root)
- Traverse the right subtree in inorder(Right)

**(c) Postorder Traversal(Left->Right->Root)**

- Traverse the left subtree in postorder(Left)
- Traverse the right subtree in postorder(Right)
- Visit the root node.(Root)

Consider the following example-



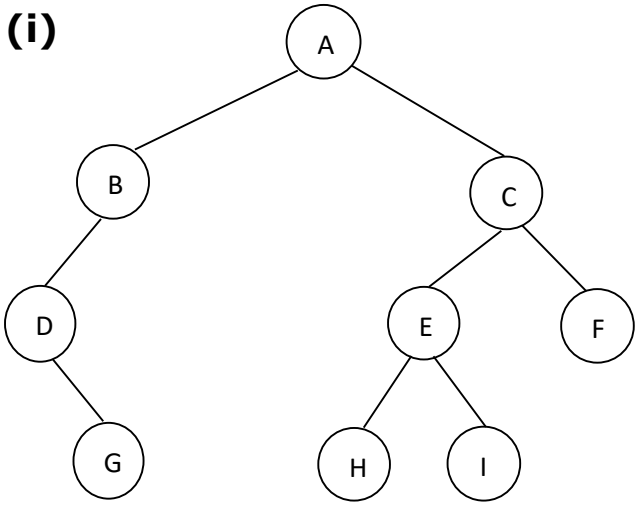

**Preorder Traversal:** A B D E C F

**Inorder Traversal:** D B E A C F

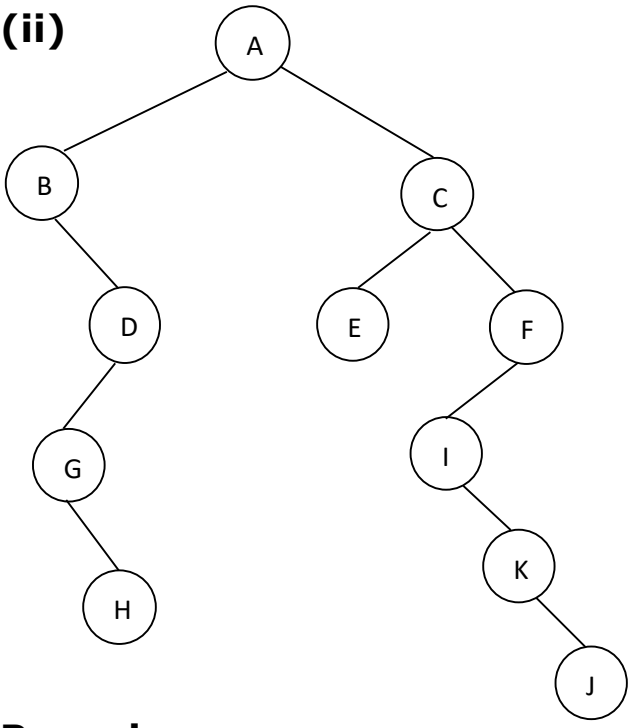
**Postorder Traversal:** D E B F C A

**Q.** Consider the following binary trees and find their preorder,inorder and postorder traversal-

**(i)**




**(ii)**




**Preorder :**

**Inorder :**

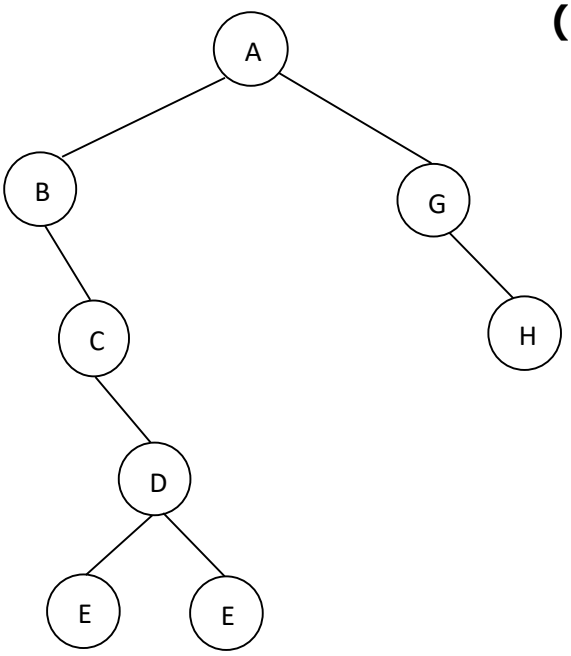
**Postorder :**

**Preorder :**

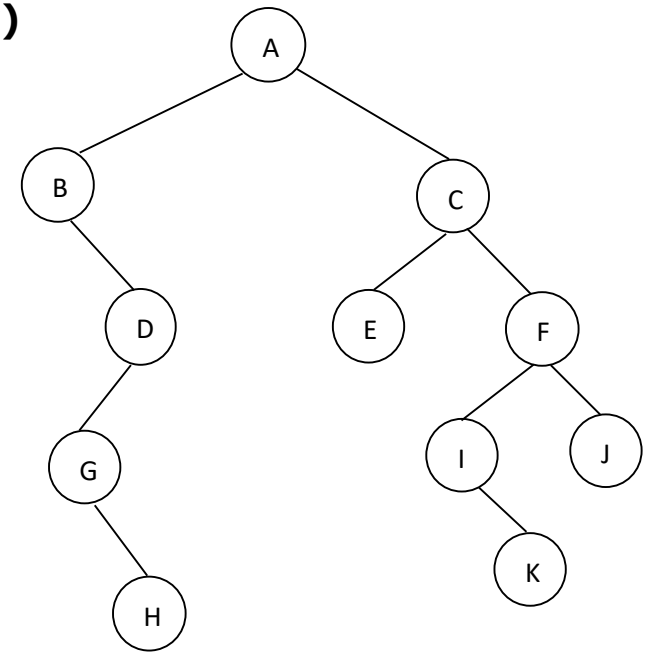
**Inorder :**

**Postorder :**

**(iii)**




**(iv)**

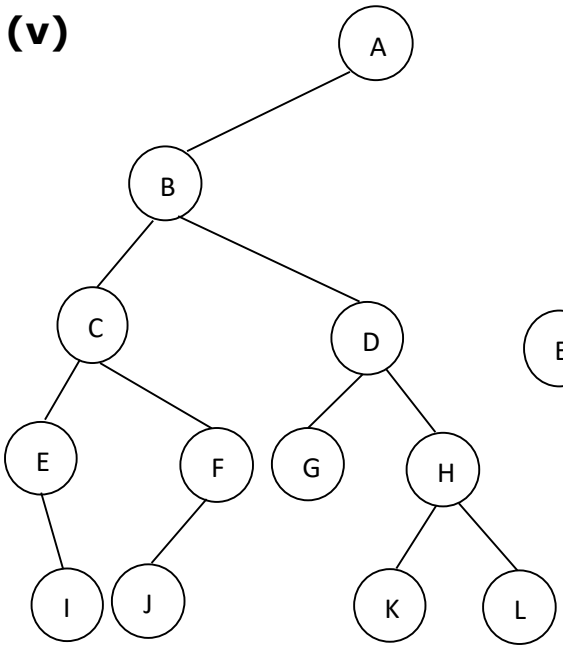


**Preorder :** **Preorder :**

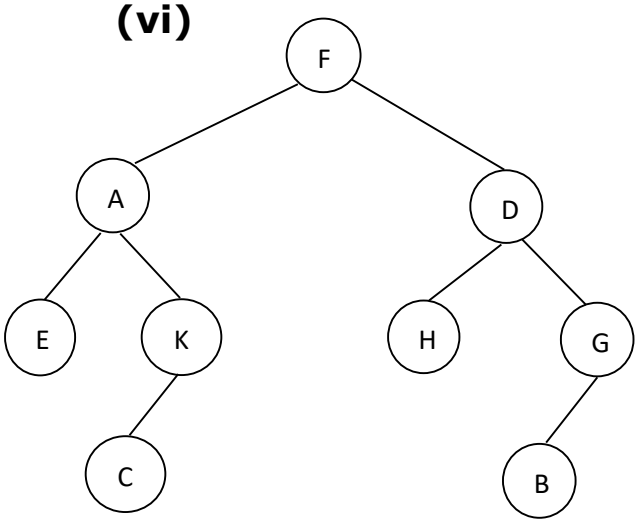
**Inorder :** **Inorder :**

**Postorder :** **Postorder :**

**(v)**




**(vi)**




**Preorder :** **Preorder :**

**Inorder :** **Inorder :**

**Postorder :** **Postorder :**

**Q.** The following sequence gives the preorder and inorder traversal of the binary tree-

**Preorder :** ABDGHCEFIKJ

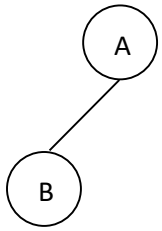
**Inorder :** BGHDAECIKFJ

Draw the diagram of the tree.

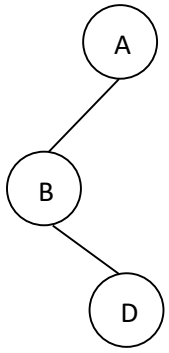
**Sol:** From the preorder traversal, we come to know that A is the root node. From inorder traversal, we find that the nodes B,G,H and D should come in the left subtree of A and the nodes E,C,I,K,F and J should come in right subtree of A.

**Step 1:** Start with the root node.

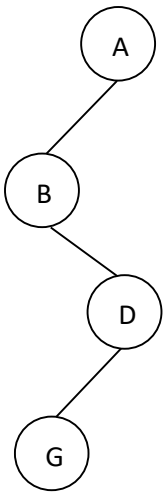
**Step 2:** In the preorder traversal, next node is B. From inorder traversal, we find that B should come on the left of A.



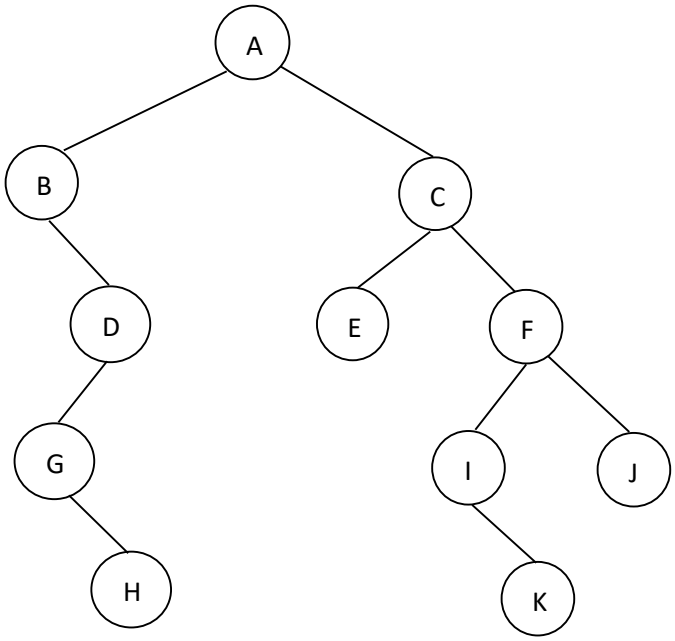

**Step 3:** Next node is D in preorder traversal. It should come on right of B.



**Step 4:** Next node is G in preorder traversal. It should come on left of D.




By Following the same process, we get the final binary tree-




**Q.** The following sequence gives the postorder and inorder traversal of the binary tree-

**Postorder** **:** HGDBEKIJFCA

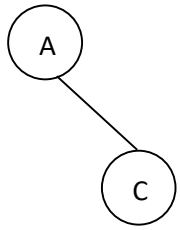
**Inorder** **:** BGHDAECIKFJ

Draw the diagram of the tree.

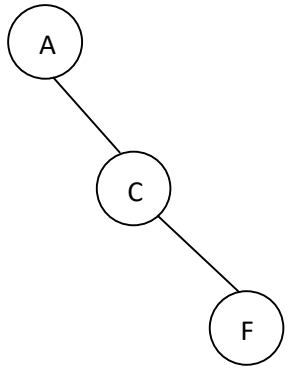
**Sol:** From the postorder traversal, we come to know that A is the root node. From inorder traversal, we find that the nodes B,G,H and D should come in the left subtree of A and the nodes E,C,I,K,F and J should come in right subtree of A.

**Step 1:** Start with the root node.

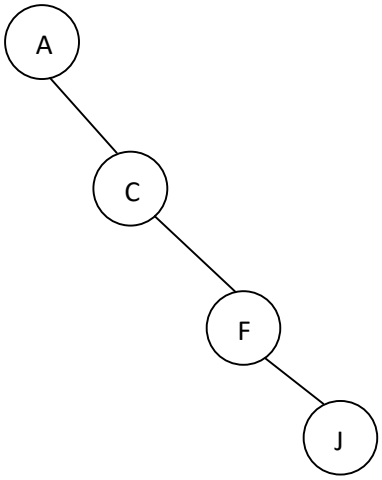
**Step 2:** In the postorder traversal, next node is C. From inorder traversal, we find that C should come on the right of A.



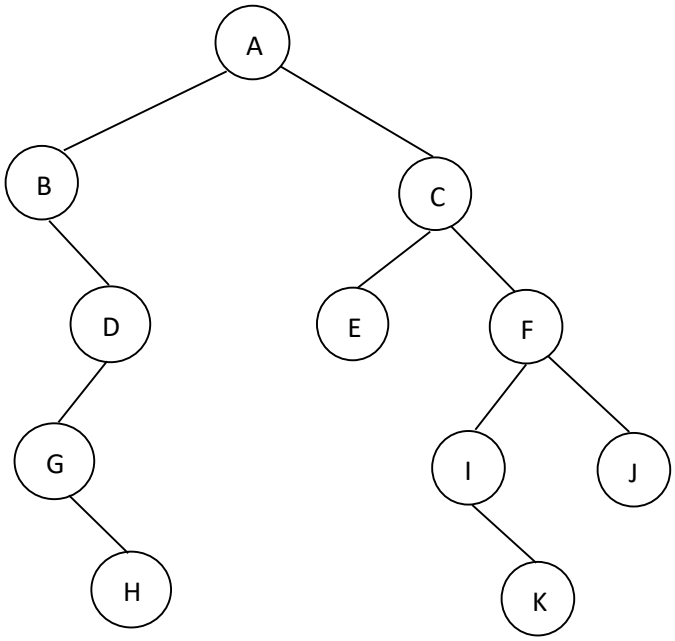

**Step 3:** Next node is F in postorder traversal. It should come on right of C.



**Step 4:** Next node is J in postorder traversal. It should come on right of F.




By Following the same process, we get the final binary tree-



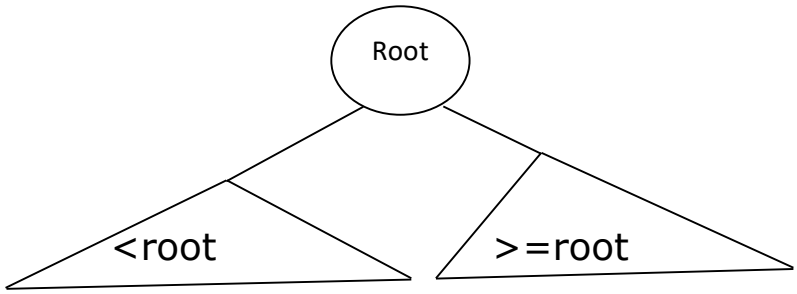
# Binary Search Tree(B.S.T)

In binary search tree, all the left subtree elements should be less than the root data and all the right subtree elements should be greater than or equal to root data.

This is called binary search tree property. This property should be satisfied at every node in the tree.

- The left subtree of a node contains only those nodes with data less than the root data.
- The right subtree of a node contains only those nodes with data greater than or equal to the root data.

  Both the left and right subtree must also be binary search tree.




**Ex.**

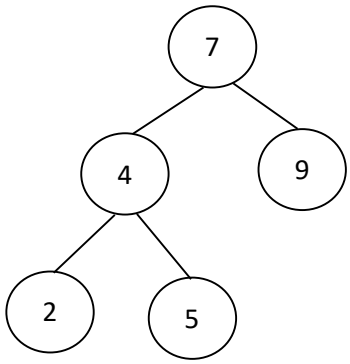



**Fig 1**

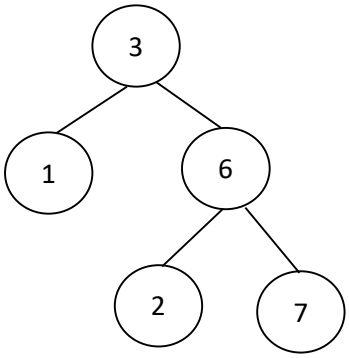



**Fig 2**

The binary tree in Fig 1 is B.S.T while the binary tree in Fig 2 is not a B.S.T.

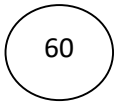
## Insertion in Binary Search Tree

**Q.** Insert the following keys(data) in binary search tree-

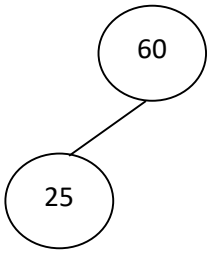
60,25,50,75,33,66,15,44

**Sol:**

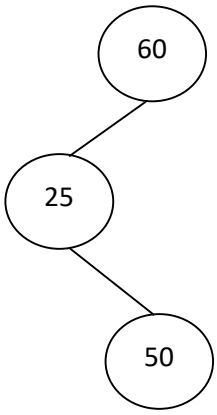
**Step 1:** We insert item = 60 as root node.



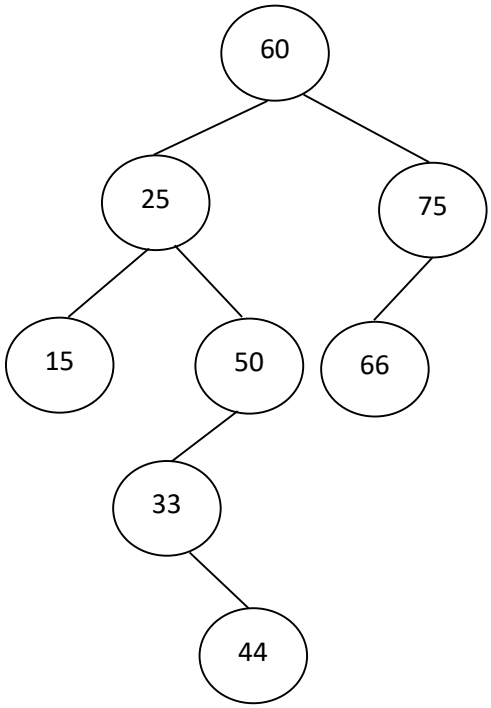

**Step 2:** Next item = 25. We compare 25 with root node 60 and find that 25 is less than 60 so we insert in on left of 60.




**Step 3:** Next item = 50. We compare 50 with root node 60 and find that 50 is less than 60. Again we compare 50 with 25 and find that 50 is greater than 25 so we insert it on right of 25.



By following the same process, we get the final B.S.T-




## Algorithm

**INSERT(root,item)**

**Step1:** **[**Insert a new node]

```
If root=NULL then
  root = (struct node*)malloc(sizeof(struct node))
  root->data = item
  root->left = NULL
  root->right = NULL
  return root
[End of if structure]
```

**Step 2:** **[**Finding the position to insert**]**

```
If item<root->data then

      Set root->left = CALL INSERT(root->left,item)

Else
```

Set root->right = CALL INSERT(root->right,item)

[End of If structure]

**Step 3:** return root

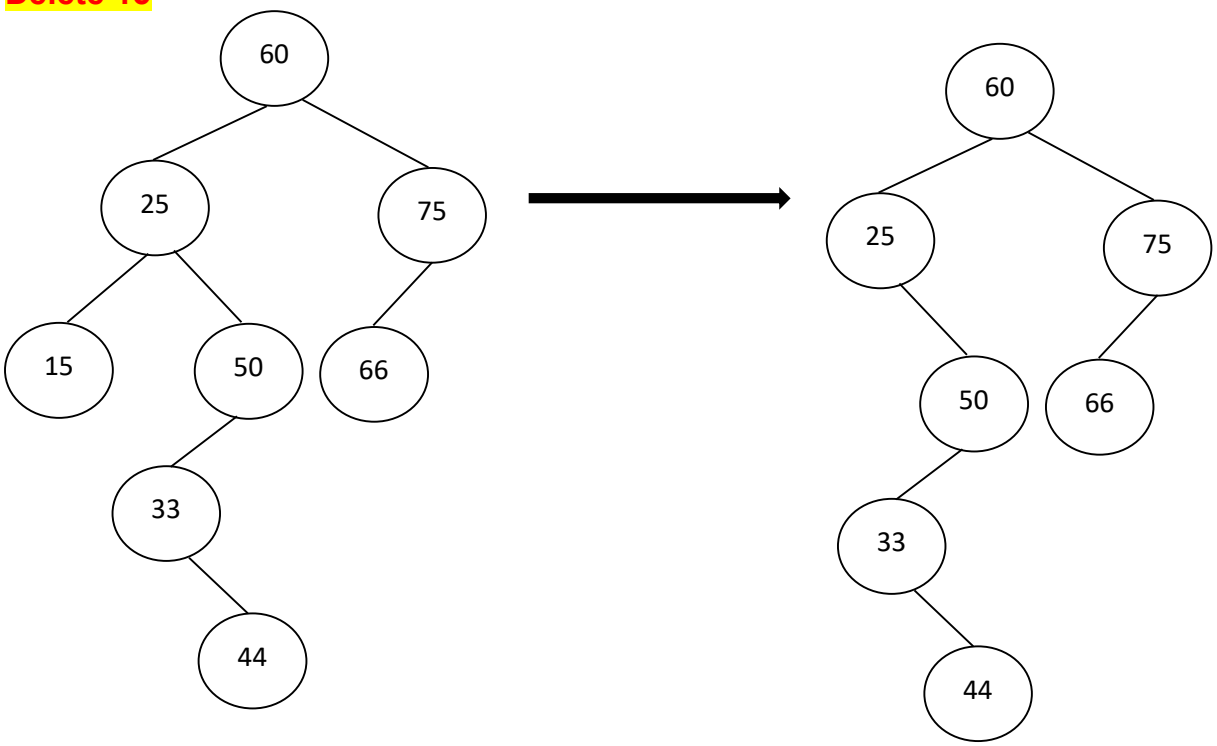
## Deletion in Binary Search Tree

It is very important to remember that once the node is deleted, the properties of binary search tree are not violated i.e elements in the left subtree should be lesser and elements in the right subtree should be greater or equal to node data.

There are 3 cases to delete a node from binary search tree-

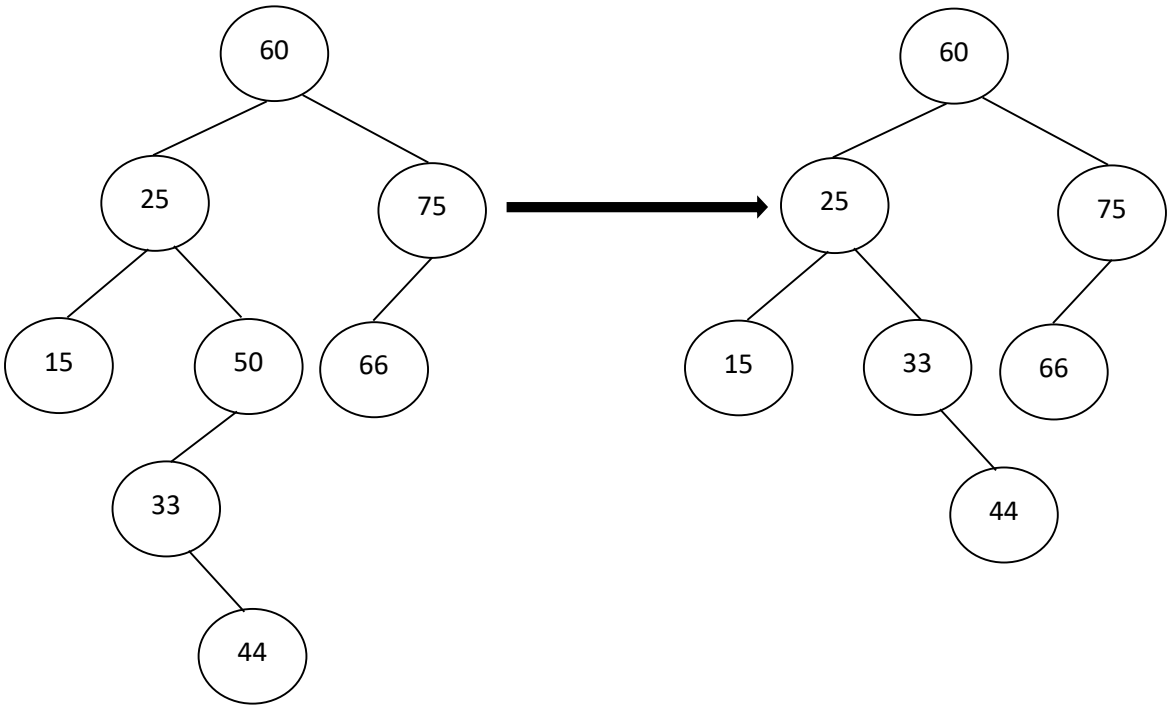
***Case 1: Deleting a Node that has No Children***

**Delete 15**



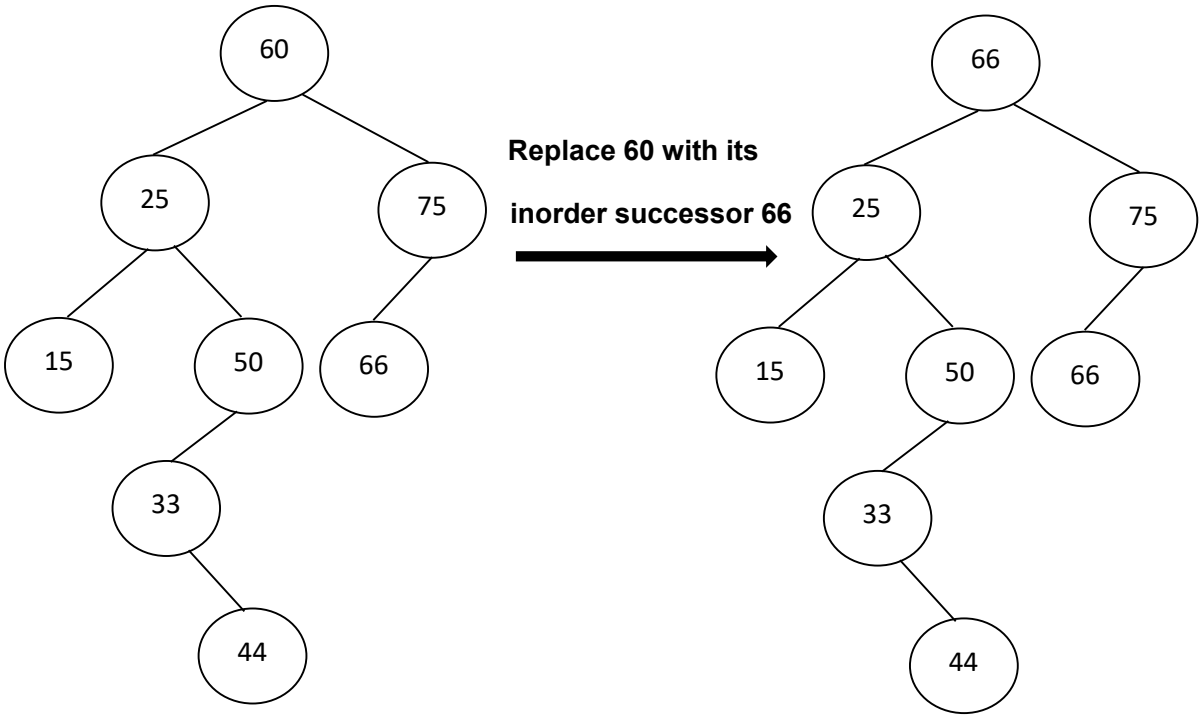
***Case 2: Deleting a Node that has one Children***

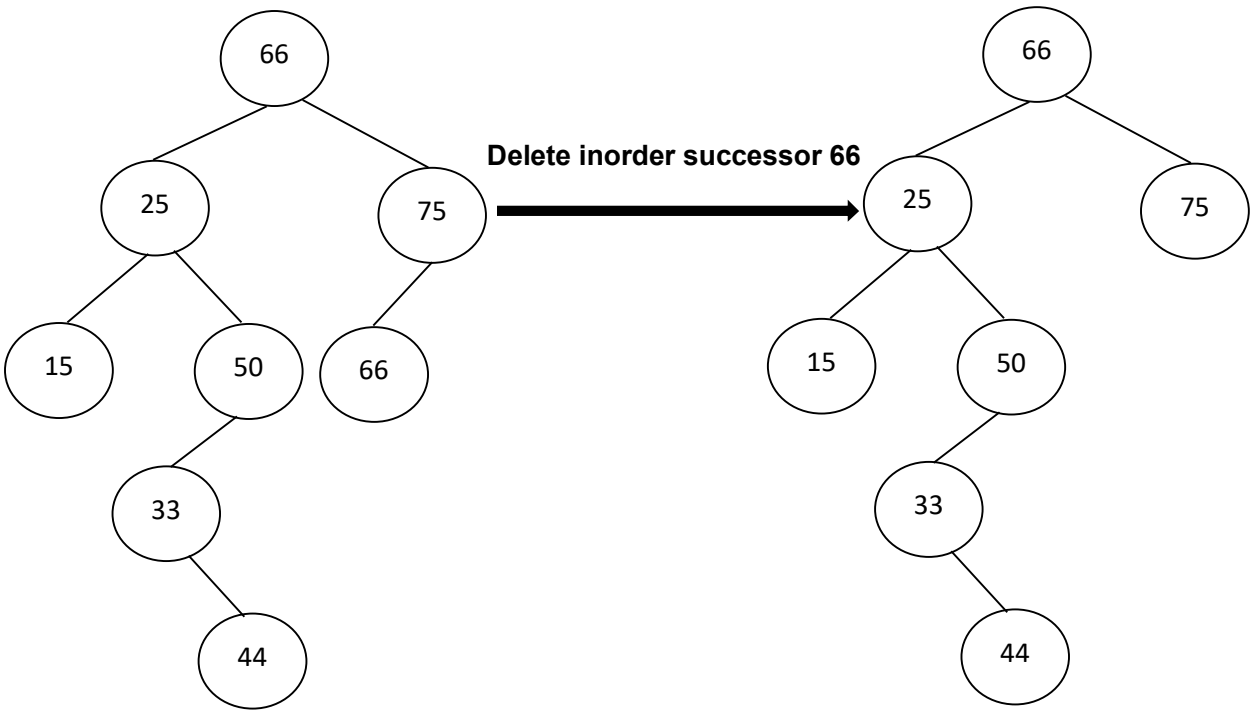
**Delete 50**




***Case 3: Deleting a Node that has two Children***

**Delete 60**






## Algorithm

**FINDSUCCESSOR(root)**

**Step1:** If root=NULL then
return NULL
[End of if structure]
**Step 2:** If root->left=NULL then

return root

[End of if structure]

**Step 3:** return FINDSUCCESSOR(root->left)

## Algorithm

**DELETE(root,item)**

```
Step1: If root=NULL then
          Write "Item not found"
          return root
       [End of if structure]
Step 2: If item<root->data then

            Set root->left = CALL DELETE(root->left,item)

       Else If item>root->data then

            Set root->right = CALL DELETE(root->right,item)

       Else

            Set temp = root

            If temp->left=NULL and temp->right=NULL then

                 Set root = NULL

            Else If temp->left=NULL then

                 Set root = root->right

            Else If temp->right = NULL then

                 Set root = root->left

            Else

                 Set temp = Call FINDSUCCESSOR(root->right)

                 Set root->data = temp->data

                 Set root->right = Call DELETE(root->right,temp->data)

            [End of If structure]

            free(temp)

      [End of If structure]

Step 3: return root
```

## Algorithm

**SEARCH(root,item)**

**Step1:** If root=NULL then
return NULL
[End of if structure]
**Step 2:** If item<root->data then

retrurn SEARCH(root->left,item)

Else If item>root->data then

retrurn SEARCH(root->right,item)

[End of If structure]

**Step 3:** return root

```
/*A program to perform insertion,deletion,searching and traversal operations on B.S.T(Binary
Search Tree)*/

#include<stdio.h>

#include<stdlib.h>


struct node

{

   int data;

   struct node *left;

   struct node *right;

};
```

```
struct node* INSERT(struct node *root,int item)
{
    if(root==NULL)
        {
            root=(struct node*)malloc(sizeof(struct node));
            root->data=item;
            root->left=NULL;
            root->right=NULL;
        }
    else if(item<root->data)
            root->left = INSERT(root->left,item);
    else
            root->right = INSERT(root->right,item);

    return root;
}

struct node* FINDSUCCESSOR(struct node *root)
{
    if(root==NULL)
        return root;
    if(root->left==NULL)
        return root;
    return FINDSUCCESSOR(root->left);
}
```

```
struct node* DELETE(struct node *root,int item)
{
    struct node *temp;
    if(root==NULL)
        printf("%d is not found",item);
    else
    {
        if(item<root->data)
            root->left= DELETE(root->left,item);
        else if(item>root->data)
            root->right= DELETE(root->right,item);
        else
        {
            temp=root;
            if(temp->left==NULL&&temp->right==NULL)
                root=NULL;
            else if(temp->left==NULL)
                root=temp->right;
            else if(temp->right==NULL)
                root=temp->left;
            else
            {
                temp=FINDSUCCESSOR(root->right);
```

```
            root->data=temp->data;
            root->right=DELETE(root->right,temp->data);
        }
        free(temp);
    }
  }
  return root;
}

//This code is little short
/*struct node* DELETE(struct node *root,int item)
{
struct node *temp;
if(root==NULL)
printf("%d is not found",item);
else
{
if(item<root->data)
root->left=DELETE(root->left,item);
else if(item>root->data)
root->right=DELETE(root->right,item);
else
{
temp=root;
if(temp->left==NULL)
root=temp->right;
else if(temp->right==NULL)
```

```
root=temp->left;
else
{
temp=FINDSUCCESSOR(root->right);
root->data=temp->data;
root->right=DELETE(root->right,temp->data);
}
free(temp);
}
}
return root;
}*/

struct node* SEARCH(struct node *root,int item)
{
    if(root==NULL)
        return root;
    if(item==root->data)
        return root;
    if(item<root->data)
        return SEARCH(root->left,item);

    return SEARCH(root->right,item);

    return root;
}
```

```
void PREORDER(struct node *root)
{
    if(root==NULL)
        return;
    printf("%d\t",root->data);
    PREORDER(root->left);
    PREORDER(root->right);
}

void INORDER(struct node *root)
{
    if(root==NULL)
        return;
    INORDER(root->left);
    printf("%d\t",root->data);
    INORDER(root->right);
}

void POSTORDER(struct node *root)
{
    if(root==NULL)
        return;
    POSTORDER(root->left);
    POSTORDER(root->right);
    printf("%d\t",root->data);
}
```

```
int main()
{
    struct node *root=NULL;
    struct node *node;
    int choice,item;

    while(1)
    {
        printf("\n\n\t\t***Operatons on Binary Search Tree***");
        printf("\n\t\t1. Insertion of node in BST");
        printf("\n\t\t2. Deletion of node from BST");
        printf("\n\t\t3. Display(preorder)the BST");
        printf("\n\t\t4. Display(inorder)the BST");
        printf("\n\t\t5. Display(postorder)the BST");
        printf("\n\t\t6. Searching");
        printf("\n\t\t7. Exit");

        printf("\n\nEnter your choice:");
        scanf("%d",&choice);

        switch(choice)
        {
            case 1:
            printf("\nEnter item:");
            scanf("%d",&item);
            root = INSERT(root,item);
```

```
break;

case 2:
printf("\nEnter item:");
scanf("%d",&item);
root = DELETE(root,item);
break;

case 3:
if(root==NULL)
    printf("Tree is empty");
else
    PREORDER(root);
break;

case 4:
if(root==NULL)
    printf("Tree is empty");
else
    INORDER(root);
break;

case 5:
if(root==NULL)
    printf("Tree is empty");
else
    POSTORDER(root);
```

```
            break;


            case 6:
            printf("\nEnter item to search:");
            scanf("%d",&item);
            node=SEARCH(root,item);
            if(node==NULL)
                printf("%d is not found",item);
            else
                printf("%d is found",item);
            break;


            case 7:
            printf("\n\t\t***Good bye***");
            exit(0);
            break;
            default:
            printf("\n\nWrong choice");
        }
    }
    return 0;
}
```

**/*FUNCTION TO FIND PREDECESSOR OF A NODE*/**

```
struct node* FINDPREDECESSOR(struct node *root)
{
            if(root==NULL)
            return root;
```

```
        if(root->right==NULL)
        return root;
        return getPredecessor(root->right);
}
/*FUNCTION TO COUNT NUMBER OF NODES*/
int COUNTNODE(struct node *root)
{
        static int count=0;
        if(root==NULL)
        return 0;
        count= COUNTNODE (root->left)+ COUNTNODE (root->right)+1;
        return count;
}
/*FUNCTION TO COUNT NUMBER OF LEAF NODES*/
int COUNTLEAFNODE(struct node *root)
{
        static int count=0;
        if(root==NULL)
        return 0;
        count= COUNTLEAFNODE (root->left)+ COUNTLEAFNODE (root->right);
        if(count==0)
        count++;
        return count;
}
/*FUNCTION TO GET LARGEST SUBTREE HEIGHT*/
int getMax(int lheight,int rheight)
{
```

```
if(lheight>rheight)
return lheight;
return rheight;
}
/*FUNCTION TO GET THE HEIGHT OF A TREE*/
int getHeight(struct node *root)
{
int height;
if(root==NULL)
return 0;
height=getMax(getHeight(root->left),getHeight(root->right))+1;
return height;
}
```

## AVL Tree